

Fabric Extensibility Toolkit documentation

Learn to use the Fabric Extensibility Toolkit to develop your own Fabric workload.

About the Extensibility Toolkit

OVERVIEW

[What is the Extensibility Toolkit?](#)

Get started

QUICKSTART

[Build your workload](#)

[QuickStart](#)

Tutorials

GET STARTED

[Add and use partner workloads](#)

[Manage and publish workloads](#)

Concepts

ARCHITECTURE

[Key concepts](#)

[Architecture](#)

APIs and samples

REFERENCE

[Starter Kit](#) 

[Samples](#) 

[REST APIs](#)

[Client-side APIs](#) 

Microsoft Fabric Extensibility Toolkit

The Extensibility Toolkit provides customers and partners with an easy way to extend Fabric to their needs. You can add more capabilities that feel, behave, and integrate like native Fabric components. The Extensibility Toolkit lets developers get started in minutes by using the [Starter-Kit](#). Local and cloud development environments are supported, reducing the entry barrier. The AI-enabled repository also allows rapid development and helps the developer along its journey to extend Fabric.

What is a workload?

A workload is a partner- or customer-built web application that integrates into the Fabric portal. The Fabric host loads your app according to its manifest, provides authentication tokens via Microsoft Entra ID, and exposes a host API to enable navigation, theming, notifications, and other platform interactions.

Key characteristics of workloads:

- **Hosted by you, rendered in Fabric via iFrame** - Your web application runs on your infrastructure but displays within Fabric
- **Manifest-driven** - Entry points, capabilities, and permissions are declared in a manifest file
- **Microsoft Entra authentication** - Scoped tokens provide secure access to resources
- **Fabric API integration** - Uses Fabric Public REST APIs for platform operations

ⓘ Note

The Extensibility Toolkit is the modern evolution of the [Workload Development Kit](#) designed to make extending Microsoft Fabric with custom functionality easier and faster than ever. It offers a streamlined development environment allowing developers to focus on innovation rather than infrastructure. This evolution reflects Microsoft's commitment to empowering developers with intuitive, scalable, and intelligent tools that accelerate productivity and unlock new possibilities within Fabric.

What the Extensibility Toolkit provides

- An easy way to extend Fabric with your own functionality
- A development environment that enables easy and fast development
- An SDK that abstracts the complexity from the developer
- A Starter-Kit that contains everything you need to get started within minutes

- AI-enabled development that helps you build your ideas
- Publish workloads to your Fabric tenant for everyone to use
- Publish workloads to all Fabric users

Who should use it

Organizations adopting Fabric: The toolkit is perfect for any organization that wants to extend Fabric to adapt it to their specific internal needs. Whether you're looking to integrate existing systems, create custom data processing workflows, or build specialized analytics tools for your teams, the Extensibility Toolkit enables you to seamlessly embed your solutions into Fabric workspaces. Developers can easily create data apps and UI experiences that feel native to Fabric, supporting scenarios like custom pipelines, specialized data stores, visualization apps, and operational tools that work with your Fabric items.

Solution builders for the Fabric ecosystem: If you're building solutions that you want to provide to the entire Fabric community through the Workload Hub, the toolkit offers a complete path to market. You can develop innovative workloads that solve common industry challenges, extend Fabric's capabilities for specific use cases, or create entirely new experiences that other organizations can discover, trial, and purchase. This represents a significant business opportunity to reach millions of Fabric users worldwide while building a sustainable revenue stream through the Workload Hub marketplace.

Items and native integration

Workloads can contribute one or more item types that appear in workspaces and participate in collaboration, sharing, search, lineage, and lifecycle operations. Data is stored in [OneLake](#) and metadata is managed via Fabric public APIs, ensuring items behave like any other Fabric artifact.

Examples of native participation:

- **Full CRUD operations** - Create, read, update, and delete items in Fabric portal and over API
- **Workspace integration** - Workspace ACLs and tenant governance automatically apply
- **Discoverability** - Items are discoverable via search and integrated in navigation
- **Data storage** - Leverage OneLake for data storage and management
- **CI/CD support** - Automatic participation in continuous integration and deployment workflows

Workload examples

Here are a few examples to help you understand potential applications of Fabric workloads:

- Data application: Workloads that bring together Fabric and non-Fabric capabilities to build a complete application.
- Data store: Workloads that manage and store data. They can provide APIs to query and write data. Examples include [Lakehouse](#) and [Azure Cosmos DB](#).
- Data visualization: Applications built on Fabric data items, such as [Power BI reports](#) and [dashboards](#).
- Fabric customization: Scenarios like provisioning preconfigured workspaces or adding admin functionality.

The [Extensibility Samples](#)  provides several item types that you can use out of the box or adapt to your needs:

- Package Installer: Installs predefined packages (items, data, job schedules) into new or existing workspaces.
- OneLake Editor: Opens and visualizes OneLake data for Fabric items, including items created via the Extensibility Toolkit.

When to use the Extensibility Toolkit

Use the Extensibility Toolkit when you want to bring a custom experience to Fabric while leveraging its identity, governance, storage, and APIs. Common scenarios include:

- **Domain-specific authoring experiences** - Build specialized tools for your industry or use case
- **Governance and compliance tooling** - Create custom governance workflows and compliance dashboards
- **System integrations** - Connect Fabric with your existing systems and workflows
- **Custom analytics and visualization** - Build specialized analytics tools that complement Power BI
- **Operational tools** - Create custom monitoring, management, and operational dashboards

Key considerations for developing a Fabric workload

There are several important concepts to understand before beginning development:

- Native Fabric experience: Review the [Fabric UX system](#) ; all published workloads must comply with these design principles.

- Integrate with the Fabric workspace: Your application must function in a [Fabric workspace](#), where users create instances and collaborate.
- Multitenant integration: Your workload is embedded in Fabric, but your code runs in your cloud. Fabric exposes APIs for user data and context so you can map between the customer's environment and your deployment.

Trademarks

The Microsoft Fabric Workload Development Kit might contain trademarks or logos for projects, products, or services. Authorized use of Microsoft trademarks or logos is subject to and must follow [Microsoft's Trademark & Brand Guidelines](#) [↗].

Use of Microsoft trademarks or logos in modified versions of this project must not cause confusion or imply Microsoft sponsorship. Any use of third-party trademarks or logos are subject to those third-party's policies.

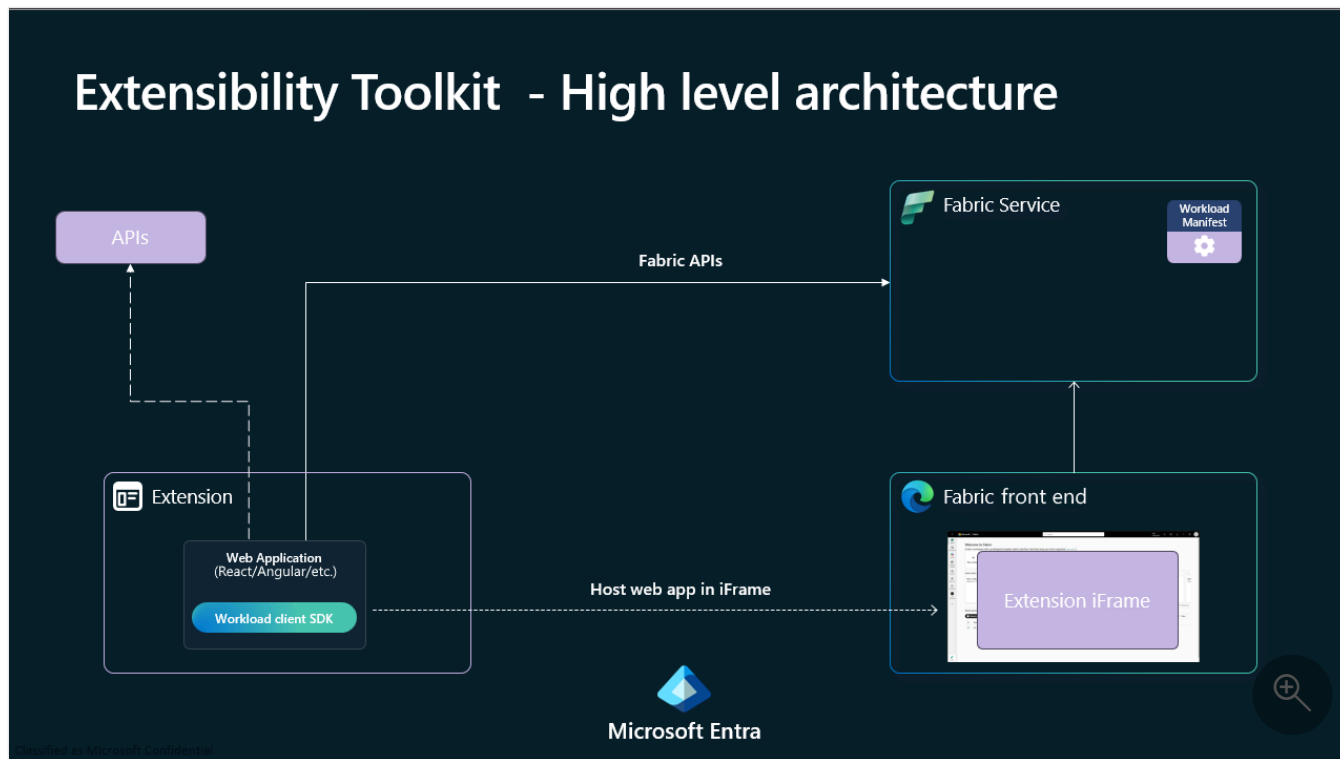
Related content

- [Architecture](#) - Understand the technical architecture
- [Key concepts and features](#) - Learn about core concepts
- [Manifest overview](#) - Understand workload manifests
- [Getting Started](#) - Start building your first workload
- [Publish a Fabric workload to Workload Hub](#) - Learn about publishing

Last updated on 12/15/2025

Extensibility Toolkit architecture

This article describes the architecture of the Microsoft Fabric Extensibility Toolkit and how custom workloads integrate with the Fabric platform. It explains the runtime components, flows between the Fabric frontend, your workload, and Fabric services, and how the manifest and item model make workloads feel native in Fabric.



Architectural components

Fabric frontend (host)

The Fabric frontend is the host environment. It renders your workload as an iFrame and exposes a secure host API to the iFrame so the workload can interact with Fabric while remaining isolated. The host is responsible for bootstrapping the workload according to its manifest (entry points, routes, and capabilities), managing authentication tokens via Microsoft Entra ID, and mediating calls from the workload to Fabric public APIs and platform services.

Workload web application

Your workload is a web application (for example, React or Angular) that you host in your cloud. Fabric loads it in an iFrame and provides host APIs to integrate with the platform. The app implements the routes and UI surfaces declared in its manifest, uses Microsoft Entra ID tokens (provided by the host) to call Fabric public APIs and, if needed, your own back-end services, and follows Fabric UX guidance so it looks and behaves like a native experience.

Fabric service and public APIs

The Fabric service exposes public APIs for reading and writing metadata and content, managing items, and integrating with platform capabilities. Workloads call these APIs using scoped tokens issued through Microsoft Entra ID—for example, to perform item CRUD operations and lifecycle actions, access data and content stored in [OneLake](#), and participate in workspace features such as search and discovery. For endpoints, scopes, and identity guidance, see the [Fabric Public REST APIs](#).

Microsoft Entra (authentication)

Authentication and authorization are handled by Microsoft Entra ID. The workload's manifest declares the permissions it needs; the Fabric host obtains the appropriate tokens and enforces consent and access according to those declarations.

Workload model and manifest

Workloads are defined by a manifest that describes their identity, capabilities, routes, UI entry points, and required permissions. The manifest is the contract between your web app and Fabric. For the schema, examples, and validation guidance, see [Manifest Overview](#).

Items and native participation

Workloads typically contribute one or more item types. Items created by your workload appear in workspaces and behave like native Fabric items. They participate in collaboration and sharing, are discoverable in search, follow lifecycle operations and governance, and store data via OneLake while using Fabric public APIs for CRUD.

End-to-end flow

1. A user opens a workspace and navigates to an item or entry point provided by your workload.
2. The Fabric frontend loads your web application in an iFrame based on the manifest.
3. The host acquires Microsoft Entra tokens with the scopes your workload requires and exposes a host API to the iFrame.
4. The workload calls Fabric public APIs (and, if applicable, your own services) using these tokens.
5. Item data is stored in OneLake and item metadata is managed through Fabric APIs so it behaves like any other Fabric item.

Next steps

To develop locally and publish your workload, see [Publish your workload](#). For manifest schema and local development guidance, see [Manifest overview](#), [DevServer](#), and [DevGateway](#). For API endpoints, scopes, and identity guidance, see the [Fabric Public REST APIs](#).

Related content

- [Microsoft Entra ID](#)
- [OneLake overview](#)
- [Fabric Client SDK](#) [↗](#)

Last updated on 12/15/2025

Key concepts and features

The Extensibility toolkit introduces a suite of capabilities designed to bring your Data Application to fabric as simple as possible. These features enable developers to build rich, integrated experiences with minimal effort. With the Extensibility Toolkit, you can easily access Fabric APIs directly from the frontend, persist the item definition (state) within Fabric, use a standardized item creation flow, and take advantage of improved security and interoperability through iFrame relaxation and public API support. Additionally, it streamlines your development lifecycle with built-in CI/CD support, making it easier to automate deployment and testing. The following sections provide an overview of the main functionality and guidance on how to incorporate them into your workloads.

Standard item creation experience

The item creation is standardized through a dedicated Fabric control that guides users through the process. This control allows users to select the workspace where the item is created, assign Sensitivity labels, and configure other relevant settings. By using this standardized experience, you no longer need to handle the complexities of item creation yourself or worry about future changes to the process. Additionally, this approach enables item creation to be surfaced directly within your workload page, providing a seamless and integrated user experience.

Use the [How to create an Item guide](#) to understand how it can be implemented.

Frontend API support

With the Extensibility Toolkit, you can obtain a Microsoft Entra On-Behalf-Of (OBO) token directly within your frontend application, enabling secure access to any Entra-protected API. This capability allows you to deeply integrate with Microsoft Fabric services—for example, you can read and store data in OneLake, create and interact with other Fabric items, or use Spark as a processing engine via the Lively APIs. For more information, see the [Microsoft Entra documentation](#), [OneLake documentation](#), [Fabric REST APIs](#), and [Spark in Fabric](#).

Use the [How to acquire Microsoft Entra Token guide](#) to understand how it can be implemented. Also use the [How to access Fabric APIs guide](#) to understand how you can interact with Fabric.

Storing item definition (state) in Fabric

This feature enables you to store your item's metadata—such as item configuration, and other relevant information—directly in OneLake within a hidden folder that isn't visible to end users.

The data is stored using the same format applied by public APIs and CI/CD processes, ensuring consistency and interoperability across different integration points. Details about the format and its use with public APIs and CI/CD is discussed in the following sections.

Use the [How to store Item definition](#) to understand how it can be implemented.

What to store in the definition

Think about the state as something that holds all the information needed to restore an item if it is deleted or copied elsewhere. This does not include the data itself, which is stored in OneLake (see [Storing Item Data in OneLake](#)).

Here are some practical examples:

- **Notebook item:** The state stores the notebook's code, cell order, and metadata such as which execution engine (e.g., Spark, SQL) should be used. The actual data processed by the notebook is not stored in the state.
- **File editor item:** The state does not store the file contents themselves, but instead stores the editor configuration—such as color scheme, auto-complete settings, font size, and other user preferences.
- **Installer or orchestrator item:** If your item installs or provisions other components (like databases or compute resources), the state should hold references (IDs, URIs) to the items it created. This allows your workload to check their state or manage them later.

By focusing on configuration, metadata, and references, you ensure your item's state is portable, lightweight, and easy to restore or migrate.

Storing item data in OneLake

Every item comes with its own Onelake item folder where developers can store structured as well as unstructured data. Similar to a [Lakehouse](#) the item has a Table folder where data can be stored in Delta or Iceberg format and a Files folder where unstructured data can be stored.

Use the [How to store data in Item](#) to understand how it can be implemented.

Shortcut data

As every item has, its own [Onelake](#) folder they can also work with Shortcuts. Over the Public [Shortcut API](#) workload developers can create different Shortcut types from or into their item to participate in the single copy promise from OneLake.

Use the [How to create shortcuts](#) to understand how it can be implemented.

CRUD item API support

Users can create, update, and delete items with content using the standard [Fabric Item Rest APIs](#). This automatic enablement makes it much easier to integrate with workload items in the same way as core Fabric items, streamlining interoperability and reducing the effort required to build robust integrations.

CI/CD support

ⓘ Note

CI/CD support for the Extensibility Toolkit is currently under development. The features described below are planned capabilities and may change before release.

CI/CD support for all items is one of the highest asks from customers. With this feature all items, participate in CI/CD out of the box, without the need to implement any specific logic or operations. This means you can automate deployment, testing, and updates for your workloads using standard Azure Pipelines and tools. The item format and APIs are designed to be fully compatible with CI/CD processes, ensuring a consistent and reliable experience across environments. For more information on integrating with CI/CD, see the [Fabric CI/CD documentation](#).

Item CRUD notification API

ⓘ Note

CRUD notification API support for the Extensibility Toolkit is currently under development. The features described below are planned capabilities and may change before release.

There are cases where your workload needs to participate in the Item CRUD events. As items are created on the platform directly via the [UX](#), [Public APIs](#) or [CI/CD](#) workload owners aren't in control when a new item is created over those entrypoints. By default items store their [state](#) in Fabric and don't need to get informed about the change of their item. Nevertheless there are some cases where workloads have a need to participate in the CRUD flow. This is mainly the case if Infrastructure for items needs to be provisioned or configured (for example, Databases). For these scenarios, we allow partners to implement a Crud notification API which Fabric calls on every event. In this scenario, Workload developer need to make sure that their API is reachable as otherwise Fabric operations fail.

Fabric scheduler

ⓘ Note

Fabric scheduler support for the Extensibility Toolkit is currently under development. The features described below are planned capabilities and may change before release.

Fabric supports Job scheduling for workloads. This feature allows developers to build workloads that get notified even if the user isn't in front of the UX and act based on the Job that should be executed (for example, copy data in Onelake). Partners need to implement an API and configure their workload to participate in this functionality.

iFrame relaxation

Developers can request more iFrame attributes to enable advanced scenarios such as file downloads or opening external websites. This feature allows your workload to prompt users for explicit consent before performing actions that require broader browser capabilities—such as initiating downloads or connecting users to external APIs using their current Fabric credentials. By specifying these requirements in your workload configuration, you ensure that users are informed and can grant the necessary permissions, enabling seamless integration with external systems while maintaining security and user trust.

Use the [How to relax the iFrame](#) to understand how it can be implemented.

ⓘ Note

Enabling this feature requires users to grant more Microsoft Entra consent for the relaxation scope, beyond the standard Fabric scope required for basic workload functionality.

Feature limitations

Private link

All workloads are blocked for consumption and development if Private Link is enabled on tenant or workspace level.

Tutorial: Get started with Fabric Extensibility Toolkit

Use this guide to get started building a Microsoft Fabric workload with the Extensibility Toolkit. You set up your environment, run the Starter-Kit in GitHub Codespaces or locally, and learn the basics you need to build and publish.

What you build

You run the [Starter-Kit](#) inside Fabric, explore its manifest and host API usage, make a small UI change, and understand how items your workload creates behave like native Fabric artifacts.

Prerequisites

- Access to a Fabric tenant and a workspace where you can create items
- A GitHub account (required for Codespaces)
- If developing locally: a recent Node.js LTS and a package manager (see the README for exact tooling)
- Access to create an [Microsoft Entra Application](#)
- Optional: an Azure subscription if you plan to host your web app in Azure

Tip

Local development is supported natively on Windows, macOS, and Linux. Before running setup scripts or starting the dev server, review and satisfy the prerequisites in the [Starter-Kit](#).

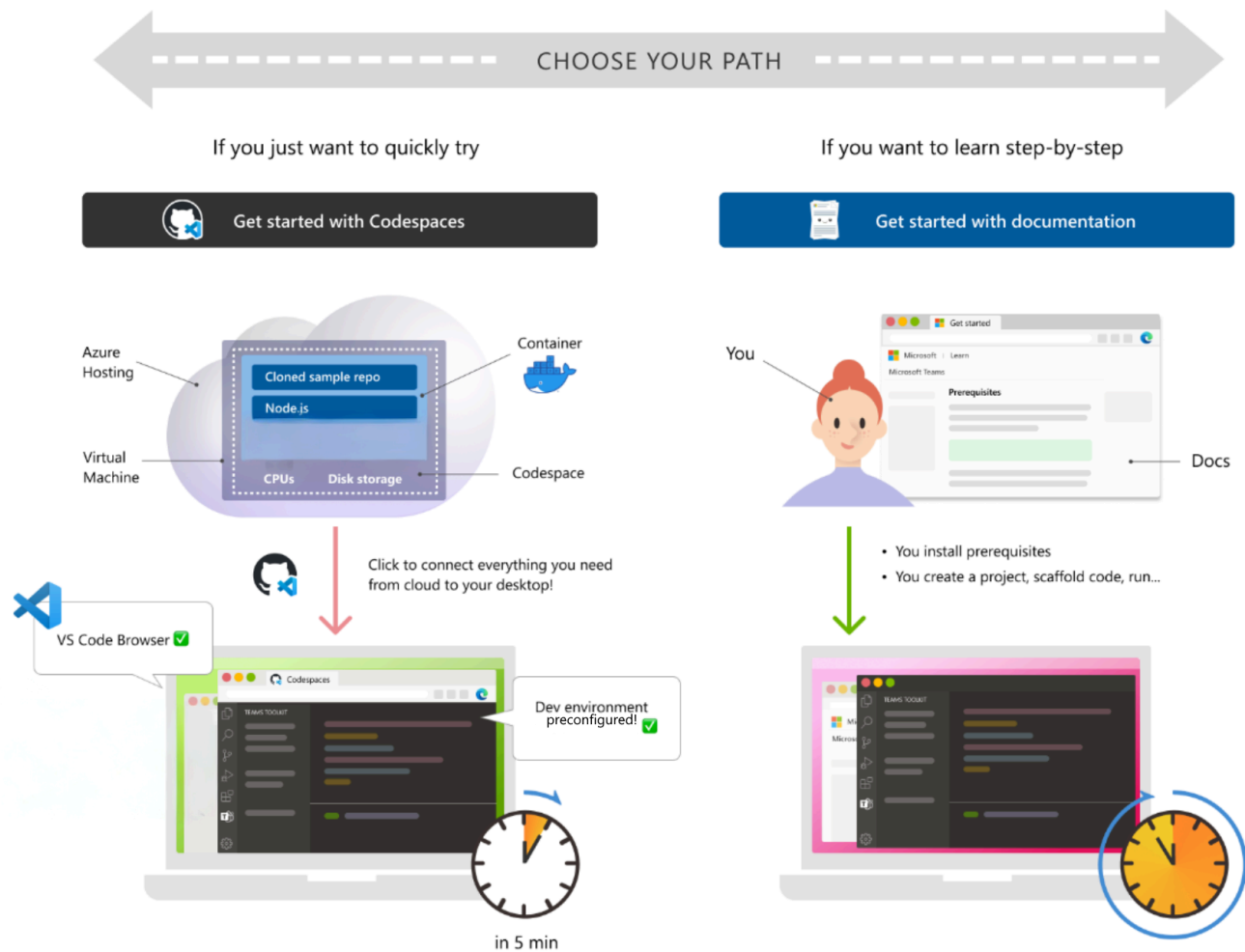
Note

You don't need to deploy anything to your tenant to try the [Starter-Kit](#). Use the DevGateway during development to emulate the Fabric host.

Important

Ensure you have access to a Fabric workspace and your tenant allows developer features. You enable Fabric Developer Mode before testing.

Choose your path



To get started quickly, pick one of the following options to access the code:

Option 1: Start in GitHub Codespaces

1. Fork the [Starter-Kit Repository](#) to your GitHub account.
2. Create a GitHub Codespace on your fork to get started with a cloud-based development environment.

For more information about GitHub Codespaces, see [GitHub Codespaces documentation](#).

Option 2: Clone and run locally

1. Fork the [Starter-Kit Repository](#) to your GitHub account.
2. Clone your fork to your local machine:

```
pwsh
```

```
git clone https://github.com/<your-account>/fabric-extensibility-toolkit.git  
cd fabric-extensibility-toolkit
```

3. Install the required prerequisites for local development:

Required tools:

- **Node.js LTS:** [Download Node.js](#) ↗
- **PowerShell 7** (pwsh on macOS/Linux): [Install PowerShell](#)
- **.NET SDK** (for macOS use the x64 build): [Download .NET](#) ↗
- **VS Code** or similar editor: [Download VS Code](#) ↗
- **Azure CLI** (used only for Microsoft Entra app creation): [Install Azure CLI](#)

AI-assisted development

This repository works well with AI pair-programming tools. Whether you develop locally or in GitHub Codespaces, you can use GitHub Copilot or other AI assistants to accelerate tasks like editing React components, update routes, or generate test scaffolding.

Tip

The Starter-Kit repository is AI-enabled and includes GitHub Copilot instructions that guide you through adapting the Hello World item to your needs. Other AI tools (for example, Anthropic Claude) can follow the same guidance, but must be configured to read the repository's guidance files or docs.

- Use AI to draft item editor/view components and then adapt to the host API patterns used in the Starter-Kit.
- Ask AI to summarize the workload manifest and propose minimal permission sets.
- In Codespaces, Copilot is available in the browser or VS Code desktop; keep the dev server running to see changes instantly.

Tip

If you're interested to see what others build open the [Extensibility Samples](#) ↗ and deploy it to your environment. There you can find rich item types that help you get started.

Best practices

- **Fork the repository:** Fork the [Starter-Kit](#) ↗ repository and use your fork as the base of your project.
- **Keep in sync:** Keep your fork in sync with upstream to pick up improvements.

- **Maintain project structure consistency:** Preserve the Starter-Kit's project structure and organization patterns to ensure compatibility with future updates and maintain code clarity.
- **Regular Starter-Kit integration:** Regularly integrate code changes from the Starter-Kit into your project to benefit from bug fixes, new features, and security updates. Set up a process to review and merge upstream changes on a regular basis (monthly or quarterly).
- **Validate manifests early:** Validate your workload manifest early and follow least-privilege permissions.
- **Use dev containers:** Use a dev container or Codespaces for a consistent, disposable environment.
- **Use provided scripts:** Use the provided scripts (Setup, StartDevServer, StartDevGateway) to automate setup and daily workflow.

Next steps

Once you have access to the code (either in Codespaces or locally), continue with the [Detailed Setup Guide](#) for step-by-step instructions on:

- Running the setup script
- Starting the development environment
- Enabling Fabric developer features
- Testing your first HelloWorld item
- Getting started with coding
- AI-assisted development tips
- Troubleshooting common issues

Additional resources

- Learn the [architecture](#) and how the host, your app, and Fabric services interact
- Read the [Manifest Overview](#) for schema and best practices
- Understand the [Workload manifest](#) structure and configuration
- Develop with the [DevGateway](#) for local testing
- [Publish your workload](#) when you're ready to share it broadly

Detailed Setup Guide

Once you have access to the code (either in Codespaces or locally) by following the [Getting Started guide](#), follow these steps:

Tip

If you don't want to follow the steps manually, ask GitHub Copilot to help you do the work. The Starter-Kit repository is AI-enabled and Copilot can guide you through the setup process interactively. Try this prompt: **"Help me get started with this repository and run the HelloWorld sample."**

Step 1: Run the setup script

The setup script automates most configuration (app registration, defaults, etc.).

```
pwsh
```

```
# From the repo root  
pwsh ./scripts/Setup/Setup.ps1 -WorkloadName "Org.MyWorkload"
```

Setup script notes:

- WorkloadName must follow the pattern Organization.WorkloadName. For development, use `Org.[YourWorkloadName]`.
- If reusing an existing Microsoft Entra app, ensure SPA redirect URIs are configured as described in the repositories manual setup guide.
- On macOS/Linux, use `pwsh` to run scripts.
- Make sure that the PowerShell execution policy is set to Unrestricted and the files are unblocked if you're getting asked if the PowerShell files should be started.
- Follow the guidance the Script provides to get everything setup

The Setup script can be run several times. If values are already present, you're asked if they should be overwritten. If you want to overwrite everything, use the Force parameter.

Note

If you encounter setup errors, see the [PowerShell setup error](#) in the troubleshooting section below.

Tip

If you have issues with the automated setup script, you can follow the [Manual Setup Guide](#) for step-by-step manual configuration.

Step 2: Start the development environment

Run the dev server (front end + APIs) and register your local instance with Fabric via DevGateway.

pwsh

```
# Terminal 1: start local dev server  
pwsh ./scripts/Run/StartDevServer.ps1
```

```
# Terminal 2: start DevGateway to register your local instance with Fabric  
pwsh ./scripts/Run/StartDevGateway.ps1
```

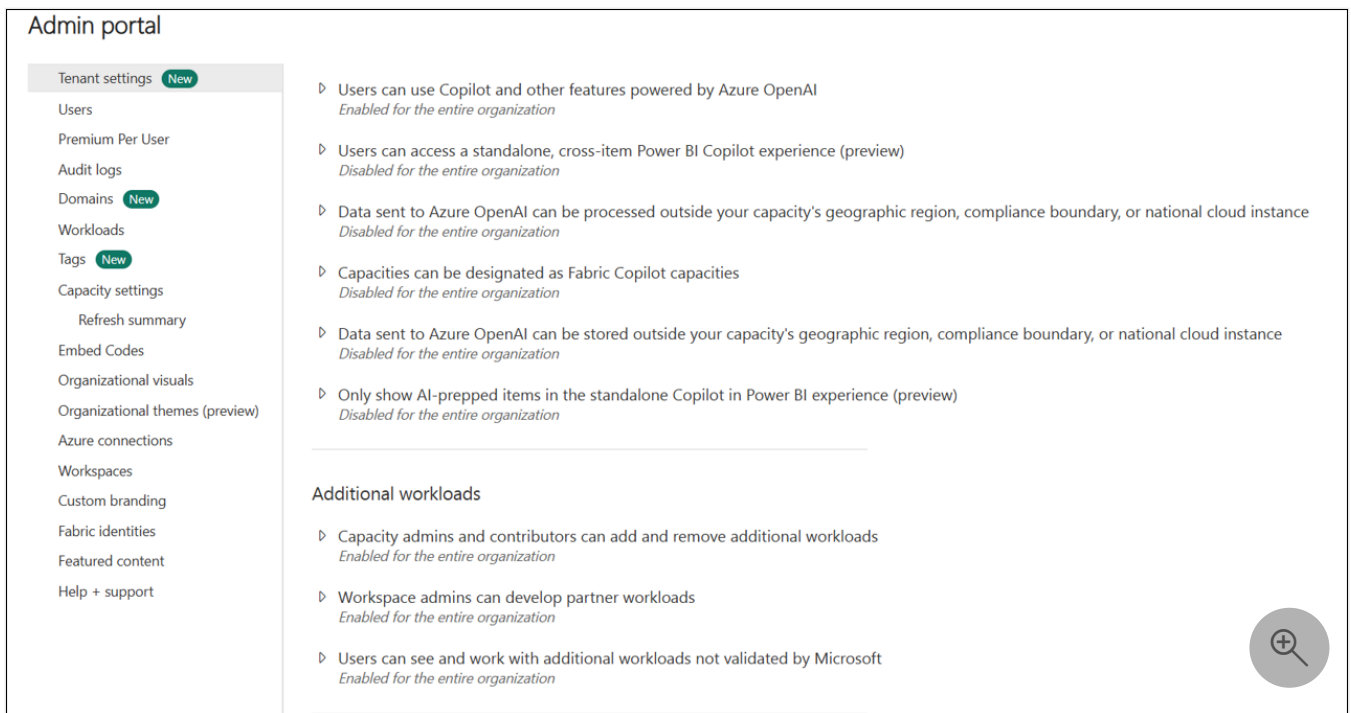
Step 3: Enable developer features in Fabric

Navigate to the Fabric portal and configure the required settings:

3.1 Configure tenant settings in Admin Portal:

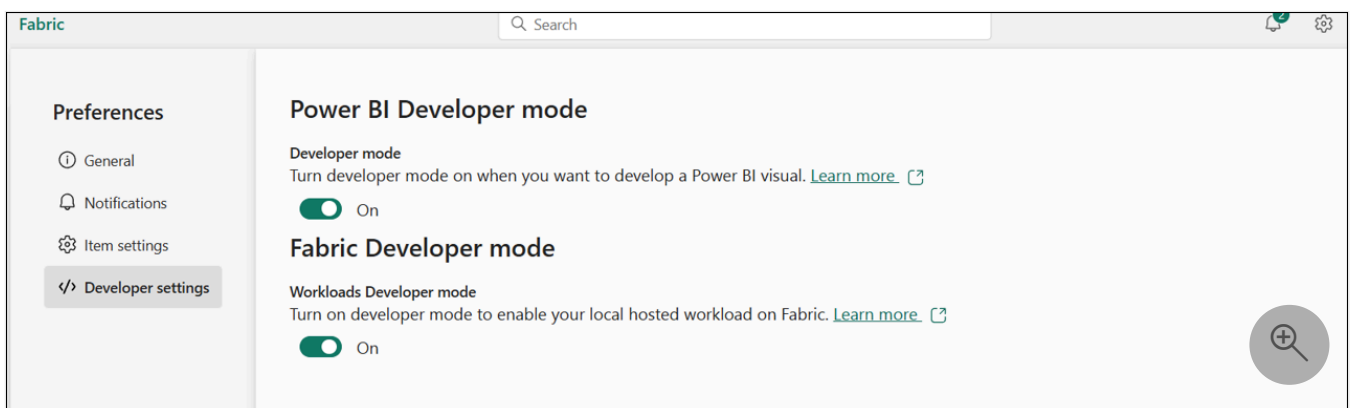
Head to the Admin Portal settings and enable the following tenant settings:

- Capacity admins and contributors can add and remove additional workloads
- Workspace admins can develop partner workloads
- Users can see and work with additional workloads not validated by Microsoft



3.2 Turn on Fabric Developer Mode:

Navigate to the Fabric Developer Settings and enable the Fabric Developer Mode:



You're now ready to create your first Hello World Item in Fabric.

Step 4: Test the HelloWorld item

You can access the workload from the Workload Hub (look for your workload name) or navigate directly. Then create a Hello World item.

Steps:

1. **Open Fabric Workload Hub:** Locate your workload (for example, `Org.MyWorkload`).

- **Direct navigation:**

```
https://app.fabric.microsoft.com/workloadhub/detail/<WORKLOAD_NAME>?
experience=fabric-developer
```

- Replace `<WORKLOAD_NAME>` with your actual workload name (e.g., `Org.MyWorkload`)
2. **Create a new item:** Select the Hello World item type and choose your development workspace.
 3. **Verify functionality:** The editor opens; confirm the item works as expected and appears like a native artifact in the workspace.

Congratulations! You created your first item from your development environment.

Step 5: Start coding

Now that you're all set up, you can start creating your own custom items. Follow our comprehensive guide to learn how to create custom Fabric items:

 **Creating Custom Fabric Items Guide** - This guide provides two approaches for creating items:

- **AI-Assisted Approach:** Use GitHub Copilot for interactive guidance (recommended for new developers)
- **Manual Scripted Approach:** Use automated PowerShell scripts for quick setup (recommended for experienced developers)

Quick start options:

- Update the existing HelloWorld editor:
`Workload/app/items/HelloWorldItem/HelloWorldItemEditor.tsx`
- Or scaffold a new item with the script: `./scripts/Setup/CreateNewItem.ps1`

Happy coding! 🚀

Best practices

- **Fork the repository:** Fork the [Starter-Kit](#) repository and use your fork as the base of your project.
- **Keep in sync:** Keep your fork in sync with upstream to pick up improvements.
- **Maintain project structure consistency:** Preserve the Starter-Kit's project structure and organization patterns to ensure compatibility with future updates and maintain code clarity.
- **Regular Starter-Kit integration:** Regularly integrate code changes from the Starter-Kit into your project to benefit from bug fixes, new features, and security updates. Set up a process to review and merge upstream changes on a regular basis (monthly or quarterly).

- **Validate manifests early:** Validate your workload manifest early and follow least-privilege permissions.
- **Use dev containers:** Use a dev container or Codespaces for a consistent, disposable environment.
- **Use provided scripts:** Use the provided scripts (Setup, StartDevServer, StartDevGateway) to automate setup and daily workflow.

Troubleshooting common issues

PowerShell setup error

If you encounter an error during setup script execution, ensure you have the latest PowerShell installed and configured in the environment you run the script.

```
PS C:\Dev\Fabric\WDK\Microsoft-Fabric-workload-development-sample\scripts\Setup> .\Setup.ps1
At C:\Dev\Fabric\WDK\Microsoft-Fabric-workload-development-sample\scripts\Setup\Setup.ps1:171 char:105
+ ... and activate it under Settings > Developer settings > Fabric Develop ...
+ ~~~~~
The output stream for this command is already redirected.
At C:\Dev\Fabric\WDK\Microsoft-Fabric-workload-development-sample\scripts\Setup\Setup.ps1:176 char:31
+ Write-Host "Happy coding! ðŸš€"
+ ~~~~~
The string is missing the terminator: ".
At C:\Dev\Fabric\WDK\Microsoft-Fabric-workload-development-sample\scripts\Setup\Setup.ps1:149 char:39
+ if (Test-Path $startDevGatewayScript) {
+ ~~~~~
Missing closing '}' in statement block or type definition.
+ CategoryInfo          : ParserError: (:) [], ParseException
+ FullyQualifiedErrorId : StreamAlreadyRedirected
```

Script execution policy error

If you encounter: `cannot be loaded because the execution policy is unrestricted:`

```
pwsh
```

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Port 5173 is in use

If the DevServer can't start on port 5173, find and terminate the process:

```
pwsh
```

```
# Find process using port
Get-Process -Id (Get-NetTCPConnection -LocalPort 5173).OwningProcess

# Terminate process
Stop-Process -Id <ProcessId> -Force
```

Dependencies errors

If you encounter errors about missing dependencies:

```
pwsh
# From Workload folder
cd Workload
npm install
```

DevGateway authentication issues

If authentication fails when starting DevGateway:

1. Verify the app registration redirect URIs in Microsoft Entra
2. Check that tenant settings allow the required developer features
3. Ensure you're signed in to the correct tenant in DevGateway

For comprehensive troubleshooting information, refer to the [Starter-Kit Repository README](#).

Important Notes

New Chrome Local Network Access Restrictions

Google has introduced new Local Network Access (LNA) restrictions in Chrome that **will break local development** with the [DevServer](#). These restrictions prevent websites from accessing local network resources without explicit user permission.

Action Required for Local Development: You must change your Chrome configuration to continue developing workloads locally:

1. Navigate to `chrome://flags/#local-network-access-check` in Chrome
2. Set the flag to "Disabled"
3. Restart Chrome

Why this is needed: The new restrictions block communication between your workload running in Fabric and the local DevGateway server, which is essential for the development workflow.

Additional Resources:

- [Local Network Access - Chrome Developers](#) - Official Chrome documentation on Local Network Access changes

Important

Without disabling this flag, local development with the DevGateway will not work in Chrome. This configuration change is only needed for development environments.

Next steps

- Learn the [architecture](#) and how the host, your app, and Fabric services interact
- Read the [Manifest Overview](#) for schema and best practices
- Understand the [Workload manifest](#) structure and configuration
- Develop with the [DevGateway](#) for local testing
- [Publish your workload](#) when you're ready to share it broadly

Last updated on 12/15/2025

Manual setup guide



Summarize this article for me

We recommend using the [Setup Guide](#) instead of following these instructions. In case you're interested in the details or try to skip some steps you can find the information on how to configure the repository manually in this article.

Create the configuration files

The repository takes the configuration values from the `.env.{ENVIRONMENT}` files in the repository. For this you first need to copy the [.env.template](#) file to `.env.dev`, `.env.test`, and `.env.prod` in the same directory and populate the values. The values in those files are used in creating the Manifest. For local development only the `.env.dev` file is normally needed.

Decide on a WorkloadName and fill it out in `.env.dev` file.

Register a front end Microsoft Entra application

You can use the [CreateDevAADApp.ps1](#) to create a new Microsoft Entra app or you can follow the steps.

1. Navigate to App registrations in the Microsoft Entra IDmin Portal.
2. Create a new Multitenant application.

[All services](#) > [App registrations](#) >

Register an application ...

*** Name**

The user-facing display name for this application (this can be changed later).

Supported account types

Who can use this application or access this API?

☐ Accounts in this organizational directory only (Contoso only - Single tenant)

☒ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)



3. Add the following SPA redirectURLs to the application manifest:

- https://app.fabric.microsoft.com/workloadSignIn/{TENANT_ID}/{WORKLOAD_NAME} ↗
- https://app.powerbi.com/workloadSignIn/{TENANT_ID}/{WORKLOAD_NAME} ↗
- https://msit.fabric.microsoft.com/workloadSignIn/{TENANT_ID}/{WORKLOAD_NAME} ↗
- https://msit.powerbi.com/workloadSignIn/{TENANT_ID}/{WORKLOAD_NAME} ↗

Choose the {WORKLOAD_NAME} in the setup process. Take a look at the Project Structure in the repository.

Looking for your {TENANT_ID}? Follow these steps:

1. Open Microsoft Fabric and select on your profile picture in the top right corner.
2. Select **About** from the dropdown menu.
3. In the About dialog, you find your Tenant ID and Tenant Region.

After the App is created, open the "Workload/.env.dev" file and insert your Workload name in the "WORKLOAD_NAME" and your frontend application client ID in the "FRONTEND_APPID" configuration property.

DevGateway

The repository required more software that needs to be downloaded. For this you need to download the [DevGateway](#) ↗ and unzip it into the `tools` directory.

To start the DevGateway, a configuration file named `workload-dev-mode.json` needs to be available in the `build/DevGateway/` directory. This file tells the DevGateway how to locate and serve your workload.

Sample configuration:

JSON

```
{
  "WorkspaceGuid": "your-workspace-guid-here",
  "ManifestPackageFilePath": "path/to/Your/Repo/build/Manifest/[WorkloadName].[Version].nupkg",
  "WorkloadEndpointURL": "http://127.0.0.1:5000/workload"
}
```

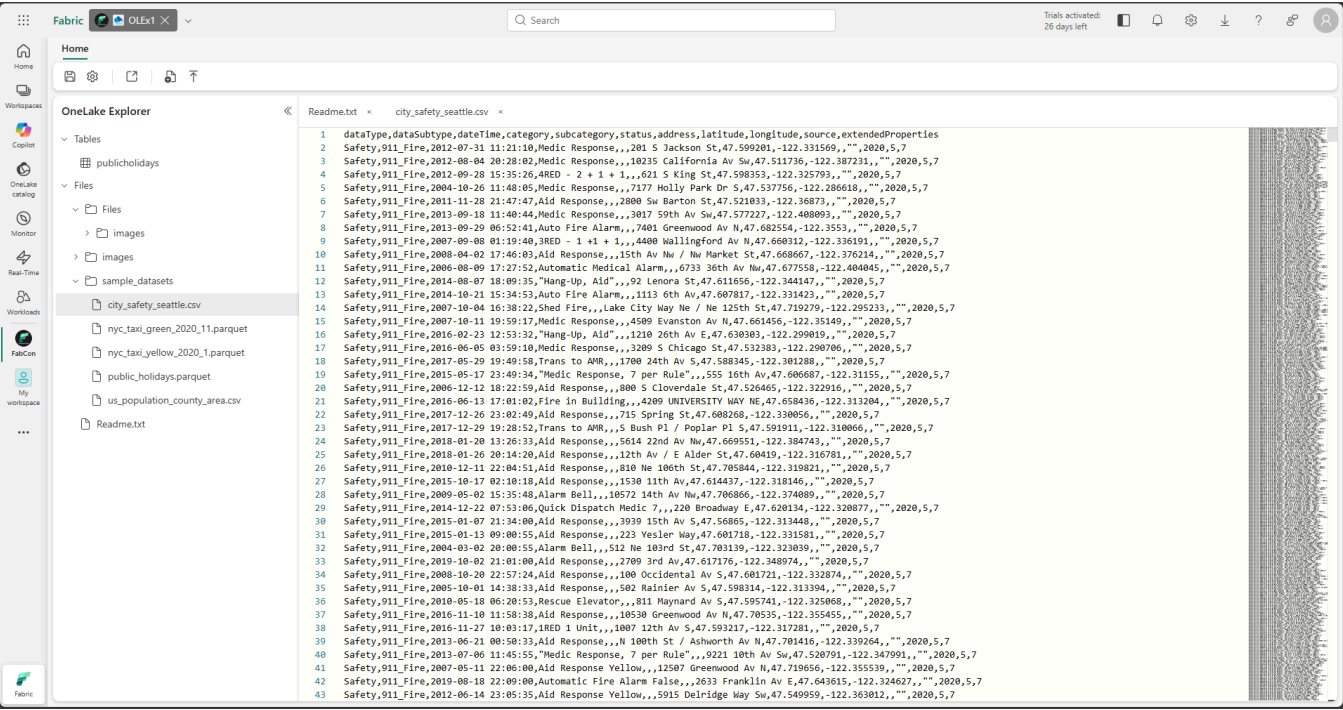
Field explanations:

- `WorkspaceGuid`: The GUID of your Fabric workspace where the workload is available.
- `ManifestPackageFilePath`: Path to your compiled workload manifest package (.nupkg file).

- `WorkloadEndpointURL`: URL where your workload backend is hosted (typically your local dev server).
-

Last updated on 02/19/2026

OneLake Explorer Item



The OneLake Explorer sample item demonstrates how to build a Fabric workload item that interacts with OneLake. It provides a file explorer interface allowing users to browse, view, and edit files stored in OneLake, and preview table files.

Overview

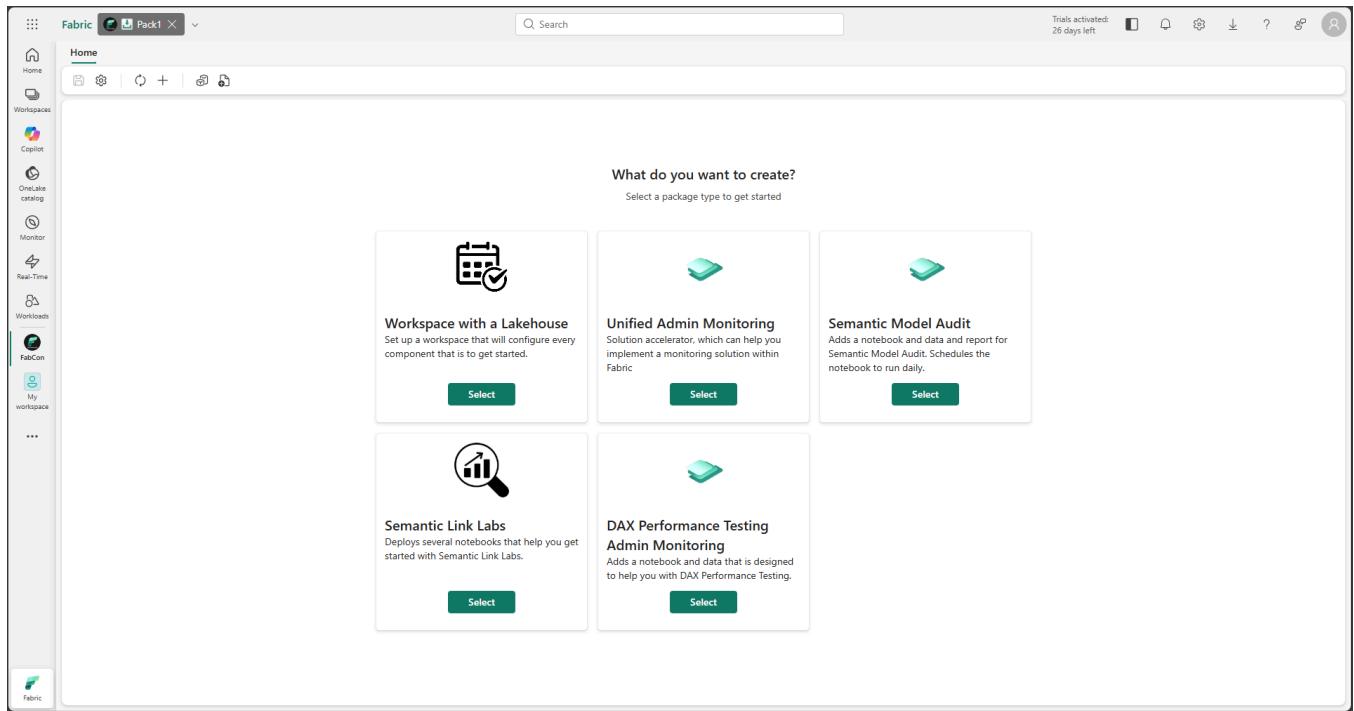
The OneLake Explorer item showcases several key capabilities of the Fabric Extensibility Toolkit:

- **OneLake Integration:** Connects to OneLake to list files and folders.
- **File Editing:** Integrates the Monaco editor to provide a rich code editing experience for various file types (JSON, XML, TypeScript, etc.).
- **Tabbed Interface:** Supports opening multiple files in tabs, managing state for each open file.
- **Table Preview:** Includes a view mode for previewing table data.
- **Item Picker:** Demonstrates how to use the Fabric item picker to select OneLake items (like Lakehouses) to explore.
- **Shortcuts:** Allows creating shortcuts to other OneLake items.

Reference

For detailed documentation, see the [OneLake Explorer Item documentation on GitHub](#).

Package Installer Item



The Package Installer sample item demonstrates how to build a Fabric workload item that manages the packaging and deployment of items. It allows users to define, configure, and execute deployments of various preconfigured Fabric items (like Notebooks, Lakehouses, etc.), define schedules, add data or create shortcuts. The deployment can happen into a newly created workspace or a folder within an existing workspace and is configurable.

Overview

The Package Installer item showcases advanced capabilities of the Fabric Extensibility Toolkit:

- **Package Management:** Defines packages containing multiple Fabric items and data assets.
- **Package Orchestration:** Manages the packaging process of existing items.
- **Deployment Orchestration:** Manages the deployment process, including creating workspaces, folders, and items.
- **Wizard-Based Interface:** Provides a user-friendly wizard for configuring and initiating deployments.
- **Status Tracking:** Tracks the progress and status of deployments.
- **Post-Deployment Actions:** Supports executing jobs (like Spark jobs) after a successful deployment.

Reference

For detailed documentation, see the [Package Installer Item documentation on GitHub](#).

Last updated on 12/16/2025

Cloud Shell Item

The Cloud Shell Item provides an interactive terminal interface for executing Cloud Shell commands, including full support for the [Fabric CLI](#), and Python scripts through Spark Livy sessions within Microsoft Fabric. This integration lets you run Fabric CLI commands in your browser to manage and automate Fabric resources, with command history and script management.

Overview

The Cloud Shell Item sample demonstrates several key capabilities of the Fabric Extensibility Toolkit:

- **Fabric CLI Integration:** Run [Fabric CLI](#) commands interactively for easy management and automation of Fabric resources.
- **Interactive Terminal** for Cloud Shell commands.
- **Python Script Management** with parameterized execution.
- **Batch Job Execution** for scripts with monitoring.
- **Session Management** with automatic reuse.
- **Execute Cloud Shell commands** through an integrated terminal interface.
- **Create and manage Python scripts** with parameter support.
- **Run scripts as batch jobs** with Spark configuration.
- **Manage Spark sessions** with automatic reuse and validation.
- **Select lakehouse and environment** for command execution context.
- **Track command history** with arrow key navigation.

Advanced Features (optional)

- **Switch execution modes** to run Python code or Bash commands.
- **Execute Python directly** in Spark sessions for data processing.
- **Run shell commands** with subprocess support.
- **Parameterized scripts** with type-safe parameter injection.

Reference

For detailed documentation, see the [Cloud Shell Item documentation on GitHub](#).

For more information about the Fabric CLI, visit the [Fabric CLI website](#).

For details about the Spark Livy API, see the [Apache Livy REST API documentation](#).

Tutorial: Create Microsoft Entra app for Fabric workload

This tutorial shows you how to create and configure a Microsoft Entra application for your Fabric workload. You can use either the automated script approach (recommended) or follow the manual steps for full control over the configuration.

Overview

A Microsoft Entra application is required for your Fabric workload to:

- **Authenticate users:** Enable single sign-on (SSO) for your workload users
- **Access Fabric APIs:** Call Microsoft Fabric APIs on behalf of the user
- **Define permissions:** Specify what resources your workload can access
- **Enable secure communication:** Establish trust between your workload and Fabric platform

Prerequisites

Before you begin, ensure you have:

- **Azure CLI installed:** Download from [Azure CLI documentation](#)
- **PowerShell 7.x or later:** Required for running the setup script
- **Tenant administrator access:** Or an account with permission to register applications
- **Workload name defined:** Following the pattern `Org.[YourWorkloadName]`

Approach 1: Automated Script (Recommended)

The extensibility toolkit provides an automated script that creates and configures your Microsoft Entra app with all required settings.

Running the Script

1. Navigate to the script directory:

```
PowerShell
cd scripts/Setup
```

2. Run the CreateDevAADApp script:

```
PowerShell
```

```
.\CreateDevAADApp.ps1
```

Or with parameters to avoid prompts:

```
PowerShell
```

```
.\CreateDevAADApp.ps1 -ApplicationName "My Workload App" -WorkloadName  
"Org.MyWorkload" -TenantId "your-tenant-id"
```

3. **Provide required information** when prompted:

- **Application Name:** Display name for your Microsoft Entra app (e.g., "My Analytics Workload")
- **Workload Name:** Must follow pattern `Org.[YourWorkloadName]` (e.g., "Org.MyWorkload")
- **Tenant ID:** Your Microsoft Entra ID tenant ID where the workload will be developed

Script Output

The script will provide you with essential information:

```
ApplicationIdUri / Audience : api://localdevinstance/[tenant-  
id]/Org.MyWorkload/[random]  
RedirectURI           : http://localhost:60006/close  
Application Id        : [your-app-id]  
secret                : [your-app-secret]
```

What the Script Configures

The automated script sets up:

Authentication Configuration

- **Redirect URIs:** Local development and Fabric platform URLs
- **Audience URI:** Unique identifier following the pattern `api://localdevinstance/[tenant-id]/[workload-name]/[random]`
- **Sign-in audience:** Multitenant support (`AzureADMultipleOrgs`)

API Permissions

- **Microsoft Fabric:** `Fabric.Extend`, workspace and item permissions
- **Azure Storage:** `user_impersonation` for OneLake access
- **Microsoft Graph:** `User.Read` for basic user information
- **Azure Data Explorer:** `user_impersonation` for KQL databases

OAuth Scopes (for Remote hosting)

- `FabricWorkloadControl`: Core workload functionality
- `Item1.Read.All` / `Item1.ReadWrite.All`: Custom item permissions
- `FabricLakehouse.Read.All` / `FabricLakehouse.ReadWrite.All`: Lakehouse access
- `KQLDatabase.ReadWrite.All`: KQL database permissions
- `FabricEventhouse.Read.All`: Eventhouse access

Pre-authorized Applications

- Microsoft Fabric platform applications
- Power BI service applications

Approach 2: Manual Configuration

If you need full control or prefer manual setup, follow these detailed steps.

Step 1: Create the Application

1. **Sign in to Microsoft Azure portal:** Navigate to [Microsoft Azure portal](#) and sign in with your administrator account.
2. **Access App Registrations:**
 - Go to **Azure Active Directory > App registrations**
 - Select **New registration**
3. **Configure Basic Settings:**
 - **Name:** Enter your application name (e.g., "My Analytics Workload")
 - **Supported account types:** Select "Accounts in any organizational directory (Any Microsoft Entra ID directory - Multitenant)"
 - **Redirect URI:** Leave empty for now (we'll add this later)
 - Select **Register**

Step 2: Configure Authentication

1. Add Redirect URIs:

- Go to **Authentication** in your app's menu
- Under **Platform configurations**, select **Add a platform**
- Choose **Single-page application (SPA)**
- Add these redirect URIs:

```
http://localhost:60006/close  
https://app.powerbi.com/workloadSignIn/[YOUR-TENANT-ID]/[YOUR-WORKLOAD-NAME]  
https://app.fabric.microsoft.com/workloadSignIn/[YOUR-TENANT-ID]/[YOUR-WORKLOAD-NAME]  
https://msit.powerbi.com/workloadSignIn/[YOUR-TENANT-ID]/[YOUR-WORKLOAD-NAME]  
https://msit.fabric.microsoft.com/workloadSignIn/[YOUR-TENANT-ID]/[YOUR-WORKLOAD-NAME]
```

2. Configure Token Settings:

- Under **Implicit grant and hybrid flows**, ensure nothing is selected (SPA uses PKCE)
- Under **Advanced settings**, enable **Allow public client flows**: No

Step 3: Set Application ID URI

1. Navigate to Expose an API:

- In your app's menu, select **Expose an API**
- Select **Add** next to "Application ID URI"

2. Create Unique URI:

```
api://localdevinstance/[YOUR-TENANT-ID]/[YOUR-WORKLOAD-NAME]/[RANDOM-STRING]
```

Example: `api://localdevinstance/12345678-1234-1234-1234-123456789012/Org.MyWorkload/AbCdE`

Step 4: Configure API Permissions

1. Add Required Permissions:

- Go to **API permissions**
- Select **Add a permission**

2. Add Microsoft Fabric Permissions:

- Select **APIs my organization uses**
- Search for "Power BI Service" (App ID: 00000009-0000-0000-c000-000000000000)
- Select **Delegated permissions** and add:
 - `Fabric.Extend`
 - `Workspace.Read.All`
 - `Item.Execute.All`
 - `Item.Read.All`
 - `Item.ReadWrite.All`
 - `Item.Reshare.All`
 - `Lakehouse.Read.All`
 - `Eventhouse.Read.All`
 - `KQLDatabase.ReadWrite.All`

3. Add Azure Storage Permissions:

- Add permission for "Azure Storage" (App ID: e406a681-f3d4-42a8-90b6-c2b029497af1)
- Select `user_impersonation`

4. Add Microsoft Graph Permissions:

- Add permission for "Microsoft Graph"
- Select `User.Read`

5. Add Azure Data Explorer Permissions:

- Add permission for "Azure Data Explorer" (App ID: 2746ea77-4702-4b45-80ca-3c97e680e8b7)
- Select `user_impersonation`

Step 5: Create OAuth Scopes (for Remote Hosting)

If your workload uses remote hosting, add custom scopes:

1. Navigate to Expose an API

2. Add Scopes:

- Select **Add a scope**
- For each scope, provide:
 - **Scope name:** E.g., `FabricWorkloadControl`
 - **Admin consent display name:** Descriptive name
 - **Admin consent description:** What the scope allows
 - **State:** Enabled

3. Required Scopes:

- `FabricWorkloadControl`
- `Item1.Read.All`
- `Item1.ReadWrite.All`
- `FabricLakehouse.Read.All`
- `FabricLakehouse.ReadWrite.All`
- `KQLDatabase.ReadWrite.All`
- `FabricEventhouse.Read.All`

Step 6: Configure Pre-authorized Applications

1. Add Authorized Client Applications:

- In **Expose an API**, select **Add a client application**
- Add these application IDs with appropriate scopes:

2. Microsoft Fabric Applications:

```
871c010f-5e61-4fb1-83ac-98610a7e9110 (Fabric client app)
00000009-0000-0000-c000-000000000000 (Power BI Service)
d2450708-699c-41e3-8077-b0c8341509aa (Additional Fabric app)
```

Step 7: Create Client Secret

1. Generate Secret:

- Go to **Certificates & secrets**
- Select **New client secret**
- Provide a description and set expiration (180 days recommended for development)
- Copy the secret value immediately (it won't be shown again)

Step 8: Grant Admin Consent

1. Consent to Permissions:

- In **API permissions**, select **Grant admin consent for [Your Organization]**
- Confirm the consent

2. Alternative Consent URL: You can also use this direct URL (wait a minute after app creation):

```
https://login.microsoftonline.com/[YOUR-TENANT-ID]/adminconsent?client_id=[YOUR-APP-ID]
```

Next Steps

After creating your Microsoft Entra app:

1. Update Environment Configuration:

- Copy the Application ID, secret, and other values
- Update your `.env` file with these values
- See [Setup Guide](#) for environment configuration

2. Test Authentication:

- Run your workload locally
- Verify that authentication works correctly
- Check that API calls succeed

3. Configure Additional Settings:

- Review and adjust token lifetimes if needed
- Add additional redirect URIs for different environments
- Configure optional claims if required

Troubleshooting

Common Issues

"Invalid Redirect URI"

- Ensure redirect URIs exactly match the configured values
- Check for trailing slashes or typos

- Verify protocol (http vs https)

"Insufficient Privileges"

- Ensure you have Application Administrator or Global Administrator rights
- Some permissions require admin consent

"Invalid Audience"

- Verify the Application ID URI follows the correct pattern
- Check that tenant ID and workload name are correct

"Permission Denied"

- Ensure admin consent has been granted
- Check that required permissions are added
- Verify the user has access to the workspace

Getting Help

If you encounter issues:

1. **Check Application Overview:** Review all configured settings
2. **Validate Permissions:** Ensure all required permissions are present and consented
3. **Review Logs:** Check Microsoft Entra ID sign-in logs for detailed error information
4. **Test with Graph Explorer:** Verify permissions using [Microsoft Graph Explorer](#)

Security Best Practices

- **Rotate Secrets Regularly:** Set up a process to rotate client secrets before expiration
- **Use Shortest Necessary Permissions:** Only grant permissions your workload actually needs
- **Monitor Usage:** Regularly review sign-in logs and API usage
- **Secure Secret Storage:** Never commit secrets to source code; use secure configuration management

Related Resources

- [Setup Guide](#) - Complete environment setup
- [Authentication Overview](#) - Understanding Fabric authentication
- [Authentication Guidelines](#) - Best practices for implementation
- [Microsoft Entra Documentation](#) - Official Microsoft Entra development docs

Last updated on 12/15/2025

Create a new Fabric Item

This comprehensive guide shows you how to create custom items in Microsoft Fabric using the Extensibility Toolkit. You can choose between two approaches based on your preference and experience level.

Prerequisites

Before creating a custom item, ensure you have:

-  Completed the [Setup Guide](#) and have a working development environment
-  Your workload is running locally and accessible in Fabric
-  Familiarity with TypeScript and React (for customization)

Choose Your Approach

AI-Assisted Approach (Recommended for New Developers)

Use GitHub Copilot to guide you through the entire process interactively. Perfect for:

- Developers new to the Fabric Extensibility Toolkit
- Learning the platform patterns and best practices
- Getting explanations and guidance as you work

Manual Scripted Approach (Recommended for Experienced Developers)

Use the automated PowerShell script for quick setup. Perfect for:

- Developers familiar with the toolkit structure
- Creating multiple items efficiently
- Production workflows and automation

AI-Assisted Item Creation

Getting Started with GitHub Copilot

GitHub Copilot can guide you through creating a complete custom Fabric item following all best practices. The AI understands the Extensibility Toolkit patterns and will help you implement them correctly.

Sample Prompts That Work

Here are proven prompts that will get you started with item creation:

Basic Item Creation:

```
@fabric create a new item called MyDataReport that shows data analysis reports
```

Specific Requirements:

```
@fabric create a custom item for managing data pipelines with these features:  
- Pipeline configuration interface  
- Status monitoring dashboard  
- Error handling and retry logic
```

Complex Item with OneLake Integration:

```
@fabric create an item called DataExplorer that:  
- Browses OneLake files in the left panel  
- Shows file preview in the center  
- Saves user preferences and settings
```

Typical AI-Assisted Process

The AI-assisted approach follows this iterative pattern:

1. Initial Planning Phase

- AI analyzes your requirements
- Suggests item structure and components
- Creates a development plan with todos

2. Component Generation

- Creates the 4-file pattern (Definition, Editor, Empty, Ribbon)

- Implements proper TypeScript interfaces
- Sets up Fluent UI components

3. Feature Implementation

- Adds your specific functionality
- Integrates with Fabric APIs
- Implements proper error handling

4. Testing and Refinement

- Tests the item in your development environment
- Fixes any issues found
- Optimizes performance and user experience

Working with the AI Assistant

Start with Clear Requirements:

I want to create a [ItemType] item that [primary purpose].
The item should have [list key features].
Users should be able to [list main actions].

Iterate and Refine:

The item looks good, but I need to add [specific feature]

How can I integrate this with OneLake storage?

Can you add error handling for when the API is unavailable?

Ask for Explanations:

Explain why you chose this pattern for the ribbon actions

What's the difference between `ItemEditor` and `ItemEditorDefaultView`?

Best Practices for AI Collaboration

- **Be Specific:** Provide clear requirements and context
- **Review Each Step:** Understand the generated code before proceeding
- **Ask Questions:** Request explanations for patterns you don't understand
- **Test Frequently:** Run and test the item after each major change
- **Follow Up:** Ask for refinements and improvements

AI Development Tools and Environment

This repository works exceptionally well with AI pair-programming tools. Whether you develop locally or in GitHub Codespaces, you can use GitHub Copilot or other AI assistants to accelerate tasks like editing React components, updating routes, or generating test scaffolding.

Tip

The Starter-Kit repository is AI-enabled and includes GitHub Copilot instructions that guide you through adapting the Hello World item to your needs. Other AI tools (for example, Anthropic Claude) can follow the same guidance, but must be configured to read the repository's guidance files or docs.

Specific AI assistance areas:

- Use AI to draft item editor/view components and then adapt to the host API patterns used in the Starter-Kit
- Ask AI to summarize the workload manifest and propose minimal permission sets
- In Codespaces, Copilot is available in the browser or VS Code desktop; keep the dev server running to see changes instantly

Tip

If you're interested to see what others build, open the [Extensibility Samples](#)  and deploy it to your environment. There you can find rich item types that help you get started.

Rapid Iteration and Debugging

The Extensibility framework is designed for rapid development when using AI assistance:

- With the dev server and DevGateway running, code changes in your app are reflected immediately when you open your item inside Fabric
- You can debug using your browser's dev tools while the workload is hosted in the Fabric iFrame
- Iterate on UI, routes, and manifest configuration quickly, and validate end-to-end behavior in your Fabric workspace
- AI tools can help identify and fix issues in real-time as you develop

Expected Timeline

- **Simple Item:** 30-60 minutes with AI guidance
- **Complex Item:** 1-3 hours with multiple iterations
- **Advanced Item:** Half day with extensive customization

Manual Scripted Approach

Using the CreateNewItem.ps1 Script

The manual approach uses an automated PowerShell script that copies the HelloWorld item template and configures it for your new item.

Quick Start

1. Navigate to the script directory:

```
PowerShell
```

```
cd scripts\Setup
```

2. Run the creation script:

```
PowerShell
```

```
.\CreateNewItem.ps1 -ItemName "MyCustomItem"
```

What the Script Does

Automated File Creation:

-  Copies all 4 core component files from HelloWorld template
-  Renames files to match your item name
-  Updates all internal references and imports
-  Creates manifest files (XML and JSON configurations)
-  Copies and renames icon assets

Generated File Structure:

```
Workload/app/items/MyCustomItemItem/
├─ MyCustomItemDefinition.ts      # Data model and interfaces
├─ MyCustomItemEditor.tsx        # Main editor component
├─ MyCustomItemEditorEmpty.tsx   # First-time user experience
├─ MyCustomItemEditorRibbon.tsx  # Action buttons and toolbar
└─ MyCustomItem.scss             # Styling

Workload/Manifest/items/MyCustomItem/
├─ MyCustomItemItem.xml          # Item type definition
└─ MyCustomItemItem.json        # Item configuration

Workload/Manifest/assets/images/
└─ MyCustomItemItem-icon.png    # Item icon
```

Required Manual Steps

After running the script, you **must** complete these manual configuration steps:

1. Update Environment Files CRITICAL

Add your new item to the `ITEM_NAMES` variable in ALL environment files:

Files to update:

- `Workload\.env.dev`
- `Workload\.env.test`
- `Workload\.env.prod`

Change from:

```
ITEM_NAMES=HelloWorld
```

Change to:

```
ITEM_NAMES=HelloWorld,MyCustomItem
```

 Without this step, your item will NOT appear in the workload!

2. Update Product.json Configuration

Add your item configuration to `Workload\Manifest\Product.json`:

JSON

```
{
  "items": [
    {
      "name": "HelloWorldItem",
      // ... existing config
    },
    {
      "name": "MyCustomItemItem",
      "displayName": "My Custom Item",
      "description": "Description of what your item does",
      "contentType": "MyCustomItem",
      "configurationSections": []
    }
  ]
}
```

3. Add Localization Strings

Update translation files in `Workload\Manifest\assets\locales\`:

In `en-US.json` (and other locale files):

JSON

```
{
  "MyCustomItemItem": {
    "displayName": "My Custom Item",
    "description": "Description of your custom item"
  }
}
```

4. Add Routing Configuration

Update `Workload\app\App.tsx` to include routing for your new item:

TypeScript

```
// Add import
import { MyCustomItemEditor } from "../items/MyCustomItemItem/MyCustomItemEditor";

// Add route in the Routes section
<Route path="/MyCustomItemItem-editor" element={<MyCustomItemEditor {...props} />} />
```

Verification Steps

After completing all manual steps:

1. Build the project:

PowerShell

```
npm run build
```

2. Restart the development server:

PowerShell

```
.\scripts\Run\StartDevServer.ps1
```

3. Test in Fabric:

- Navigate to your workload in Fabric
- Verify your new item type appears in the creation dialog
- Create an instance and verify it loads correctly

Best Practices and Guidelines

Why Not Rename HelloWorld?

The HelloWorld item serves as the **reference template** that receives regular updates from Microsoft. Key reasons to keep it unchanged:

- **Updates and Improvements:** Microsoft regularly updates HelloWorld with new features, bug fixes, and best practices
- **Integration Testing:** HelloWorld ensures your environment is working correctly
- **Reference Documentation:** It serves as live documentation for implementation patterns
- **Backward Compatibility:** Updates maintain compatibility with existing customizations

Recommended Workflow

1. **Keep HelloWorld Untouched:** Use it as your reference and testing item
2. **Create New Items:** Use the script or AI assistance to create separate items
3. **Regular Updates:** Periodically pull updates from the base repository
4. **Merge Patterns:** Apply new patterns from updated HelloWorld to your custom items

Item Development Best Practices

Component Architecture:

-  Follow the 4-component pattern (Definition, Editor, Empty, Ribbon)
-  Use `ItemEditor` container for consistent layout and behavior
-  Implement proper loading states and error handling
-  Follow Fluent UI design patterns

Data Management:

-  Use `saveWorkloadItem()` for items with complex data
-  Use `getWorkloadItem()` for loading with fallback defaults
-  Implement proper TypeScript interfaces in Definition file
-  Handle undefined/empty states gracefully

User Experience:

-  Provide clear empty states for first-time users
-  Use proper loading indicators
-  Implement helpful error messages
-  Follow Fabric design patterns and accessibility guidelines

Performance Considerations

- **Lazy Loading:** Only load data when needed
- **Efficient Updates:** Use React patterns to minimize re-renders
- **OneLake Integration:** Use `createItemWrapper()` for proper scoping
- **Error Boundaries:** Implement proper error handling throughout

Next Steps

Once your item is created:

1. **Customize the Data Model:** Update the Definition file with your specific interfaces
2. **Implement Core Features:** Build out the main functionality in the Editor component
3. **Design Empty State:** Create an engaging first-time user experience
4. **Configure Actions:** Set up appropriate ribbon actions for your workflow
5. **Test Thoroughly:** Verify all functionality in your development environment
6. **Prepare for Production:** Follow the [publishing guide](#) when ready

Troubleshooting

Item doesn't appear in workload:

- ☒ Check ITEM_NAMES in all .env files
- ☒ Verify Product.json configuration
- ☒ Restart the development server

Build errors:

- ☒ Check TypeScript interfaces in Definition file
- ☒ Verify all imports are correct
- ☒ Ensure routing is properly configured

Runtime errors:

- ☒ Check browser console for detailed error messages
- ☒ Verify API integration and authentication
- ☒ Test with simplified data first

Additional Resources

- [ItemEditor Control Documentation](#)
- [OneLake Integration Guide](#)
- [Fabric API Usage](#)
- [Publishing Requirements](#)

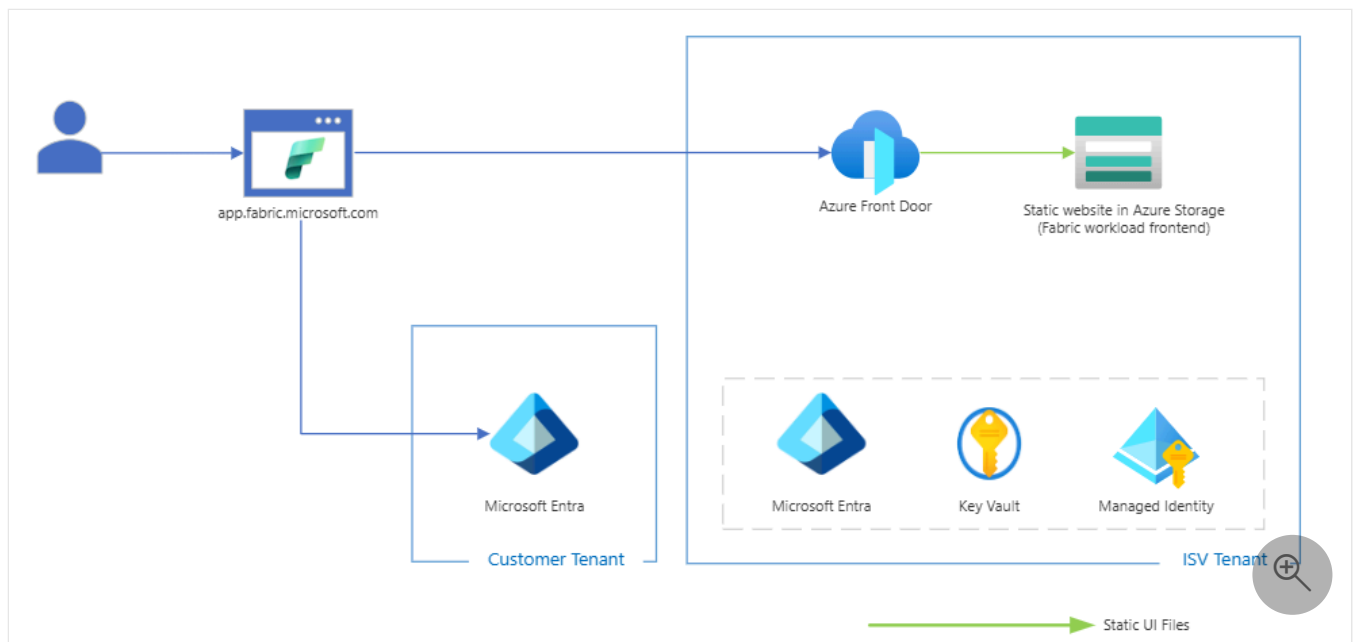
Happy coding! 🚀

How to host your workload in Azure

This article describes how to host your Fabric Extensibility Toolkit workload in Azure using a frontend-only architecture. The Extensibility Toolkit uses a cloud-native deployment approach with Azure services optimized for static web applications.

Architecture overview

The Fabric Extensibility Toolkit uses an architecture focused on frontend deployment with Azure services:



Extensibility Toolkit architecture

The Extensibility Toolkit architecture includes the following characteristics:

- **Frontend-only deployment**: Your workload runs entirely in the browser without a separate backend
- **Static website hosting**: Uses Azure Storage static websites for hosting
- **Azure Front Door**: Provides global CDN, SSL termination, and routing
- **Managed services**: Leverages Entra ID, Key Vault, and Managed Identity for security
- **Single deployment process**: Uses provided PowerShell scripts for deployment

Azure services used

Azure Front Door

[Azure Front Door](#) serves as the global entry point for your workload. It provides:

- **Global load balancing:** Routes users to the closest edge location
- **SSL termination:** Handles HTTPS certificates automatically
- **Web Application Firewall (WAF):** Protects against common web vulnerabilities
- **Caching:** Improves performance by caching static assets at edge locations

For the Extensibility Toolkit, Front Door routes traffic to your static website hosted on Azure Storage, ensuring high availability and low latency worldwide.

Azure Storage Account (Static Website)

An [Azure Storage Account with static website hosting](#) hosts your built workload application. This provides:

- **Cost-effective hosting:** Pay only for storage and bandwidth used
- **Automatic scaling:** Handles traffic spikes without configuration
- **Global availability:** Content is served from Azure's global storage infrastructure
- **File-based deployment:** Upload files to deploy versions

The Extensibility Toolkit builds your React application into static HTML, JavaScript, CSS, and assets that are deployed directly to the storage account.

Azure Key Vault

[Azure Key Vault](#) manages sensitive configuration and secrets:

- **Client secrets:** Stores Entra ID application secrets securely
- **API keys:** Manages external service API keys
- **Configuration:** Stores environment-specific settings
- **Certificates:** Manages SSL certificates if needed

Your workload accesses Key Vault through Managed Identity for secure, credential-free authentication.

Azure Entra ID (Azure Active Directory)

[Azure Entra ID](#) provides authentication and authorization:

- **App registration:** Registers your workload as an Entra application
- **OAuth 2.0 flows:** Handles user authentication with Fabric
- **API permissions:** Manages access to Fabric and Microsoft Graph APIs
- **Token management:** Provides secure access tokens for API calls

Managed Identity

[Managed Identity](#) provides secure service-to-service authentication:

- **No credential management:** Eliminates need to store secrets in code
- **Token handling:** Azure handles token acquisition and renewal
- **Secure access:** Connects to Key Vault and other Azure services securely
- **Identity lifecycle:** Tied to your Azure resources for cleanup

Deployment process

Prerequisites

Before deploying to Azure, ensure you have:

- Azure subscription with appropriate permissions
- [Azure CLI](#) installed and authenticated
- Fabric Extensibility Toolkit repository cloned locally
- Your workload built using `.\scripts\Build\BuildRelease.ps1`

Using the deployment script

The Extensibility Toolkit includes a PowerShell deployment script at `scripts\Deploy\DeployToAzureWebApp.ps1` that automates the deployment process.

Basic deployment

PowerShell

```
# Deploy to an existing Azure Web App
.\scripts\Deploy\DeployToAzureWebApp.ps1 -WebAppName "my-fabric-workload" -
ResourceGroupName "fabric-workload-rg"
```

Additional deployment options

PowerShell

```
# Deploy to staging slot with custom settings
.\scripts\Deploy\DeployToAzureWebApp.ps1 `
  -WebAppName "my-fabric-workload" `
  -ResourceGroupName "fabric-workload-rg" `
  -SlotName "staging" `
```

```
-Force $true `
-CreateBackup $true `
-RestartAfterDeploy $true
```

Deployment script parameters

 Expand table

Parameter	Description	Required	Default
WebAppName	Name of the Azure Web App to deploy to	Yes	-
ResourceGroupName	Resource group containing the Web App	Yes	-
ReleasePath	Path to built application files	No	..\..\release\app
DeploymentMethod	Deployment method (ZipDeploy, FTP, LocalGit)	No	ZipDeploy
SlotName	Deployment slot for staging	No	-
Force	Skip confirmation prompts	No	\$false
CreateBackup	Create backup before deployment	No	\$true
RestartAfterDeploy	Restart app after deployment	No	\$true

Deployment script features

The deployment script provides comprehensive deployment capabilities:

Validation and safety

- **Prerequisites check:** Validates Azure CLI installation and authentication
- **Resource validation:** Confirms target Web App exists and is accessible
- **Build validation:** Ensures release directory contains required files
- **Backup creation:** Creates deployment backup for rollback capabilities

Deployment workflow

- **ZIP package creation:** Compresses built application into deployment package
- **Size reporting:** Shows deployment package size for verification
- **Progress monitoring:** Provides real-time deployment status updates
- **Error handling:** Detailed error messages with troubleshooting guidance

Post-deployment

- **Health check:** Validates deployed application responds correctly
- **URL reporting:** Provides direct link to deployed application
- **Manifest guidance:** Shows next steps for manifest upload to Fabric
- **Timing metrics:** Reports total deployment duration

Manual deployment

You can also deploy your frontend application manually using Azure PowerShell commands:

Build your application

First, build your frontend application for the test environment:

PowerShell

```
npm run build:test
```

Create deployment package

1. Navigate to the `build\Frontend` folder in your project
2. Select all files and the `assets` folder under the build directory
3. Create a `.zip` file containing all selected files

Deploy using Azure PowerShell

PowerShell

```
# Connect to Azure
Connect-AzAccount

# Set your subscription context
Set-AzContext -Subscription "<subscription_id>"

# Deploy the zip file to your web app
Publish-AzWebApp -ResourceGroupName <resource_group_name> -Name <web_app_name> -
ArchivePath <zip_file_path>
```

Manual deployment parameters

Parameter	Description	Example
<code><subscription_id></code>	Your Azure subscription ID	<code>12345678-1234-1234-1234-123456789012</code>
<code><resource_group_name></code>	Resource group containing your web app	<code>fabric-workload-rg</code>
<code><web_app_name></code>	Name of your Azure web app	<code>my-fabric-workload</code>
<code><zip_file_path></code>	Full path to your deployment zip file	<code>C:\path\to\deployment.zip</code>

ⓘ Note

Manual deployment requires you to have the appropriate Azure permissions and the [Azure PowerShell module](#) installed on your machine.

Security considerations

Authentication flow

Your workload authenticates with Fabric using the standard OAuth 2.0 flow:

1. User accesses your workload through Fabric
2. Fabric redirects to your Azure-hosted application
3. Your app redirects to Entra ID for authentication
4. Entra ID returns authentication token
5. Your app uses token to access Fabric APIs

Secure configuration

- **Environment variables:** Store configuration in Azure App Settings, not in code
- **Key Vault integration:** Access secrets through Managed Identity
- **HTTPS only:** Enforce HTTPS for all communication
- **CORS configuration:** Configure appropriate CORS policies for Fabric domains

Best practices

- **Least privilege:** Grant minimal required permissions to Managed Identity
- **Secret rotation:** Regularly rotate secrets stored in Key Vault
- **Network security:** Use Private Endpoints where possible

- **Monitoring:** Enable Application Insights for security monitoring

Configuration after deployment

Azure App Service configuration

After deployment, configure your Azure App Service:

Bash

```
# Set environment variables for your workload
az webapp config appsettings set --name "my-fabric-workload" --resource-group
"fabric-workload-rg" --settings \
    "FABRIC_CLIENT_ID=your-client-id" \
    "FABRIC_TENANT_ID=your-tenant-id" \
    "KEY_VAULT_URL=https://your-keyvault.vault.azure.net/"
```

Key Vault setup

Store sensitive configuration in Key Vault:

Bash

```
# Store client secret
az keyvault secret set --vault-name "your-keyvault" --name "FabricClientSecret" --
value "your-client-secret"

# Store API keys
az keyvault secret set --vault-name "your-keyvault" --name "ExternalApiKey" --value
"your-api-key"
```

Managed Identity configuration

Configure Managed Identity for Key Vault access:

Bash

```
# Enable system-assigned managed identity
az webapp identity assign --name "my-fabric-workload" --resource-group "fabric-
workload-rg"

# Grant access to Key Vault
az keyvault set-policy --name "your-keyvault" \
    --object-id "managed-identity-principal-id" \
    --secret-permissions get list
```

Manifest deployment

After deploying your application to Azure, you must upload the manifest package to Fabric:

Build manifest package

First, build the manifest package:

```
PowerShell
```

```
.\scripts\Build\BuildManifestPackage.ps1
```

This creates `release\ManifestPackage.1.0.0.nupkg` containing your workload manifest.

Upload to Fabric Admin Portal

1. Open the [Microsoft Fabric Admin Portal](#) 
2. Navigate to **Workload Management** > **Upload workload**
3. Upload your `ManifestPackage.1.0.0.nupkg` file
4. Configure workload settings and permissions
5. Activate the workload for your tenant

Update manifest for Azure deployment

Before building your manifest package for Azure deployment, ensure that the values in your `.env` file (such as `FRONTEND_URL`, `FRONTEND_APPID`, and any other required variables) are set correctly for your Azure environment. These values will be substituted into your `WorkloadManifest.xml` during the build process.

For a full list of required environment variables and guidance on setting them, see the [General publishing requirements](#).

Monitoring and troubleshooting

Application Insights

Enable Application Insights for monitoring:

- **Performance monitoring:** Track page load times and user interactions
- **Error tracking:** Monitor JavaScript errors and failed requests

- **Usage analytics:** Understand how users interact with your workload
- **Custom telemetry:** Add custom metrics for business logic

Common issues and solutions

Deployment failures

- **Authentication errors:** Verify Azure CLI login with `az account show`
- **Resource not found:** Confirm Web App name and resource group are correct
- **Permission denied:** Ensure your account has Contributor role on the resource group

ⓘ Note

Error: Frontend Uri is not in the tenant domains list: This error means your workload's custom domain is not registered in your Entra ID tenant's list of accepted domains. To resolve, add your custom domain to Entra ID. For more information, see [Custom Domain Verification](#) in the general publishing requirements.

Runtime issues

- **White screen:** Check browser console for JavaScript errors
- **Authentication failures:** Verify Entra ID app registration and redirect URIs
- **API call failures:** Check CORS configuration and API permissions

Performance optimization

- **Slow loading:** Enable compression and optimize bundle size
- **Caching issues:** Configure proper cache headers in Front Door
- **Geographic latency:** Ensure Front Door is properly configured for global routing

Related guides

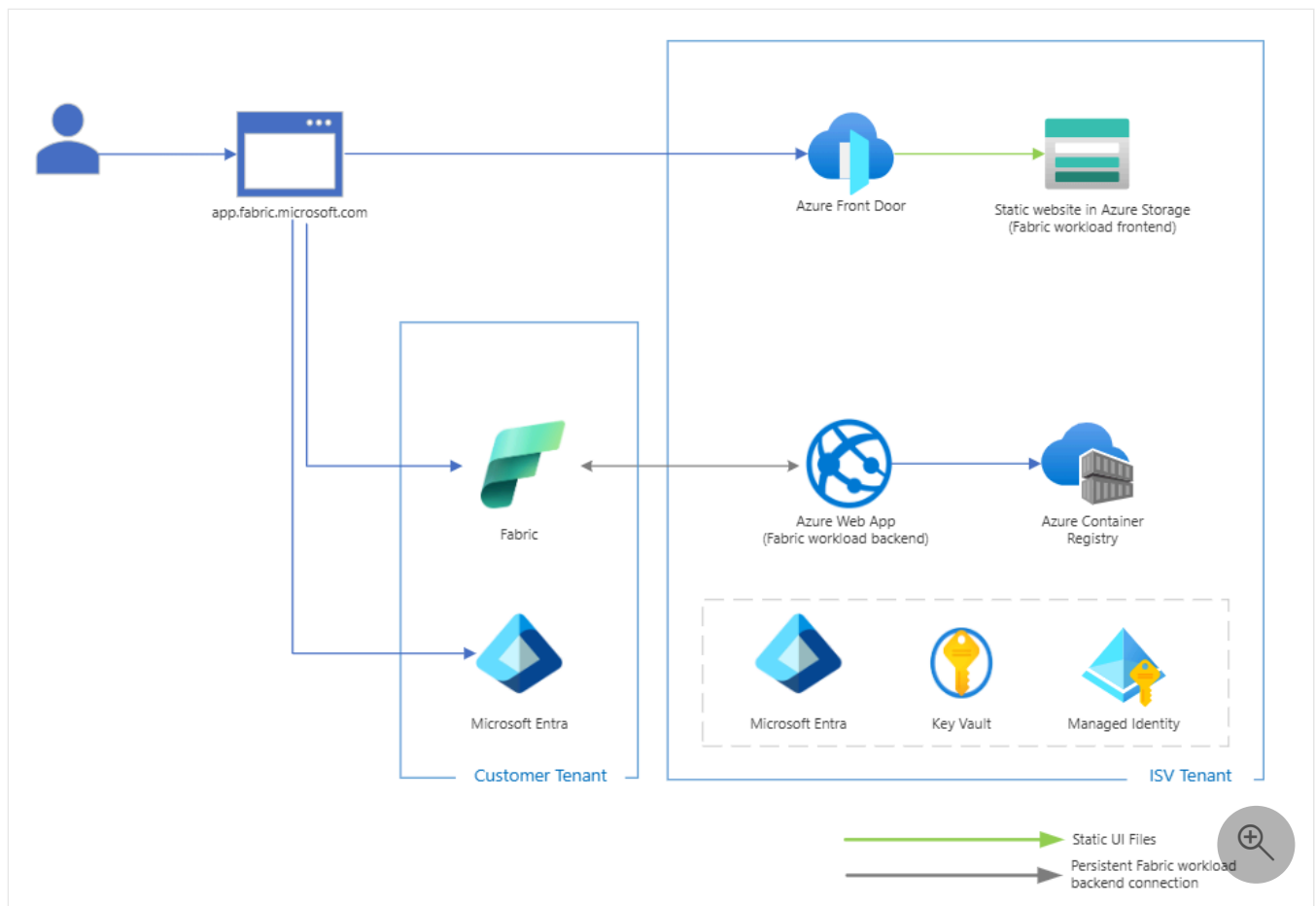
- [How to publish and manage a workload](#) - Uploading manifest to Fabric
- [Setup Guide](#) - Initial development environment setup
- [Tutorial: Getting Started](#) - Getting started with the Extensibility Toolkit

How to publish your workload in Azure

This article describes how to deploy your Fabric Extensibility Toolkit workload to Azure using a frontend-only architecture. The Extensibility Toolkit uses a cloud-native deployment approach with Azure services optimized for static web applications.

Architecture overview

The Fabric Extensibility Toolkit uses an architecture focused on frontend deployment with Azure services:



Extensibility Toolkit architecture

The Extensibility Toolkit architecture includes the following characteristics:

- **Frontend-only deployment:** Your workload runs entirely in the browser without a separate backend
- **Static website hosting:** Uses Azure Storage static websites for hosting
- **Azure Front Door:** Provides global CDN, SSL termination, and routing
- **Managed services:** Leverages Entra ID, Key Vault, and Managed Identity for security
- **Single deployment process:** Uses provided PowerShell scripts for deployment

Azure services used

Azure Front Door

[Azure Front Door](#) serves as the global entry point for your workload. It provides:

- **Global load balancing:** Routes users to the closest edge location
- **SSL termination:** Handles HTTPS certificates automatically
- **Web Application Firewall (WAF):** Protects against common web vulnerabilities
- **Caching:** Improves performance by caching static assets at edge locations

For the Extensibility Toolkit, Front Door routes traffic to your static website hosted on Azure Storage, ensuring high availability and low latency worldwide.

Azure Storage Account (Static Website)

An [Azure Storage Account with static website hosting](#) hosts your built workload application. This provides:

- **Cost-effective hosting:** Pay only for storage and bandwidth used
- **Automatic scaling:** Handles traffic spikes without configuration
- **Global availability:** Content is served from Azure's global storage infrastructure
- **File-based deployment:** Upload files to deploy versions

The Extensibility Toolkit builds your React application into static HTML, JavaScript, CSS, and assets that are deployed directly to the storage account.

Azure Key Vault

[Azure Key Vault](#) manages sensitive configuration and secrets:

- **Client secrets:** Stores Entra ID application secrets securely
- **API keys:** Manages external service API keys
- **Configuration:** Stores environment-specific settings
- **Certificates:** Manages SSL certificates if needed

Your workload accesses Key Vault through Managed Identity for secure, credential-free authentication.

Azure Entra ID (Azure Active Directory)

[Azure Entra ID](#) provides authentication and authorization:

- **App registration:** Registers your workload as an Entra application
- **OAuth 2.0 flows:** Handles user authentication with Fabric
- **API permissions:** Manages access to Fabric and Microsoft Graph APIs
- **Token management:** Provides secure access tokens for API calls

Managed Identity

[Managed Identity](#) provides secure service-to-service authentication:

- **No credential management:** Eliminates need to store secrets in code
- **Token handling:** Azure handles token acquisition and renewal
- **Secure access:** Connects to Key Vault and other Azure services securely
- **Identity lifecycle:** Tied to your Azure resources for cleanup

Deployment process

Prerequisites

Before deploying to Azure, ensure you have:

- Azure subscription with appropriate permissions
- [Azure CLI](#) installed and authenticated
- Fabric Extensibility Toolkit repository cloned locally
- Your workload built using `.\scripts\Build\BuildRelease.ps1`

Using the deployment script

The Extensibility Toolkit includes a PowerShell deployment script at `scripts\Deploy\DeployToAzureWebApp.ps1` that automates the deployment process.

Basic deployment

PowerShell

```
# Deploy to an existing Azure Web App
.\scripts\Deploy\DeployToAzureWebApp.ps1 -WebAppName "my-fabric-workload" -
ResourceGroupName "fabric-workload-rg"
```

Additional deployment options

PowerShell

```
# Deploy to staging slot with custom settings
.\scripts\Deploy\DeployToAzureWebApp.ps1 `
  -WebAppName "my-fabric-workload" `
  -ResourceGroupName "fabric-workload-rg" `
  -SlotName "staging" `
  -Force $true `
  -CreateBackup $true `
  -RestartAfterDeploy $true
```

Deployment script parameters

 Expand table

Parameter	Description	Required	Default
WebAppName	Name of the Azure Web App to deploy to	Yes	-
ResourceGroupName	Resource group containing the Web App	Yes	-
ReleasePath	Path to built application files	No	..\..\release\app
DeploymentMethod	Deployment method (ZipDeploy, FTP, LocalGit)	No	ZipDeploy
SlotName	Deployment slot for staging	No	-
Force	Skip confirmation prompts	No	\$false
CreateBackup	Create backup before deployment	No	\$true
RestartAfterDeploy	Restart app after deployment	No	\$true

Deployment script features

The deployment script provides comprehensive deployment capabilities:

Validation and safety

- **Prerequisites check:** Validates Azure CLI installation and authentication
- **Resource validation:** Confirms target Web App exists and is accessible
- **Build validation:** Ensures release directory contains required files
- **Backup creation:** Creates deployment backup for rollback capabilities

Deployment workflow

- **ZIP package creation:** Compresses built application into deployment package
- **Size reporting:** Shows deployment package size for verification
- **Progress monitoring:** Provides real-time deployment status updates
- **Error handling:** Detailed error messages with troubleshooting guidance

Post-deployment

- **Health check:** Validates deployed application responds correctly
- **URL reporting:** Provides direct link to deployed application
- **Manifest guidance:** Shows next steps for manifest upload to Fabric
- **Timing metrics:** Reports total deployment duration

Manual publishing

You can also deploy your frontend application manually using Azure PowerShell commands:

Build your application

First, build your frontend application for the test environment:

PowerShell

```
npm run build:test
```

Create deployment package

1. Navigate to the `build\Frontend` folder in your project
2. Select all files and the `assets` folder under the build directory
3. Create a `.zip` file containing all selected files

Deploy using Azure PowerShell

PowerShell

```
# Connect to Azure
Connect-AzAccount

# Set your subscription context
Set-AzContext -Subscription "<subscription_id>"

# Deploy the zip file to your web app
```

```
Publish-AzWebApp -ResourceGroupName <resource_group_name> -Name <web_app_name> -  
ArchivePath <zip_file_path>
```

Manual deployment parameters

 Expand table

Parameter	Description	Example
<subscription_id>	Your Azure subscription ID	12345678-1234-1234-1234- 123456789012
<resource_group_name>	Resource group containing your web app	fabric-workload-rg
<web_app_name>	Name of your Azure web app	my-fabric-workload
<zip_file_path>	Full path to your deployment zip file	C:\path\to\deployment.zip

Note

Manual deployment requires you to have the appropriate Azure permissions and the [Azure PowerShell module](#) installed on your machine.

Security considerations

Authentication flow

Your workload authenticates with Fabric using the standard OAuth 2.0 flow:

1. User accesses your workload through Fabric
2. Fabric redirects to your Azure-hosted application
3. Your app redirects to Entra ID for authentication
4. Entra ID returns authentication token
5. Your app uses token to access Fabric APIs

Secure configuration

- **Environment variables:** Store configuration in Azure App Settings, not in code
- **Key Vault integration:** Access secrets through Managed Identity
- **HTTPS only:** Enforce HTTPS for all communication

- **CORS configuration:** Configure appropriate CORS policies for Fabric domains

Best practices

- **Least privilege:** Grant minimal required permissions to Managed Identity
- **Secret rotation:** Regularly rotate secrets stored in Key Vault
- **Network security:** Use Private Endpoints where possible
- **Monitoring:** Enable Application Insights for security monitoring

Configuration after deployment

Azure App Service configuration

After deployment, configure your Azure App Service:

Bash

```
# Set environment variables for your workload
az webapp config appsettings set --name "my-fabric-workload" --resource-group
"fabric-workload-rg" --settings \
    "FABRIC_CLIENT_ID=your-client-id" \
    "FABRIC_TENANT_ID=your-tenant-id" \
    "KEY_VAULT_URL=https://your-keyvault.vault.azure.net/"
```

Key Vault setup

Store sensitive configuration in Key Vault:

Bash

```
# Store client secret
az keyvault secret set --vault-name "your-keyvault" --name "FabricClientSecret" --
value "your-client-secret"

# Store API keys
az keyvault secret set --vault-name "your-keyvault" --name "ExternalApiKey" --value
"your-api-key"
```

Managed Identity configuration

Configure Managed Identity for Key Vault access:

Bash


```
# Enable system-assigned managed identity
az webapp identity assign --name "my-fabric-workload" --resource-group "fabric-workload-rg"

# Grant access to Key Vault
az keyvault set-policy --name "your-keyvault" \
  --object-id "managed-identity-principal-id" \
  --secret-permissions get list
```

Manifest deployment

After deploying your application to Azure, you must upload the manifest package to Fabric:

Build manifest package

First, build the manifest package:

PowerShell

```
.\scripts\Build\BuildManifestPackage.ps1
```

This creates `release\ManifestPackage.1.0.0.nupkg` containing your workload manifest.

Upload to Fabric Admin Portal

1. Open the [Microsoft Fabric Admin Portal](#) 
2. Navigate to **Workload Management** > **Upload workload**
3. Upload your `ManifestPackage.1.0.0.nupkg` file
4. Configure workload settings and permissions
5. Activate the workload for your tenant

Update manifest for Azure deployment

Ensure your `Product.json` manifest references your Azure-deployed application:

JSON

```
{
  "name": "Product",
  "displayName": "Your_Workload_Name",
  "frontend": {
    "path": "https://your-workload.azurewebsites.net"
```

```
}  
}
```

Monitoring and troubleshooting

Application Insights

Enable Application Insights for monitoring:

- **Performance monitoring:** Track page load times and user interactions
- **Error tracking:** Monitor JavaScript errors and failed requests
- **Usage analytics:** Understand how users interact with your workload
- **Custom telemetry:** Add custom metrics for business logic

Common issues and solutions

Deployment failures

- **Authentication errors:** Verify Azure CLI login with `az account show`
- **Resource not found:** Confirm Web App name and resource group are correct
- **Permission denied:** Ensure your account has Contributor role on the resource group

Runtime issues

- **White screen:** Check browser console for JavaScript errors
- **Authentication failures:** Verify Entra ID app registration and redirect URIs
- **API call failures:** Check CORS configuration and API permissions

Performance optimization

- **Slow loading:** Enable compression and optimize bundle size
- **Caching issues:** Configure proper cache headers in Front Door
- **Geographic latency:** Ensure Front Door is properly configured for global routing

Related guides

- [How to publish and manage a workload](#) - Uploading manifest to Fabric
- [Setup Guide](#) - Initial development environment setup
- [Tutorial: Getting Started](#) - Getting started with the Extensibility Toolkit

Last updated on 12/04/2025

How to publish and manage your workload

This article describes how to upload and manage a workload in Microsoft Fabric. The **Workloads** page in **OneLake catalog** > **Govern** has two tabs:

- **Manage my tenant** - Browse and add published workloads to your tenant.
- **Publish** - Upload and manage your own workload packages.

Workloads enhance the usability of your service within a familiar workspace, eliminating the need to leave the Fabric environment. Fabric workloads increase user engagement and improve your application's discoverability in the Fabric marketplace. The Fabric workspace includes various components, known as Fabric items, which handle storage, analysis, and presentation of your data.

Prerequisites

To deploy a workload, you must have the following prerequisites:

- A Fabric account with admin permissions.
- The [NuGet package](#) workload solution that you want to deploy. For information about creating a workload solution, see [Create a new Item](#).

ⓘ Note

When DevGateway is connected to the Fabric backend, the developer workload takes precedence over the workload that's activated in the tenant. If you try to create an item in a workspace that's registered in the DevGateway configuration,, the Fabric backend calls your local workload instead of the activated workload.

Manage my tenant

The **Manage my tenant** tab lists all workloads that have been published and are available for your tenant. From this tab, admins can:

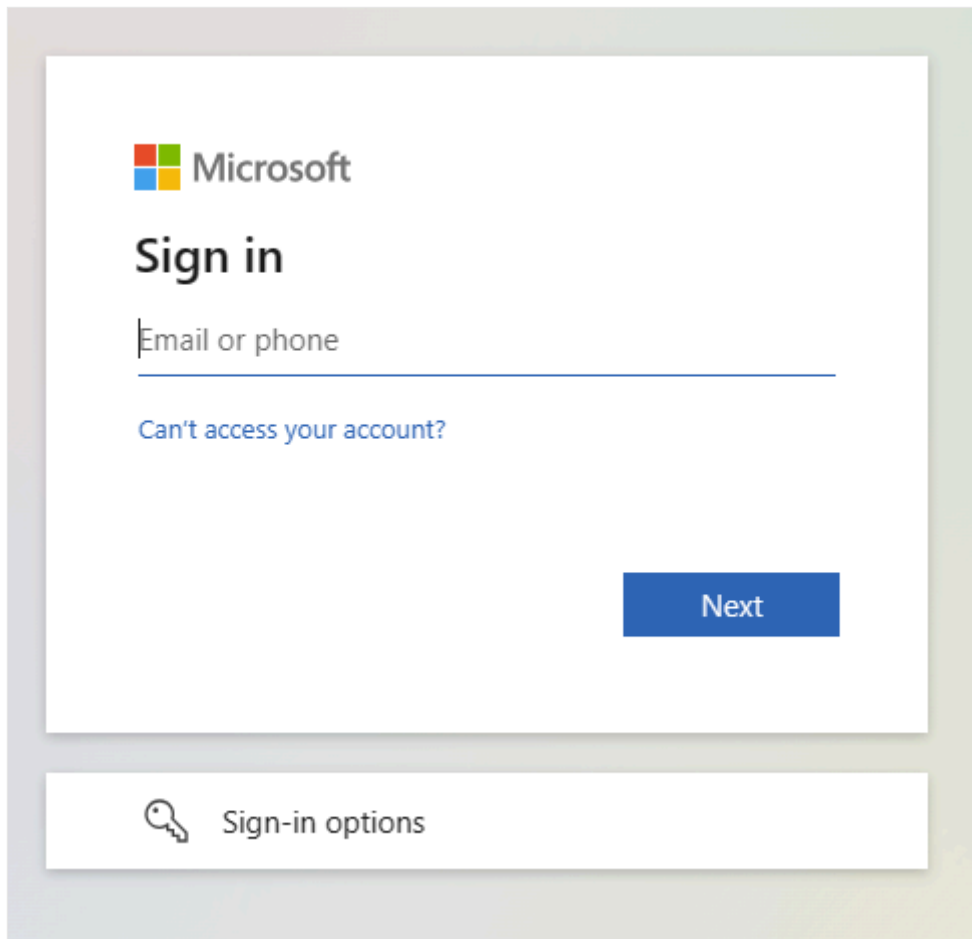
- **Browse available workloads** - View all workloads published to the Workload Hub, including their publisher and status.

- **Add workloads** - Select a workload to view its details, including an overview, documentation links, and licensing information. Select **Add Workload** to add it to your tenant.
- **Consent on behalf of the organization** - If you have sufficient Microsoft Entra permissions, select **Consent** to grant organization-wide consent for the workload's required permissions. This eliminates the need for individual users to consent when they first use the workload.

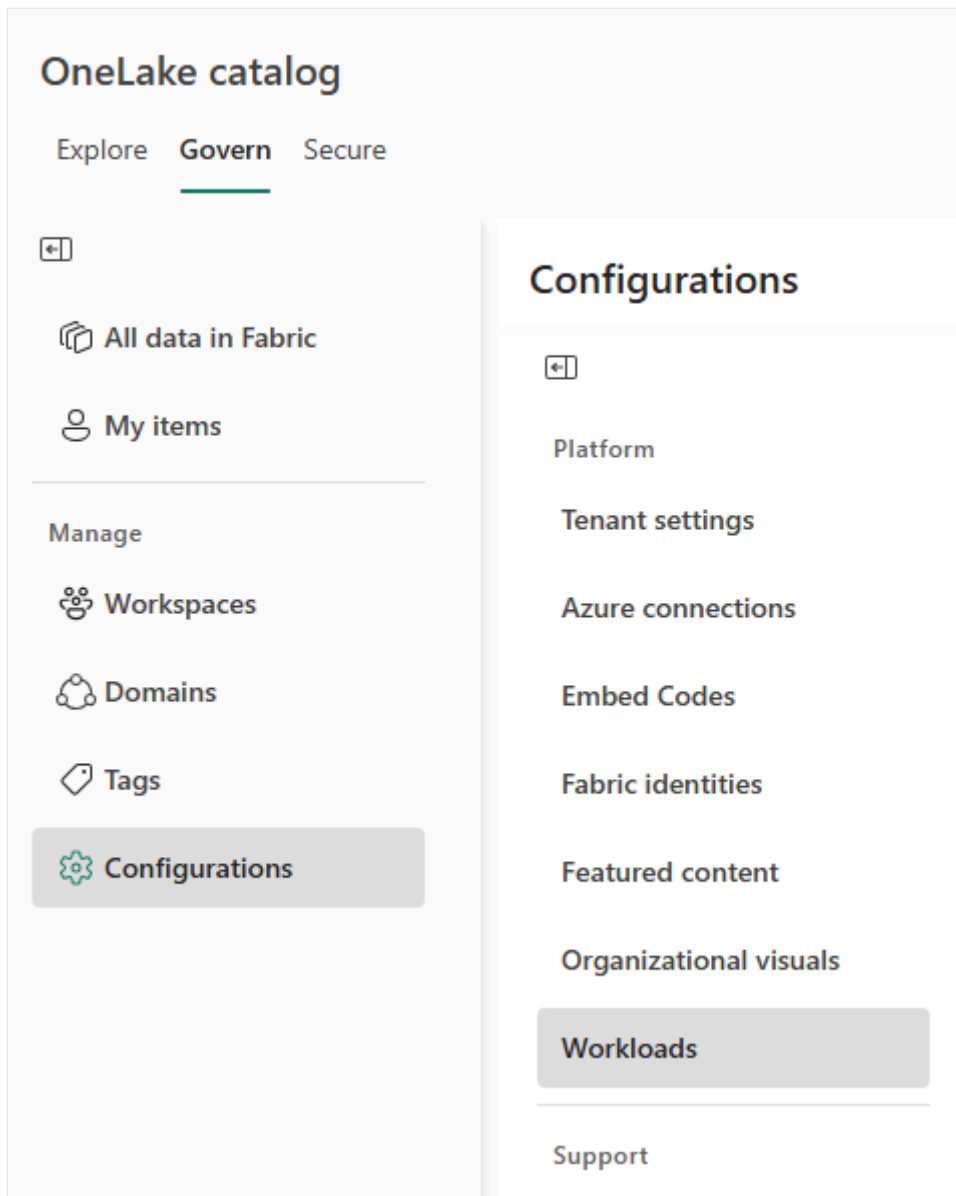
Upload a workload

Use the **Publish** tab to upload your own workload packages. To upload a workload to Microsoft Fabric:

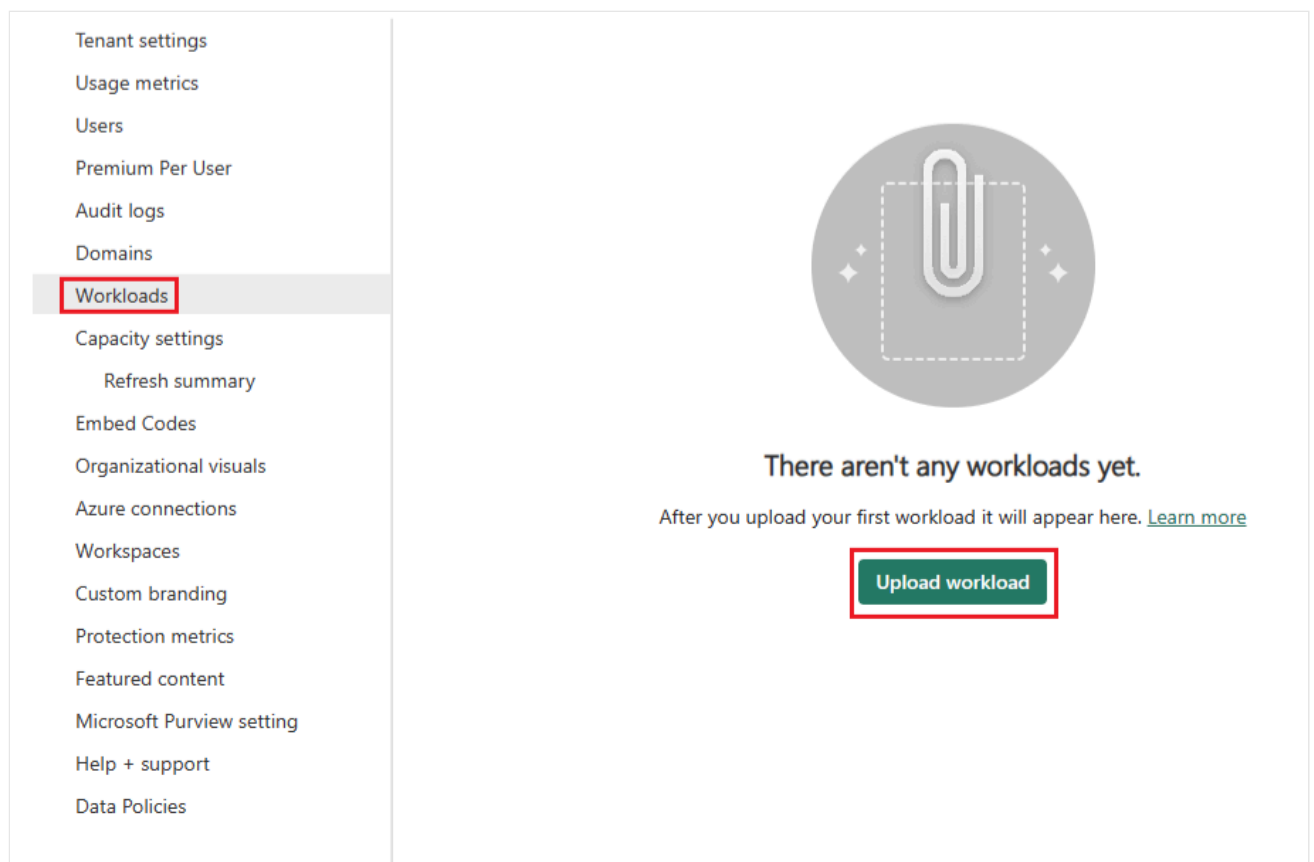
1. Sign in to [Fabric](#) with an admin account.



2. Go to **OneLake catalog > Govern > Configurations > Workloads**.



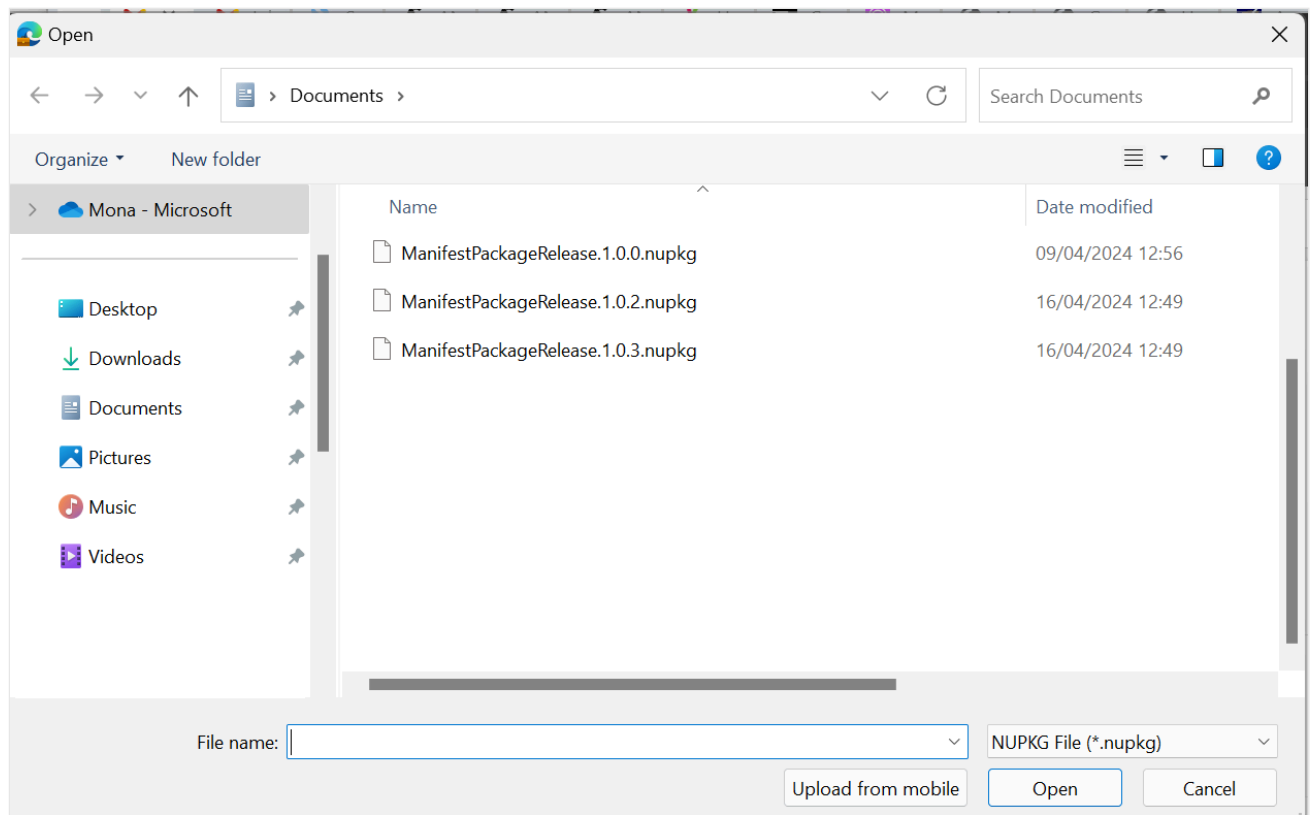
3. Select **Upload workload**.



4. Go to the NuGet package you want to upload and select **Open**.

i Important

Each uploaded NuGet package must have a unique version number. If you try to upload a package with a version that already exists, you get a **This package has already been uploaded** error. To resolve this issue, increment the minor version in your NuGet package (for example, change 1.0.0 to 1.1.0) and upload it again.



5. Select the workload.

6. Select the uploaded version.

Sample Workload

Add to tenant

Select Version*

Select a version to add to the entire tenant.

Workload version

Select a version

1.0.2

7. Select Add.

The version number is now listed, and **Status** is **Active in tenant**.

Workloads		
Manage all workloads uploaded to this tenant.		
Upload workload		
Workload name	Version	Status
Sample Workload	1.0.2	Active in tenant

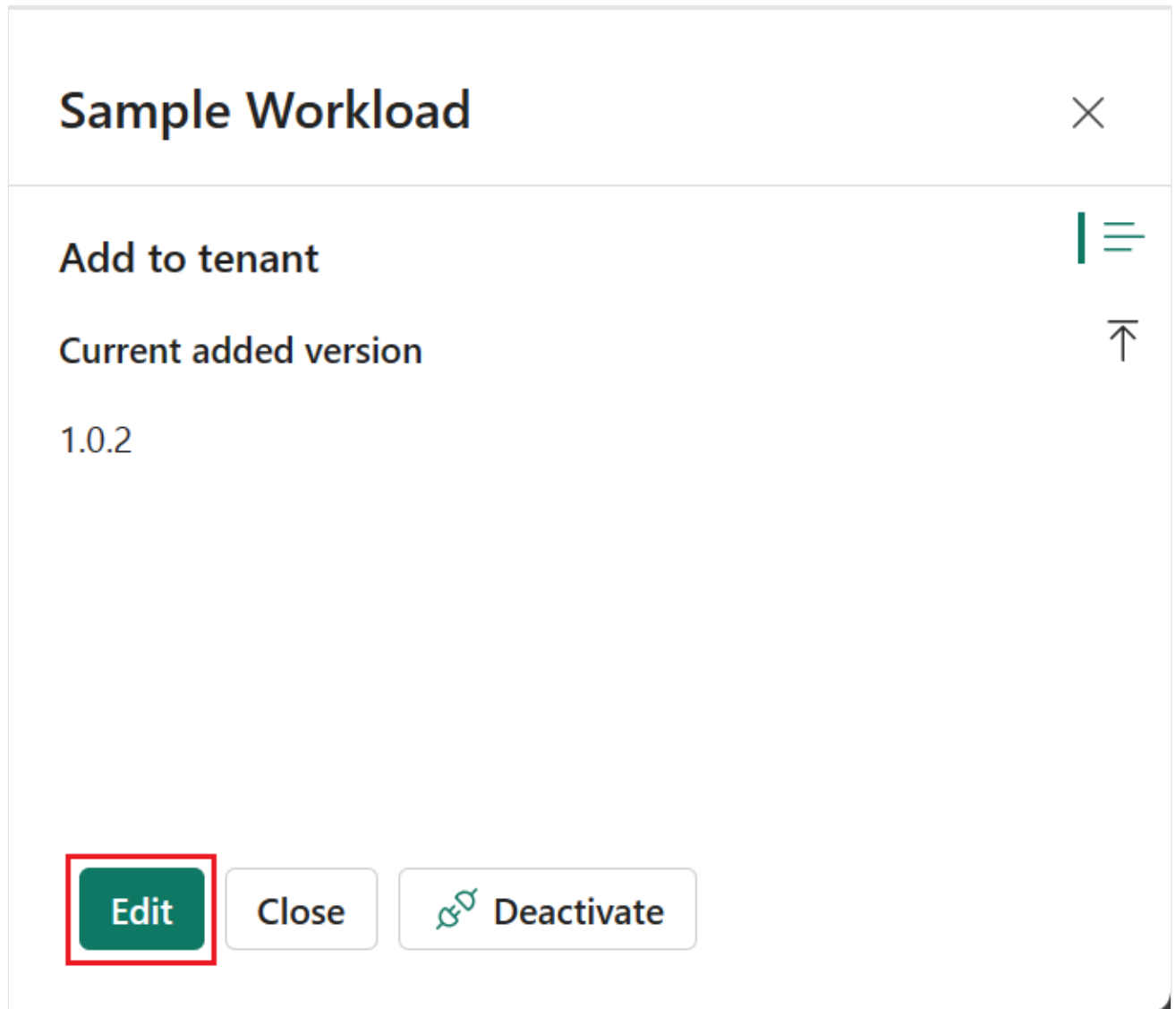
Manage a workload

After a workload is added, you can update, delete, or deactivate the workload.

Update a workload

To change to a different active version of a workload:

1. In **OneLake catalog**, select **Govern > Configurations > Workloads**, and then select a workload to activate.
2. On the **Add** tab, select **Edit**.



3. Select the version you want to activate and select **Add**.

Sample Workload

Add to tenant

Select Version*

Select a version to add to the entire tenant.

Workload version

Select a version

1.0.2

4. Select **Add** again to confirm the change.

Add version 1.0.3

The current added version will be removed and version 1.0.3 will be added to the entire tenant.

Add

Cancel

The new version number is now listed, and **Status** is **Active in tenant**.

Delete a workload

To delete a workload:

1. In **OneLake catalog**, select **Govern > Configurations > Workloads**, and then select the workload to delete.
2. On the **Uploads** tab, next to the version you want to delete, select the **Delete** icon.

Sample Workload

Uploads

↑

Version	Date	
1.0.2	16 Apr 24	Delete 
1.0.3	16 Apr 24	

You can't delete the active version of a workload. To delete an active version of a workload, first [deactivate](#) the workload.

Deactivate a workload

To deactivate a workload:

1. In **OneLake catalog**, select **Govern > Configurations > Workloads**, and then select the workload to deactivate.
2. On the **Add** tab, select **Deactivate**.

Sample Workload

Add to tenant

Current added version

1.0.2

Edit

Close

Deactivate

Related content

- [Setup guide](#)

Last updated on 09/29/2026

Supportability best practices

This tutorial provides guidance for Fabric Extensibility Toolkit developers on how to support their customers. It covers best practices for maintaining, saving, and correlating important request headers, as well as recommendations for logging, monitoring, and support.

Overview

The Fabric Extensibility Toolkit enables developers to create workloads that integrate seamlessly with the Microsoft Fabric platform. This document outlines the supportability practices that developers should follow to ensure effective troubleshooting and customer support.

Request Headers

Fabric exposes two important headers in requests made between customers and Fabric, which are crucial for supportability:

- **ActivityId:** A globally unique ID that can be used by you, the developer, to correlate the request with your systems when a Fabric user interacts with your workload.
- **RequestId:** A globally unique ID that helps Fabric to correlate your request with platform traces, which will help investigate issues and provide assistance. If you cannot determine the issue on your end using the `ActivityId`, you can provide the `RequestId` to Fabric support when reporting an issue. Ensure that you maintain and save the `RequestId` so it can be easily provided later.

Best Practices for Supportability

To effectively support your customers, it is essential to maintain, save, and correlate these IDs. Here are some best practices:

1. Logging

- **Retention of Logs:** Ensure that logs are retained for a sufficient period to allow for effective troubleshooting.
- **Detailed Logging:** Include the `ActivityId` and `RequestId` in all relevant log entries. This will help you trace the flow of a request through your system.
- **Application Insights Integration:** For Azure-hosted workloads, consider integrating with [Azure Application Insights](#) to automatically capture telemetry and correlation IDs.

2. Monitoring

- **Real-time Monitoring:** Implement real-time monitoring to detect and alert on issues as they occur. This includes monitoring request rates, error rates, and system performance.
- **Correlation with Headers:** Ensure that your monitoring tools can correlate logs and metrics using the `ActivityId` and `RequestId`. This will help you quickly identify and diagnose issues.
- **Azure Monitor:** Consider using [Azure Monitor](#) for comprehensive monitoring and alerting capabilities.

3. Support

- **Customer Support:** Train your support team to ask for the `ActivityId` and `RequestId` when dealing with customer issues. This will expedite the troubleshooting process.
- **Documentation:** Provide clear documentation to your customers on how to obtain and use these IDs when reporting issues.
- **Escalation Process:** Establish clear procedures for escalating issues to Fabric support using the `RequestId` when platform-level investigation is needed.

Example Workflow

1. **Log Entry:** When a request is received, log the `ActivityId` and `RequestId` along with other relevant information.
2. **Monitor:** Use monitoring tools to track the request in real time, correlating with the IDs.
3. **Troubleshoot:** If an issue arises, use the `ActivityId` and `RequestId` to trace the request through your logs and systems.
4. **Support:** When a customer reports an issue, ask for the `ActivityId` and `RequestId` to quickly locate the relevant logs and provide assistance. If the issue cannot be determined on your end, provide the `RequestId` to Fabric support for further investigation.

Implementation Considerations

For Frontend-Only Workloads

- Extract correlation headers from the workload client context when available
- Pass correlation IDs to any external API calls your workload makes
- Include correlation IDs in browser console logs for debugging

For Workloads with Backend Components

- Propagate correlation headers through your service architecture
- Implement structured logging that includes correlation IDs
- Set up monitoring dashboards that can filter by correlation IDs

Azure Integration

- **Application Insights:** Automatically captures request correlation and provides built-in dashboards
- **Azure Log Analytics:** Query logs using correlation IDs for troubleshooting
- **Azure Alerts:** Set up alerts based on error patterns or performance metrics

Customer Support Guidelines

For Your Customers

Provide customers with clear instructions on how to report issues:

1. **Activity ID:** Can often be found in browser developer tools console logs
2. **Request ID:** May be displayed in error messages or available through support tools
3. **Timestamp:** When the issue occurred
4. **Context:** What operation they were performing when the issue occurred

For Your Support Team

Train your support team to:

1. **Collect Correlation IDs:** Always ask for `ActivityId` and `RequestId` when investigating issues
2. **Use Diagnostic Tools:** Leverage your logging and monitoring systems to trace issues
3. **Escalate When Needed:** Know when to escalate to Fabric support with the `RequestId`
4. **Document Solutions:** Build a knowledge base of common issues and their correlation patterns

Conclusion

By following these best practices, developers can effectively support their customers on the Fabric platform, ensuring a seamless and efficient troubleshooting process. Proper logging,

monitoring, request management, and support protocols are essential to maintaining high service quality and customer satisfaction.

For more detailed implementation guidance, refer to:

- [Azure Application Insights documentation](#)
- [Azure Monitor documentation](#)
- [Architecture Overview](#)
- [Authentication Guidelines](#)

Last updated on 12/15/2025

How to validate your workload for publishing

This article explains how to use the Fabric Extensibility Toolkit Validator to ensure your workload meets all publishing requirements before submitting your publishing request form. The validator helps identify compliance issues early in the development process, reducing review time and improving approval success rates.

Overview

The Fabric Extensibility Toolkit Validator is a command-line tool designed to help partners validate their workloads against Microsoft Fabric publishing requirements. The tool combines automated testing with manual guided tests to provide comprehensive validation coverage.

Validation Types

The validator provides two types of validation:

- **Automated Tests:** Programmatic checks that scan your workload for compliance issues automatically
- **Manual Guided Tests:** Step-by-step instructions for validation tasks that require human verification

Prerequisites

Before using the validation tool, ensure you have:

- A completed workload ready for validation
- Access to the Fabric Extensibility Toolkit Validator repository
- Command-line interface access on your development machine
- Your workload properly configured and deployable

Accessing the Validator

The Fabric Extensibility Toolkit Validator is available through GitHub:

 **Repository:** <https://github.com/microsoft/fabric-extensibility-toolkit-validator> 

For detailed installation and usage instructions, see [Microsoft Fabric Extensibility Toolkit Validator](#).

Visit the repository to:

- Download the latest version of the validator
- Review available command-line parameters
- Access documentation for tool usage
- Check system requirements and setup instructions

Using the Validator

Command-Line Interface

The validator operates as a command-line tool that you run against your workload. The specific parameters and usage instructions are available in the validator repository.

Refer to the repository documentation for:

- Complete parameter reference
- Command-line syntax
- Configuration options
- Output format details

Validation Workflow

Follow this general workflow when using the validator:

1. **Prepare Your Workload:** Ensure your workload is built and ready for testing
2. **Run Automated Tests:** Execute the validator's automated compliance checks
3. **Complete Manual Tests:** Follow guided instructions for manual verification tasks
4. **Review Results:** Analyze the validation report and identify any issues
5. **Address Findings:** Fix any compliance gaps identified by the validator
6. **Re-validate:** Run the validator again to confirm all issues are resolved
7. **Submit for Publishing:** Proceed with your publishing request once validation passes

Validation Coverage

The validator checks your workload against:

- **Publishing requirements for workloads:** Infrastructure, security, and compliance standards
- **Publishing requirements for items:** User experience and functionality requirements
- **Manifest compliance:** Proper configuration and required properties

- **Platform integration:** Correct implementation of Fabric platform features

Interpreting Results

Automated Test Results

The validator provides detailed reports on automated test results, including:

- **Pass/Fail Status:** Clear indication of test outcomes
- **Issue Descriptions:** Specific explanations of any compliance gaps
- **Remediation Guidance:** Suggestions for fixing identified issues
- **Severity Levels:** Priority indicators for addressing different types of issues

Manual Test Guidance

For manual validation tasks, the tool provides:

- **Step-by-step instructions:** Clear guidance for manual verification processes
- **Verification criteria:** Specific standards to check against
- **Documentation requirements:** Evidence needed to demonstrate compliance
- **Best practices:** Recommendations for optimal implementation

Best Practices

Pre-Validation Preparation

- **Complete development:** Ensure your workload is feature-complete before validation
- **Test thoroughly:** Conduct your own testing before running the validator
- **Review requirements:** Familiarize yourself with publishing requirements beforehand
- **Prepare documentation:** Ensure all required documentation is ready and accessible

Validation Execution

- **Run regularly:** Use the validator throughout development, not just at the end
- **Address issues promptly:** Fix identified problems as soon as possible
- **Document changes:** Keep track of modifications made based on validation feedback
- **Verify fixes:** Re-run validation after making changes to confirm resolution

Troubleshooting

Common Issues

- **Tool setup problems:** Refer to the repository documentation for installation guidance
- **Permission issues:** Ensure you have appropriate access rights to run the validator
- **Configuration errors:** Verify your workload configuration matches expected formats
- **Environment issues:** Check that your development environment meets tool requirements

Getting Help

- **Repository Documentation:** Check the validator repository for detailed guidance
- **Issue Reporting:** Use the repository's issue tracker to report problems or ask questions
- **Community Support:** Engage with other developers using the validator
- **Microsoft Support:** Contact your Microsoft partner representative for additional assistance

Related Content

- [Publishing Requirements](#) - General requirements overview
- [Publishing Requirements Cross tenants](#)
- [Publishing requirements for workloads](#) - Workload-specific standards
- [Publishing requirements for items](#) - Item-specific requirements
- [Publishing Overview](#) - High-level publishing process
- [Manifest overview](#) - Workload manifest configuration

Last updated on 12/15/2025

How-To: Define item creation

The item creation is standardized through a dedicated Fabric control that guides users through the process. This control allows users to select the workspace where the item is created, assign Sensitivity labels, and configure other relevant settings. By using this standardized experience, you no longer need to handle the complexities of item creation yourself or worry about future changes to the process. Additionally, this approach enables item creation directly from your workload page, providing a seamless and integrated user experience.

The configuration for the standard item creation experience can be found in the [Product.json](#) file, specifically within the `create` section under `createItemDialogConfig`. It allows you to define event handlers for failure and success. Here's a snippet for reference:

JSON

```
{
  "name": "Product",
  "version": "1.100",
  "displayName": "Workload_Display_Name",
  "createExperience": {
    "description": "Workload_Description",
    "cards": [
      {
        "title": "CreateHub_Card_2_Title",
        "description": "CreateHub_Card_2_Description",
        "icon": {
          "name": "assets/images/HelloWorldItem-icon.png"
        },
        "icon_small": {
          "name": "assets/images/HelloWorldItem-icon.png"
        },
        "availableIn": [
          "home",
          "create-hub",
          "workspace-plus-new",
          "workspace-plus-new-teams"
        ],
        "itemType": "SampleItem",
        "createItemDialogConfig": {
          "onCreationFailure": { "action": "item.onCreationFailure" },
          "onCreationSuccess": { "action": "item.onCreationSuccess" }
        }
      }
    ]
  }
}
```

This configuration defines the dialog used for creating new items, including the fields, labels, and button text.

Last updated on 12/15/2025

How-To: Store item definition (state)



Summarize this article for me

This article explains what item definitions are, their role in CI/CD and public APIs, and how to implement them in your custom workload.

What is an item definition?

The item definition captures the functional state of a workload item. It contains all the information necessary to recreate the item's functional state in a new workspace or environment.

Control plane vs. data plane

It's important to understand that the item definition represents the **control plane** – the configuration and metadata of an item – while OneLake represents the **data plane** where the actual data is stored.

The control plane contains the instructions and settings needed to recreate the item's functional state in a new environment without requiring the actual data. This separation enables scenarios like:

- Deploying item configurations across environments (Dev → Test → Prod) without moving the underlying data
- Storing item configurations in source control for version tracking
- Recreating items in new workspaces with different data sources

For example, a lakehouse item definition contains the schema, transformations, and configuration settings, but not the actual data stored in OneLake. This allows you to deploy the lakehouse configuration to a new workspace where it can then connect to appropriate data sources for that environment.

An item definition consists of two main components:

- **Workload-owned content** – Configuration and metadata provided by your workload through the update API that define the functional state of the item. This content should be sufficient to recreate the item's functional state in a new workspace or stage.
- **Fabric system-owned content** – Platform-managed files (for example, `.platform`) required by Fabric to host and manage the item.

Item definition guidelines

Your workload should follow these guidelines when implementing item definitions:

Structure requirements:

1. The content should be human-readable (for example, JSON, YAML).
2. Item definition contains a reasonable number of files (up to 5).
3. File types should be text-based files (avoid images).

For more information about item definition structure and best practices, see [Overview of item definitions](#).

Item definitions in CI/CD and public APIs

Item definitions play a critical role in CI/CD workflows and public API interactions:

CI/CD integration

During CI/CD processes, the item definition is carried across environments. The Git integration feature in Fabric uses item definitions to:

- Store item configurations in source control
- Track changes to item states over time
- Deploy items across different workspaces and environments
- Enable collaborative development workflows

The workload-owned content must be sufficient to recreate the item's functional state when deploying to a new workspace or stage in the CI/CD pipeline. For more information about Git integration, see [Overview of Fabric Git integration](#).

Public APIs

Item definitions are exposed through Fabric's public APIs, allowing external tools and automation scripts to:

- Read the current state of a workload item
- Update item configurations programmatically
- Integrate with external deployment tools
- Automate item management tasks

Environment-specific values

If your item definition contains environment-specific values that might change between stages (for example, Test → Prod), we highly encourage you to use the [Variables Library](#).

Examples of environment-specific values include:

- Connections to databases
- References to other Fabric items
- Workspace-specific identifiers

When you don't use variables, environment-specific values stored directly in the item definition are preserved as-is when the item is carried from the source workspace to the target workspace. The values don't automatically adapt to the target environment. You need to manually update them in the target workspace (for example, replacing Test connections with Prod connections). Any such changes are reflected as differences between source and target.

For detailed guidance on implementing variables in your custom workload, see [How-To: Use variables in custom workloads](#).

How to store an item definition

The Fabric SDK provides abstractions to simplify storing item definitions. While the example shown here demonstrates storing a single object as a single definition part, item definitions can contain multiple parts to organize different aspects of your item's configuration.

For example, you might split your item definition into separate parts such as:

- A main configuration file containing core settings
- A schema definition file
- Connection or data source configurations
- User preferences or display settings

You can find the code for storing item definition in the `saveItemDefinition` method within the [SampleWorkloadEditor.tsx](#) file. Here's the method content for reference:

TypeScript

```
async function SaveItem(definition?: HelloWorldItemDefinition) {
  var successResult = await saveItemDefinition<HelloWorldItemDefinition>(
    workloadClient,
    editorItem.id,
    definition || editorItem.definition);
  setIsUnsaved(!successResult);
}
```

This method demonstrates how to persist the definition of an item using the SDK. It's a simplified version that uses a single object that is stored in a single definition part. For most users, this method is a good starting point.

If you need to store more than one definition part, you can implement this yourself. Review the `saveItemDefinition` method to understand how parts are handled and how to add more parts.

Implementation considerations

When implementing item definition in your item, keep these points in mind:

- Ensure your definition content is human-readable and uses text-based formats like JSON or YAML.
- Limit the number of files in your item definition (up to five files).
- Consider using the [Variables Library](#) for environment-specific values to enable smoother transitions between Test, Staging, and Production environments.
- Review the [Git integration guidelines](#) to ensure your item definition structure complies with Fabric's CI/CD requirements.

⚠ Note

All definition parts that are stored aren't parsed as is and not validated. Schema-based validation for both the structure and the definition parts themselves (for example, JSON) is not supported at the moment and workloads have to make sure they can handle it.

Versioning and backward compatibility

As your item evolves, the structure and content of your item definition parts might change over time. We encourage you to maintain backward compatibility with older versions or design a dedicated experience within your item to handle version differences.

Fabric can't guarantee that older versions of item definitions won't be restored or remain in the system for extended periods without the item being opened. This can happen when users restore items from Git history, deploy from older environments, or when items remain unopened while you publish updates.

Maintain backward compatibility:

Design your item to recognize and handle multiple versions. Include a version identifier in your item definition and detect the version when loading:

TypeScript

```
interface ItemDefinitionV1 {
  version: "1.0";
  settings: string;
}

interface ItemDefinitionV2 {
  version: "2.0";
  settings: SettingsObject;
  newFeature: string;
}

async function loadItemDefinition(rawDefinition: any): Promise<ItemDefinitionV2> {
  if (rawDefinition.version === "1.0") {
    return migrateV1ToV2(rawDefinition);
  }
  return rawDefinition as ItemDefinitionV2;
}
```

Design a migration experience:

Alternatively, create a user experience within your item that detects version mismatches and guides users through updating. Display notifications when older versions are detected and provide an upgrade process.

Best practices:

- Always include version information in your item definition from the start
- Test that older item definitions can be successfully loaded and migrated
- Avoid removing required fields; keep deprecated fields as optional for backward compatibility
- Use sensible defaults for new fields when loading older versions

Related content

- [Overview of item definitions](#)
- [Overview of Fabric Git integration](#)
- [How-To: Use variables in custom workloads](#)
- [Get started with Variables Library](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

How-To: Use variables in custom workloads



Summarize this article for me

This article explains how to use the Variables Library in your item to manage environment-specific values in item definitions. Using variables enables smooth transitions between different environments (for example, Dev → Test → Prod) without manual updates.

Why use variables?

When your item definition contains environment-specific values, those values might need to change as you promote items across different stages. Without variables, these values are preserved as-is when the item is carried from the source workspace to the target workspace, requiring manual updates.

For example, if your item references a Lakehouse in Fabric without using variables, it remains connected to the original lakehouse even after deployment. A lakehouse that exists in the same workspace in your Dev environment might need to reference a different lakehouse in your Test or Prod workspace, but the hardcoded reference won't automatically adapt.

Environment-specific values that benefit from variables include:

- **Database connections** – Different connection strings for Test and Production databases
- **Fabric item references** – References to Lakehouse in Fabric, semantic models, or other items that differ across environments (for example, Dev lakehouse vs. Prod lakehouse). Learn more about [item reference variable types](#).
- **Workspace identifiers** – Workspace-specific settings that change between stages
- **API endpoints** – Different service URLs for development, staging, and production
- **Configuration settings** – Environment-dependent settings like batch sizes or timeout values

How variables work

The Variables Library in Fabric provides a centralized way to manage values that change across environments. Instead of hardcoding environment-specific values directly in your item definition, you reference variables. When you deploy the item to a different environment, you can update the variable values in that environment without modifying the item definition itself.

Variables Library supports various data types including strings, numbers, and boolean values. For a complete list of supported types, see [Supported types in variable libraries](#).

This approach:

- Reduces the differences between source and target item definitions
- Minimizes manual updates during deployment
- Centralizes environment-specific configuration management
- Enables consistent deployment processes across environments

Using variables in your workload

Select variables using the Variable Picker

The Fabric client SDK provides access to the Variables Library through the `workloadClient` API. The recommended approach is to use the Variable Picker dialog, which allows users to browse and select variables from the Variables Library.

Here's an example of using the Variable Picker dialog:

TypeScript

```
// Open the variable picker dialog
const result = await workloadClient.variableLibrary.openVariablePickerDialog({
  workspaceObjectId: item.workspaceId,
  filters: undefined // Optional filters to restrict variable selection see
supported types below
});

// Check if variables were selected
if (result.selectedVariables && result.selectedVariables.length > 0) {
  const variableReference = result.selectedVariables[0].variableReference;
  // Store the variable reference in your item definition
  // See the item definition article for details on storing definitions
}
```

Store variable references in item definitions

When storing your [item definition](#), store the variable reference rather than hardcoded values. The variable reference is a string that Fabric uses to resolve the variable value at runtime:

TypeScript

```
// Store the variable reference in your item definition
const itemDefinition = {
  itemReference: variableReference, // e.g., "{{var:ItemReference}}"
  // other configuration...
};
```

When the item is loaded in a different environment, Fabric automatically resolves the variable reference using the variable values defined in that environment.

Handle variable lifecycle

Variable references stored in your item definition aren't guaranteed to remain valid over time. Users can delete variables or change their values at any time, independent of your workload items. Your item should handle these scenarios gracefully.

Variables can be deleted

A variable reference stored in your item definition might point to a variable that no longer exists. This can happen when:

- Users delete variables from the Variables Library
- Items are deployed to environments where the referenced variables haven't been created
- Workspace configurations change over time

Recommendation: Design a user experience that detects missing variables and guides users to either recreate the variable or select a different one. Display clear error messages or warnings when variable references can't be resolved.

Variables can change

Variable values can change independently of your item. The value stored in a variable today might be different tomorrow if a user updates it in the Variables Library.

Recommendation: Always resolve variable references at runtime when you need the value, rather than caching variable values. This ensures your item uses the most current value. For example, resolve variables when:

- Opening or initializing an item
- Executing a job or operation that depends on the variable
- Validating configuration before running a process

Example: Handle missing variables

TypeScript

```
async function getVariableValue(  
  workloadClient: any,  
  variableReference: string,  
  item: any
```

```

): Promise<string | null> {
  try {
    // Resolve the variable reference to get current value
    const value = await workloadClient.variableLibrary.resolveVariableReference(
      variableReference,
      item.workspaceId
    );
    return value;
  } catch (error) {
    // Variable doesn't exist or can't be resolved
    console.warn(`Variable reference ${variableReference} could not be resolved:`,
error);

    // Provide user feedback - example using a notification
    showNotification({
      type: 'warning',
      message: `Variable not found. Please select a valid variable.`,
      action: {
        label: 'Select Variable',
        onClick: () => openVariablePicker()
      }
    });

    return null;
  }
}

```

Example implementation

Here's a complete example showing how to use the Variable Picker in a custom workload item editor:

TypeScript

```

interface ItemConfiguration {
  lakehouseReference?: string;
  apiEndpoint?: string;
}

async function handleSelectVariable(
  workloadClient: any,
  item: any,
  configField: keyof ItemConfiguration
): Promise<void> {
  try {
    // Open the variable picker dialog
    const result = await workloadClient.variableLibrary.openVariablePickerDialog({
      workspaceObjectId: item.workspaceId,
      filters: undefined
    });
  }
}

```



```

// Check if a variable was selected
if (result.selectedVariables && result.selectedVariables.length > 0) {
  const variableReference = result.selectedVariables[0].variableReference;

  // Update the item configuration with the variable reference
  const updatedConfig: ItemConfiguration = {
    ...item.definition.configuration,
    [configField]: variableReference
  };

  // Save the updated configuration to the item definition
  await saveItemDefinition(workloadClient, item.id, {
    ...item.definition,
    configuration: updatedConfig
  });
}
} catch (error) {
  console.error('Failed to select variable:', error);
}
}

// Example usage in a React component button handler
<Button
  onClick={() => handleSelectVariable(
    workloadClient,
    item,
    'lakehouseReference'
  )}
>
  Select Variable
</Button>

```

This example demonstrates:

1. Opening the Variable Picker dialog with the current workspace context
2. Retrieving the selected variable reference from the dialog result
3. Storing the variable reference in the item definition
4. Integrating with a UI component to allow users to select variables

For more information about storing item definitions, see [How-To: Store item definition \(state\)](#).

Related content

- [Get started with Variables Library](#)
- [How-To: Store item definition \(state\)](#)
- [Overview of Fabric Git integration](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

How-To: Store data in OneLake

Every Fabric item has access to OneLake storage, which provides a secure, scalable way to store files associated with your item. This guide explains how to upload files to your Fabric item using the `FabricPlatformAPIClient` and the underlying `OneLakeClient`.

Understanding item storage in Fabric

Each item in Fabric has its own dedicated storage area in OneLake. This storage is organized into folders, with the main folders being:

- **Files** - For storing general files and documents
- **Tables** - For storing table data

Prerequisites

Before uploading files to your item, you need:

- A valid Fabric workspace
- An existing item where you want to upload the file
- Appropriate permissions to write to the item

Creating the FabricPlatformAPIClient

First, create a `FabricPlatformAPIClient` instance:

TypeScript

```
import { getWorkloadClient } from "../controller/WorkloadClient";
import { FabricPlatformAPIClient } from "../clients/FabricPlatformAPIClient";

// Create client using the current user's context
const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());
```

Example: Uploading a text file to an item

Here's how to upload a text file to an item using the OneLake client:

TypeScript

```
async function uploadTextFileToItem(
  workspaceId: string,
```

```

    itemId: string,
    fileName: string,
    content: string
  ) {
    try {
      // Get the FabricPlatformAPIClient
      const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());

      // Access the OneLake client
      const oneLakeClient = fabricClient.oneLake;

      // Generate the file path in OneLake for this item
      // This follows the pattern: workspaceId/itemId/Files/fileName
      const filePath = oneLakeClient.constructor.getFilePath(workspaceId, itemId,
fileName);

      // Write the text content to the file
      await oneLakeClient.writeFileAsText(filePath, content);

      console.log(`Successfully uploaded ${fileName} to item ${itemId}`);
      return true;
    } catch (error) {
      console.error("Error uploading file to item:", error);
      throw error;
    }
  }
}

```

Example: Uploading a binary file to an item

For binary files like images or PDFs, you need to convert the file to base64 first:

TypeScript

```

async function uploadBinaryFileToItem(
  workspaceId: string,
  itemId: string,
  fileName: string,
  fileData: ArrayBuffer // Binary file data
) {
  try {
    const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());
    const oneLakeClient = fabricClient.oneLake;

    // Convert binary data to base64
    const base64Content = arrayBufferToBase64(fileData);

    // Generate the file path
    const filePath = oneLakeClient.constructor.getFilePath(workspaceId, itemId,
fileName);

    // Write the binary content to the file
    await oneLakeClient.writeFileAsBase64(filePath, base64Content);
  }
}

```

```

        console.log(`Successfully uploaded binary file ${fileName} to item ${itemId}`);
        return true;
    } catch (error) {
        console.error("Error uploading binary file to item:", error);
        throw error;
    }
}

// Helper function to convert ArrayBuffer to base64
function arrayBufferToBase64(buffer: ArrayBuffer): string {
    let binary = '';
    const bytes = new Uint8Array(buffer);
    const len = bytes.byteLength;

    for (let i = 0; i < len; i++) {
        binary += String.fromCharCode(bytes[i]);
    }

    return btoa(binary);
}

```

Example: Uploading a file from the browser

If you're building a web interface, you can use this function to handle file uploads from a file input:

TypeScript

```

async function handleFileUpload(
    workspaceId: string,
    itemId: string,
    fileInputElement: HTMLInputElement
) {
    if (!fileInputElement.files || fileInputElement.files.length === 0) {
        console.warn("No file selected");
        return false;
    }

    const file = fileInputElement.files[0];
    const fileName = file.name;

    try {
        // Read the file as ArrayBuffer
        const fileBuffer = await readFileAsArrayBuffer(file);

        // Upload based on file type
        if (file.type.startsWith('text/')) {
            // For text files, convert to string and upload as text
            const textDecoder = new TextDecoder();
            const textContent = textDecoder.decode(fileBuffer);

```

```

        return await uploadTextFileToItem(workspaceId, itemId, fileName, textContent);
    } else {
        // For binary files, upload as base64
        return await uploadBinaryFileToItem(workspaceId, itemId, fileName,
fileBuffer);
    }
} catch (error) {
    console.error("Error processing file upload:", error);
    throw error;
}
}

// Helper function to read file as ArrayBuffer
function readFileAsArrayBuffer(file: File): Promise<ArrayBuffer> {
    return new Promise((resolve, reject) => {
        const reader = new FileReader();
        reader.onload = () => resolve(reader.result as ArrayBuffer);
        reader.onerror = reject;
        reader.readAsArrayBuffer(file);
    });
}

```

Managing files in your item

Once files are uploaded, you can also:

Check if a file exists

TypeScript

```

async function checkFileExists(workspaceId: string, itemId: string, fileName:
string) {
    const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());
    const filePath = fabricClient.oneLake.constructor.getFilePath(workspaceId, itemId,
fileName);

    return await fabricClient.oneLake.checkIfFileExists(filePath);
}

```

Read file content

TypeScript

```

async function readTextFile(workspaceId: string, itemId: string, fileName: string) {
    const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());
    const filePath = fabricClient.oneLake.constructor.getFilePath(workspaceId, itemId,
fileName);
}

```

```
return await fabricClient.oneLake.readFileAsText(filePath);
}
```

Delete a file

TypeScript

```
async function deleteFile(workspaceId: string, itemId: string, fileName: string) {
  const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());
  const filePath = fabricClient.oneLake.constructor.getFilePath(workspaceId, itemId,
    fileName);

  await fabricClient.oneLake.deleteFile(filePath);
  console.log(`File ${fileName} deleted successfully`);
}
```

Best practices for file uploads

- **Use appropriate file formats:** Consider the purpose of the file and use formats that are widely supported.
- **Handle errors gracefully:** Always include error handling for network issues or permission problems.
- **Validate file sizes:** Large files may take longer to upload and process.
- **Check permissions:** Ensure the user has appropriate permissions before attempting uploads.
- **Use file prefixes or folders:** For complex items with many files, consider organizing them in subfolders.

Using the OneLakeClientItemWrapper

For simplified access to item files, you can use the `OneLakeClientItemWrapper`:

TypeScript

```
async function uploadFileWithItemWrapper(item, fileName, content) {
  const fabricClient = FabricPlatformAPIClient.create(getWorkloadClient());

  // Create a wrapper for simpler access to this specific item
  const itemWrapper = fabricClient.oneLake.createItemWrapper({
    workspaceId: item.workspaceId,
    itemId: item.id
  });

  // Upload directly to the item (no need to specify paths)
  await itemWrapper.writeFileAsText(fileName, content);
}
```

```
// Read the file back
const fileContent = await itemWrapper.readFileAsText(fileName);

console.log(`File uploaded and read back: ${fileContent.substring(0, 50)}...`);
}
```

This wrapper simplifies file operations by automatically handling the full path construction.

Last updated on 12/15/2025

How-To: Create shortcuts in your item

This article explains how to implement shortcut functionality in your Microsoft Fabric workload item to reference data from lakehouses or external cloud storage sources like Amazon S3 without copying the data. Shortcuts allow your application to access and work with data from various sources through a unified interface, supporting the "single copy promise" of OneLake.

Understanding shortcuts in Microsoft Fabric

Shortcuts in Microsoft Fabric provide a way to access data across domains, clouds, and accounts by creating a virtual view of data residing in different locations. When you create a shortcut in your workload item, you're creating a reference to the data without physically copying it, which:

- Reduces storage costs and data duplication
- Ensures data consistency by always pointing to the source data
- Simplifies data access for users through a unified namespace
- Provides potential performance benefits through intelligent caching

Using the Shortcut Creation Control

The Fabric Extensibility Toolkit includes a Shortcut Creation Control that simplifies the process of creating shortcuts. This control provides a standardized user interface for shortcut creation, similar to the experience in native Fabric items.

To use the Shortcut Creation Control in your workload:

1. Import the control from the Extensibility Toolkit:

TypeScript

```
import { ShortcutCreationControl } from '@fabric/extensibility-toolkit';
```

2. Add the control to your component:

TypeScript

```
<ShortcutCreationControl
  supportedSourceTypes=[ 'Lakehouse', 'MyWorkloadName.MyItemName' ]}, // Specify
  which source types your workload supports
  supportedTargetTypes=[ 'Lakehouse', 'MyWorkloadName.MyItemName' ]} // Specify
  which source types your workload supports
/>
```

Creating shortcuts programmatically

If you need to create shortcuts programmatically, you can use the `OneLakeShortcutClient` which provides a simplified interface for shortcut management:

Setting up the OneLakeShortcutClient

TypeScript

```
import { OneLakeShortcutClient } from '../clients/OneLakeShortcutClient';
import { WorkloadClientAPI } from "@ms-fabric/workload-client";

// Initialize the client in your component or service
const shortcutClient = new OneLakeShortcutClient(workloadClient);
```

Creating different types of shortcuts

OneLake Shortcuts (Fabric to Fabric):

TypeScript

```
// Create a shortcut to another Fabric item (Lakehouse, KQL Database, etc.)
const oneLakeShortcut = await shortcutClient.createOneLakeShortcut(
  workspaceId,           // Current workspace ID
  itemId,                // Current item ID
  'SharedLakehouse',     // Shortcut name
  '/Files',              // Target path in current item
  'source-workspace-id', // Source workspace ID
  'source-item-id',      // Source item ID (Lakehouse, etc.)
  '/Tables/Customers'    // Optional: specific path in source item
);
```

Amazon S3 Shortcuts:

TypeScript

```
// Create an S3 shortcut
const s3Shortcut = await shortcutClient.createS3Shortcut(
  workspaceId,
  itemId,
  'S3CustomerData',      // Shortcut name
  '/Files',              // Target path in your item
  's3-connection-id',    // S3 connection ID (configured in Fabric)
  'my-bucket',           // S3 bucket name
  '/customer-data'       // Path within bucket
);
```

Azure Data Lake Storage Gen2 Shortcuts:

TypeScript

```
// Create an ADLS Gen2 shortcut
const adlsShortcut = await shortcutClient.createAdlsGen2Shortcut(
  workspaceId,
  itemId,
  'ADLSData',           // Shortcut name
  '/Files',            // Target path in your item
  'adls-connection-id', // ADLS Gen2 connection ID
  'mycontainer',        // Container name
  '/raw-data/analytics' // Path within container
);
```

Managing existing shortcuts

TypeScript

```
// List all shortcuts in a folder
const shortcuts = await shortcutClient.getAllShortcuts(workspaceId, itemId,
  '/Files');

// Get a specific shortcut
const shortcut = await shortcutClient.getShortcut(workspaceId, itemId,
  '/Files/MyShortcut');

// Delete a shortcut
await shortcutClient.deleteShortcut(workspaceId, itemId, '/Files/MyShortcut');

// Filter shortcuts by type
const oneLakeShortcuts = await shortcutClient.getOneLakeShortcuts(workspaceId,
  itemId, '/Files');
const s3Shortcuts = await shortcutClient.getS3Shortcuts(workspaceId, itemId,
  '/Files');
const adlsShortcuts = await shortcutClient.getAdlsGen2Shortcuts(workspaceId, itemId,
  '/Files');

// Search shortcuts by name pattern
const customerShortcuts = await shortcutClient.searchShortcutsByName(
  workspaceId, itemId, '/Files', 'customer'
);
```

Shortcut types and source configuration

Your workload can support different types of shortcuts depending on your use case. The `OneLakeShortcutClient` provides helper methods for common shortcut types:

OneLake shortcuts (Fabric items)

OneLake shortcuts allow your workload to access data stored in other Fabric items like Lakehouses, KQL Databases, etc.:

TypeScript

```
// Using the OneLakeShortcutClient helper method
const lakehouseShortcut = await shortcutClient.createOneLakeShortcut(
  workspaceId,
  itemId,
  "SharedLakehouse",    // Shortcut name
  "/Files",             // Target path in your item
  "source-workspace-id", // Source workspace ID
  "lakehouse-item-id",  // Source lakehouse ID
  "/Tables/myTable"     // Optional: specific path in source
);
```

Amazon S3 shortcuts

S3 shortcuts enable access to data stored in Amazon S3 buckets:

TypeScript

```
// Using the OneLakeShortcutClient helper method
const s3Shortcut = await shortcutClient.createS3Shortcut(
  workspaceId,
  itemId,
  "CustomerDataS3",     // Shortcut name
  "/Files",             // Target path in your item
  "s3-connection-id",   // S3 connection ID (pre-configured in Fabric)
  "my-bucket",          // S3 bucket name
  "/customer-folder"    // Path within the bucket
);
```

Working with shortcut data

Once a shortcut is created, your workload can work with the data using the `OneLakeStorageClient`. This client provides methods to interact with both regular OneLake content and shortcut data through a unified interface.

Setting up the OneLakeStorageClient

TypeScript

```
import { OneLakeStorageClient } from '../clients/OneLakeStorageClient';
import { WorkloadClientAPI } from "@ms-fabric/workload-client";

// Initialize the storage client
const storageClient = new OneLakeStorageClient(workloadClient);
```

Working with shortcut metadata

When working with shortcuts, it's important to understand that shortcut metadata and shortcut content are retrieved differently:

Getting shortcut information

To get information about shortcuts in a path, use `getPathMetadata` with `shortcutMetadata: true`:

TypeScript

```
// Get metadata for a path including shortcut information
const metadata = await storageClient.getPathMetadata(
  workspaceId,
  'itemId/Files', // Path to check for shortcuts
  false,          // recursive: false for current level only
  true           // shortcutMetadata: true to include shortcut info
);

// Filter for shortcuts
const shortcuts = metadata.paths.filter(path => path.isShortcut);

console.log('Found shortcuts:', shortcuts.map(s => ({
  name: s.name,
  isShortcut: s.isShortcut,
  lastModified: s.lastModified
})));
```

Accessing shortcut content

Important

When `shortcutMetadata: true`, you only get information about the shortcut itself, not the content inside the shortcut. To access the actual data within a shortcut, you need to make a separate call using the shortcut's path:

TypeScript

```

// First, get the shortcuts in the directory
const dirMetadata = await storageClient.getPathMetadata(
  workspaceId,
  'itemId/Files',
  false,
  true // Get shortcut metadata
);

// Find a specific shortcut
const myShortcut = dirMetadata.paths.find(path =>
  path.isShortcut && path.name === 'MyS3Shortcut'
);

if (myShortcut) {
  // Now get the content INSIDE the shortcut
  const shortcutContent = await storageClient.getPathMetadata(
    workspaceId,
    myShortcut.path, // Use the shortcut path
    true, // recursive: true to see all content
    false // shortcutMetadata: false to get actual
  content
  );

  console.log('Content inside shortcut:', shortcutContent.paths);
}

```

Reading and writing shortcut data

Once you have the shortcut content structure, you can read and write files just like regular OneLake files:

TypeScript

```

// Read a file from within a shortcut
const fileContent = await storageClient.readFileAsText(
  OneLakeStorageClient.getPath(workspaceId, itemId, 'Files/MyS3Shortcut/data.csv')
);

// Write a file to a shortcut (if the shortcut supports writes)
await storageClient.writeFileAsText(
  OneLakeStorageClient.getPath(workspaceId, itemId,
    'Files/MyS3Shortcut/output.txt'),
  'Processed data content'
);

```

Using the item wrapper for simplified access

For cleaner code, you can use the `OneLakeStorageClientItemWrapper`:

TypeScript

```
// Create an item wrapper for simplified access
const itemStorage = storageClient.createItemWrapper({
  workspaceId: workspaceId,
  id: itemId
});

// Get shortcuts in the Files directory
const filesMetadata = await itemStorage.getPathMetadata(
  'Files',
  false,
  true // Include shortcut metadata
);

// Access content within a shortcut
const shortcutContent = await itemStorage.getPathMetadata(
  'Files/MyShortcut',
  true, // recursive
  false // Get actual content, not shortcut metadata
);

// Read/write files with simpler paths
const fileContent = await itemStorage.readFileAsText('Files/MyShortcut/data.txt');
await itemStorage.writeFileAsText('Files/MyShortcut/processed.txt', 'Result data');
```

Complete example: Working with shortcut data

TypeScript

```
async function analyzeShortcutData(workspaceId: string, itemId: string) {
  const storageClient = new OneLakeStorageClient(workloadClient);

  try {
    // Step 1: Find all shortcuts in the Files directory
    const dirMetadata = await storageClient.getPathMetadata(
      workspaceId,
      `${itemId}/Files`,
      false,
      true // Get shortcut info
    );

    const shortcuts = dirMetadata.paths.filter(path => path.isShortcut);
    console.log(`Found ${shortcuts.length} shortcuts`);

    // Step 2: For each shortcut, analyze its content
    for (const shortcut of shortcuts) {
      console.log(`\nAnalyzing shortcut: ${shortcut.name}`);

      // Get the content inside this shortcut
      const shortcutContent = await storageClient.getPathMetadata(
        workspaceId,
```

```

    `${itemId}/Files/${shortcut.name}`,
    true, // recursive to see all files
    false // get actual content, not shortcut metadata
  );

  console.log(` - Contains ${shortcutContent.paths.length} items`);

  // List all files in the shortcut
  const files = shortcutContent.paths.filter(p => !p.isDirectory);
  for (const file of files) {
    console.log(` - File: ${file.name} (${file.contentLength} bytes)`);

    // Optionally read the file content
    if (file.name.endsWith('.txt') || file.name.endsWith('.csv')) {
      try {
        const content = await storageClient.readFileAsText(
          OneLakeStorageClient.getPath(workspaceId, itemId,
            `Files/${shortcut.name}/${file.name}`)
        );
        console.log(`    Preview: ${content.substring(0, 100)}...`);
      } catch (error) {
        console.log(`    Could not read file: ${error.message}`);
      }
    }
  }
} catch (error) {
  console.error('Error analyzing shortcut data:', error);
}
}

```

Security considerations

Shortcuts respect the security context of the user:

- For Fabric internal sources (like Lakehouses), the calling user's identity is used
- For external sources (like S3), the connection credentials specified during shortcut creation are used

Ensure your workload correctly handles authentication and provides appropriate user interface elements for securely entering connection details.

Related content

- [OneLake shortcuts overview](#)
- [OneLake shortcuts REST APIs](#)
- [Key concepts in the Extensibility Toolkit](#)
- [How to store data in an item](#)

Last updated on 12/15/2025

Integrate with OneLake catalog

Configure your Fabric items to appear in OneLake Catalog with proper categorization and DataHub support.

Configuration

Configure in your item's `Item.json`:

JSON

```
{
  "name": "MyDataItem",
  "displayName": "My Data Item",
  "oneLakeCatalogCategory": ["Data", "Process"],
  "supportedInDatahubL1": true
}
```

Categories

Each item can have up to 2 categories from:

- **Data**: Data storage and management
- **Insight**: Analytics and reporting
- **Process**: Data processing and transformation
- **Solution**: Complete applications
- **Configuration**: Setup and configuration
- **Other**: Items that don't fit other categories

JSON

```
{
  "oneLakeCatalogCategory": ["Data"]
}
```

JSON

```
{
  "oneLakeCatalogCategory": ["Process", "Insight"]
}
```

DataHub support

Enable DataHub L1 filtering:

JSON

```
{  
  "supportedInDatahubL1": true  
}
```

Important notes

- **No category specified:** Items won't appear in OneLake Catalog
- **Category limit:** Maximum 2 categories per item
- **DataHub integration:** Enable `supportedInDatahubL1` for data-related items

Next steps

- [Configure item jobs](#)

Last updated on 12/15/2025

Configure at a glance section for the workload hub

Configure the at-a-glance section in your workload's product details page using slide media content to showcase your workload's capabilities and features.

Overview

The at-a-glance section displays slide media content on your workload's product details page, allowing you to:

- **Showcase features:** Display images and videos highlighting key capabilities
- **Provide demos:** Include video demonstrations of your workload
- **Visual branding:** Enhance your workload's visual presentation
- **User engagement:** Help users understand your workload through rich media

Configuration in Product.json

Configure slide media in the `productDetail.slideMedia` array in your Product.json manifest:

JSON

```
{
  "productDetail": {
    "publisher": "Your Company",
    "slogan": "Your workload slogan",
    "description": "Brief description of your workload",
    "image": {
      "mediaType": 0,
      "source": "assets/images/main-image.png"
    },
    "slideMedia": [
      {
        "mediaType": 1,
        "source": "https://youtube.com/embed/UNgpB0Cvwa8?si=KwsR879MaVZd5CJi"
      },
      {
        "mediaType": 0,
        "source": "assets/images/feature-overview.png"
      },
      {
        "mediaType": 0,
        "source": "assets/images/dashboard-preview.png"
      }
    ]
  }
}
```

```
}  
}
```

Media types

Configure different media types in your slide media:

Images (mediaType: 0)

JSON

```
{  
  "mediaType": 0,  
  "source": "assets/images/your-image.png"  
}
```

- Use PNG, JPG, or SVG formats
- Store images in the `assets/images` folder
- Optimize for web display

Videos (mediaType: 1)

JSON

```
{  
  "mediaType": 1,  
  "source": "https://youtube.com/embed/VIDEO_ID"  
}
```

Supported video formats:

- **YouTube:** `https://youtube.com/embed/<id>` Or `https://www.youtube.com/embed/<id>`
- **Vimeo:** `https://player.vimeo.com/video/<number>` (without `www.`)

Configuration limits

- **Maximum slides:** 10 items in the `slideMedia` array
- **Recommended mix:** Combine images and videos for best user experience
- **Performance:** Optimize media files for web delivery

Best practices

Content strategy

- **Lead with video:** Place demo videos first to engage users immediately
- **Show key features:** Include screenshots of main functionality
- **Progressive disclosure:** Arrange content from overview to detailed features

Media optimization

- **Image quality:** Use high-resolution images that scale well
- **Video length:** Keep videos concise (2-3 minutes maximum)
- **Thumbnails:** Ensure video thumbnails are compelling

Example configuration

JSON

```
{
  "slideMedia": [
    {
      "mediaType": 1,
      "source": "https://youtube.com/embed/demo-overview"
    },
    {
      "mediaType": 0,
      "source": "assets/images/dashboard.png"
    },
    {
      "mediaType": 0,
      "source": "assets/images/analytics.png"
    },
    {
      "mediaType": 1,
      "source": "https://player.vimeo.com/video/tutorial"
    }
  ]
}
```

Next steps

- [Product Manifest](#)

How-To: Add a trial experience

Create trial experiences for your Fabric items using ribbon actions that open dialogs aligned with the pattern defined in [Fabric templates](#).

Overview

Trial experiences help users explore your workload's capabilities through guided dialogs that follow established Fabric patterns.

Item concept integration

Trial experiences are built around the [Item concept](#), where ribbon actions can trigger dialogs that guide users through your item's functionality.

Ribbon action implementation

Use RibbonAction to open dialogs aligned with patterns defined in [Fabric templates](#):

TypeScript

```
// TrialExperienceRibbon.tsx
import React from 'react';
import { Tooltip, ToolbarButton } from '@fluentui/react-components';
import { Play24Regular } from '@fluentui/react-icons';
import { createSaveAction, createSettingsAction } from '../controls/ItemEditor';

export const TrialExperienceRibbon = ({ onStartTrial }) => {
  const homeToolbarActions = [
    createSaveAction(),
    createSettingsAction(),
    {
      key: 'start-trial',
      element: (
        <Tooltip content="Start trial experience" relationship="label">
          <ToolbarButton
            icon={<Play24Regular />}
            onClick={onStartTrial}
            appearance="primary"
          >
            Try Now
          </ToolbarButton>
        </Tooltip>
      )
    }
  ];
};
```

```
return { homeToolbarActions };  
};
```

Dialog implementation

The ribbon action should open a dialog that follows the pattern defined in [Fabric templates](#):

TypeScript

```
export const handleStartTrial = () => {  
  // Open dialog aligned with Fabric templates pattern  
  openTrialDialog({  
    templatePattern: 'fabric-trial-experience',  
    alignment: 'https://aka.ms/fabrictemplates'  
  });  
};
```

Next steps

- [Item concept overview](#)
- [Fabric templates](#)

Last updated on 12/15/2025

How-To: Add jobs to be done to the item

Fabric workloads can define Item Task Flow Categories that integrate with the Fabric Task Flow Framework. This article explains how to configure these categories in your workload using the Product.json manifest.

Overview

Item Task Flow Categories enable:

- **Task integration:** Connect your items with Fabric's task flow framework
- **Workflow organization:** Categorize items by their primary purpose
- **User guidance:** Help users understand what your items can do
- **Framework integration:** Leverage Fabric's built-in task orchestration

Configuring Item Task Flow Categories in Product.json

Item Task Flow Categories are configured in the `itemJobTypes` array in your Product.json manifest file. Only **3 categories are currently supported**. If no categories are specified, items will appear in the "other" section:

JSON

```
{
  "name": "Product",
  "version": "1.100",
  "displayName": "My Custom Workload",

  "itemJobTypes": [
    "getData",
    "storeData"
  ],

  "compatibleItemTypes": ["Lakehouse"]
}
```

Available Item Task Flow Categories

Fabric currently supports **3 predefined categories** that you can include in your `itemJobTypes` array:

Data acquisition

- `getData` - Items that retrieve, import, or acquire data from various sources

Data storage

- `storeData` - Items that persist, save, or store data in various formats and locations

Custom operations

- `others` - Items that perform specialized operations not covered by the other categories

ⓘ Note

If you don't specify any `itemJobTypes` categories, your items will automatically appear in the "other" section of the Task Flow Framework.

Example configurations

Data acquisition workload

JSON

```
{
  "name": "DataAcquisitionWorkload",
  "itemJobTypes": [
    "getData"
  ]
}
```

Data storage workload

JSON

```
{
  "name": "DataStorageWorkload",
  "itemJobTypes": [
    "storeData"
  ]
}
```

Data pipeline workload

JSON

```
{
  "name": "DataPipelineWorkload",
  "itemJobTypes": [
    "getData",
    "storeData"
  ]
}
```

Custom workload

JSON

```
{
  "name": "CustomWorkload",
  "itemJobTypes": [
    "others"
  ]
}
```

Workload without categories

JSON

```
{
  "name": "GeneralWorkload",
  "itemJobTypes": []
}
```

⚠ Note

The above workload will appear in the "other" section since no categories are specified.

Category behavior

Task Flow Integration

When you include Item Task Flow Categories, Fabric automatically:

- Places your items in the appropriate sections of the Task Flow Framework

- Provides relevant task suggestions based on categories
- Enables workflow organization and discovery
- Connects with related Fabric services and features

Category limitations

Important constraints to consider:

- **Only 3 categories supported:** `getData`, `storeData`, and `others`
- **Default behavior:** Items without categories appear in the "other" section
- **Case sensitive:** Category names must match exactly as specified
- **No custom names:** Only the 3 predefined categories are supported

Best practices

Choosing categories

- **Be specific:** Select the category that best represents your item's primary purpose
- **Consider workflows:** Think about how your item fits into data acquisition or storage workflows
- **Use 'others' sparingly:** Only use `others` when your item doesn't clearly fit data acquisition or storage

Configuration guidelines

- **Match functionality:** Use `getData` for items that acquire/import data, `storeData` for items that persist data
- **Default to none:** If unsure, omit categories to let items appear in the "other" section
- **Test placement:** Verify your items appear in the expected Task Flow Framework sections

Category selection guide

1. `getData`: Choose if your item primarily acquires, imports, or retrieves data
2. `storeData`: Choose if your item primarily saves, persists, or stores data
3. `others`: Choose for specialized functionality not covered by data acquisition/storage
4. **No category:** Let items appear in "other" section for general-purpose functionality

Related configuration

Compatible item types

Categories work with specific Fabric item types:

JSON

```
{
  "itemJobTypes": ["getData", "storeData"],
  "compatibleItemTypes": ["Lakehouse", "Dataset", "Dataflow"]
}
```

Category combinations

Since only 3 categories are supported, your options are:

JSON

```
// Data acquisition only
"itemJobTypes": ["getData"]

// Data storage only
"itemJobTypes": ["storeData"]

// Custom operations
"itemJobTypes": ["others"]

// Combined data operations
"itemJobTypes": ["getData", "storeData"]

// No categories (appears in "other" section)
"itemJobTypes": []
```

Next steps

After configuring Item Task Flow Categories in Product.json:

- **Test integration** - Verify your item appears correctly in Task Flow Framework
- **Validate workflows** - Ensure categories enable appropriate workflow suggestions
- **User testing** - Confirm users can discover and use your item effectively
- **Monitor usage** - Track how categories affect item adoption and usage patterns

Related topics

- [Workload Manifest](#) - Complete manifest configuration
- [Product Manifest](#) - Product.json specification

- [Concept: Fabric Item Overview](#) - Understanding Fabric item structure

For more information about Fabric's Task Flow Framework, see the [Fabric Task Flow documentation](#).

Last updated on 12/15/2025

Define Jobs for Your Workload



Summarize this article for me

Jobs in Microsoft Fabric are long-running operations that execute background processing tasks, scheduled operations, and automated workflows. Fabric provides a [job scheduler](#) to manage job execution and a [monitoring hub](#) where users can view the status and history of their jobs.

What are Fabric Jobs?

Jobs are background processes that can be triggered on-demand or scheduled to run at specific intervals. Jobs play an important role when:

- **Items need to run code on a regular basis** - Scheduled data refresh, periodic maintenance, or automated report generation
- **The user is not in front of the UI** - Background processing, overnight operations, or long-running data transformations
- **Process data asynchronously** - Handle large data operations without blocking the user interface
- **Scale operations** - Leverage compute resources for intensive processing tasks

Jobs integrate with Fabric's unified [monitoring hub](#), allowing users to track job execution status, view history, and troubleshoot issues across all their Fabric items.

Defining Jobs in the Item Manifest

Custom items define their jobs in the item manifest using the `<JobScheduler>` configuration section. Jobs defined in the manifest appear in Fabric's [job scheduler](#) and [monitoring hub](#), but the job execution logic runs outside of Fabric.

Job Scheduler Structure

Here's an example of job configuration in the item manifest:

XML

```
<JobScheduler>
  <OnDemandJobDeduplicateOptions>PerItem</OnDemandJobDeduplicateOptions>
  <ScheduledJobDeduplicateOptions>PerItem</ScheduledJobDeduplicateOptions>
  <ItemJobTypes>
    <ItemJobType Name="{{WORKLOAD_NAME}}.HelloWorld.RunJob" />
  </ItemJobTypes>
</JobScheduler>
```

```
</ItemJobTypes>
</JobScheduler>
```

Deduplication Options

Control how concurrent job executions are handled:

- **None** - No deduplication; multiple instances of the same job can run simultaneously
- **PerItem** - Only one job of the same type can run per item at a time
- **PerUser** - Only one job of the same type can run per user and item combination

Defining Job Types

Each job type represents a specific operation your workload can perform:

XML

```
<ItemJobTypes>
  <ItemJobType Name="MyWorkload.DataProcessor.RefreshData" />
  <ItemJobType Name="MyWorkload.DataProcessor.GenerateReport" />
  <ItemJobType Name="MyWorkload.DataProcessor.CleanupTemp" />
</ItemJobTypes>
```

Job Naming Convention

Job names must follow the pattern: `{WorkloadName}.{ItemType}.{JobOperation}`

- **WorkloadName**: Your workload identifier (for example, "MyWorkload")
- **ItemType**: The item type that owns the job (for example, "DataProcessor")
- **JobOperation**: Descriptive name for the operation (for example, "RefreshData")

Implementing Job Execution

Jobs defined in custom items can't run directly within Fabric. Instead, you need to implement a remote endpoint that Fabric calls when a job is triggered. This gives you full control over where and how your job logic executes.

For detailed information about implementing and configuring remote endpoints, see [How-To: Enable remote endpoints for custom items](#).

Example Job Configuration

Here's an example of how a data processing workload might configure jobs:

XML

```
<JobScheduler>
  <OnDemandJobDeduplicateOptions>PerItem</OnDemandJobDeduplicateOptions>
  <ScheduledJobDeduplicateOptions>PerUser</ScheduledJobDeduplicateOptions>
  <ItemJobTypes>
    <!-- Scheduled data refresh job -->
    <ItemJobType Name="DataInsights.Pipeline.ScheduledRefresh" />

    <!-- On-demand data export job -->
    <ItemJobType Name="DataInsights.Pipeline.ExportToParquet" />

    <!-- Data transformation job -->
    <ItemJobType Name="DataInsights.Pipeline.TransformData" />

    <!-- Cleanup and maintenance job -->
    <ItemJobType Name="DataInsights.Pipeline.MaintenanceCleanup" />
  </ItemJobTypes>
</JobScheduler>
```

Job Implementation Patterns

Jobs are triggered by Fabric and execute at your remote endpoint. Consider these common patterns:

On-Demand Jobs

Triggered by user actions or API calls:

- Data exports and transformations
- Ad-hoc analysis operations
- User-initiated processing tasks

Scheduled Jobs

Run automatically based on time intervals:

- Regular data refresh operations
- Periodic maintenance tasks
- Automated report generation

Job Monitoring and Management

Jobs integrate with Fabric's [monitoring hub](#) infrastructure:

- **Execution tracking** - Monitor job status, duration, and resource usage in the monitoring hub
- **Error handling** - Implement retry policies and failure notifications in your remote endpoint
- **Performance metrics** - Track job performance through the monitoring hub
- **Audit logging** - Complete audit trail of job executions available in Fabric

Best Practices for Job Design

When designing jobs for your workload:

Job Granularity

- Keep individual jobs focused on specific tasks
- Break complex operations into smaller, manageable jobs
- Design for fault tolerance and restart capability

Resource Management

- Choose appropriate compute resources for your remote endpoint
- Consider memory and compute requirements for job execution
- Implement proper cleanup of temporary resources
- Use appropriate deduplication settings to prevent concurrent execution issues

Error Handling

- Implement robust error handling and logging in your remote endpoint
- Design for graceful degradation on failures
- Provide meaningful error messages that appear in the monitoring hub
- Implement retry logic for transient failures

Security

- Ensure proper authentication and authorization for your remote endpoint
- Secure communication between Fabric and your endpoint
- Follow least-privilege principles for job execution
- Validate all input data and parameters

Integration with OneLake

Jobs can leverage OneLake for data operations:

- Read input data from OneLake storage
- Write processed results back to OneLake
- Use shortcuts for external data access
- Maintain data lineage and governance

Integrate with the Monitoring Hub

When jobs are configured for your items, they automatically appear in Fabric's [monitoring hub](#). To provide the best user experience, you can integrate additional features that allow users to monitor, manage, and interact with jobs.

Enable Your Item in the Monitoring Hub Filter Pane

To make your item type available in the monitoring hub's filter pane, add the `supportedInMonitoringHub` property to your item's frontend manifest:

JSON

```
{
  "supportedInMonitoringHub": true
}
```

This allows users to filter jobs by your custom item type in the monitoring hub.

Integrate with Job Quick Actions

The monitoring hub provides quick action buttons for each job, allowing users to perform operations like cancel, retry, and view details directly from the monitoring hub interface.

Monitoring hub						
Monitoring hub is a station to view and track active activities across different products.						
Refresh						
Reset to default Item name: Copy of itemjkl X Sample Workload Artifact X IdanTest2 X						
Activity name	Status	Item type	Start time	Submitter	Location	
Copy of itemjkl	Failed	Sample Workload ...	12:56 PM, 10/10/23	Admin EM3	IdanTest2	
Copy of itemjkl	Succeeded	Sample Workload ...	12:55 PM, 10/10/23	Admin EM3	IdanTest2	

Configure which actions are available for your jobs by adding the `itemJobActionConfig` property to the item frontend manifest:

JSON

```
"itemJobActionConfig": {
  "registeredActions": {
    "detail": {
      "extensionName": "YourWorkload.ExtensionName",
      "action": "item.job.detail"
    },
    "cancel": {
      "extensionName": "YourWorkload.ExtensionName",
      "action": "item.job.cancel"
    },
    "retry": {
      "extensionName": "YourWorkload.ExtensionName",
      "action": "item.job.retry"
    }
  }
}
```

When a user selects an action button, Fabric calls the registered action in your extension. For example, when a user selects the Cancel icon, Fabric calls the `item.job.cancel` action, which your extension must implement.

Your action handler receives the job-related context and must respond with the operation result to notify the user.

Job Details Pane

When you register the detail action, Fabric displays job information in a side panel when users select the details button.

Details

×

^ Job Details

job type

Failing Job

job status

Failed

job start time UTC

2023-10-10T12:56:02.38

job end time UTC

2023-10-10T12:56:21.4266667

job instance id

b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2

^ Artifact Details

Artifact Type

Sample Workload Artifact

Artifact Name

Copy of itemjkl

Artifact Id

c2c2c2c2-dddd-eeee-ffff-a3a3a3a3a3

Workspace Name

IdanTest2

Workspace Id

d3d3d3d3-eeee-ffff-aaaa-b4b4b4b4b4b4

Artifact Editor

[Open](#) 

Your detail action must return data in a specific format for Fabric to display it correctly. Currently, Fabric supports key/value pairs as plain text or hyperlinks.

Example response format:

TypeScript

```
{
  details: [
```

```

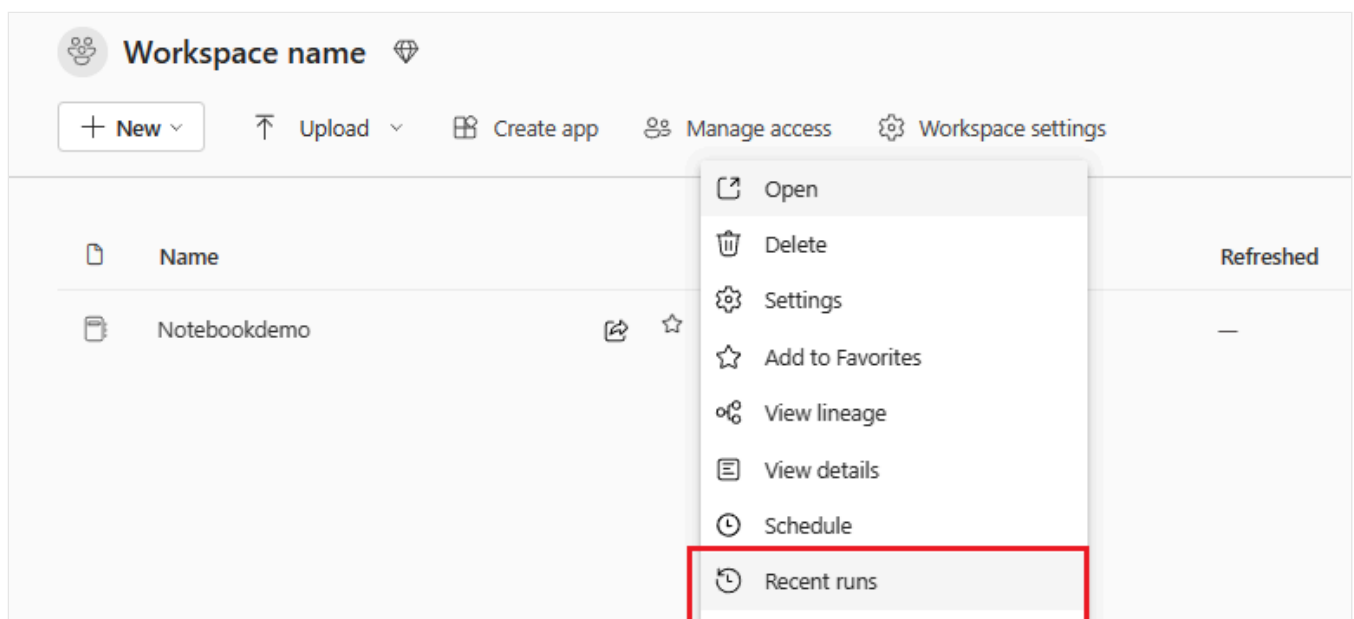
{ key: "Job ID", value: "12345" },
{ key: "Start Time", value: "2025-01-15 10:30:00" },
{ key: "Status", value: "Completed" },
{
  key: "Logs",
  value: "https://your-logs-url.com",
  isLink: true
}
]
}

```

Recent Runs

In addition to the monitoring hub, Fabric provides a Recent Runs view specifically for individual items. Users can access this view through:

- Context menu > **Recent runs**
- Using the `workloadClient.itemRecentRuns.open()` API



Enable Recent Runs

Step 1: Add recentruns context menu item

To show the **Recent runs** button in the item context menu, add a new entry to the `contextMenuItems` property in the item frontend manifest:

JSON

```

{
  "name": "recentruns"
}

```

Step 2: Add item recentRun settings

Add a new `recentRun` entry to the item settings property in the frontend manifest:

JSON

```
"recentRun": {  
  "useRecentRunsComponent": true  
}
```

This enables the shared Fabric Recent Runs UI component for your item.

Jobs Integration Example

For a complete example of job action implementation, see the [Fabric Workload Development Sample repository](#). Look for:

- **Backend implementation:** `JobsControllerImpl.cs` - Shows how to implement the job APIs
- **Frontend action handlers:** `index.worker.ts` - Search for actions beginning with `item.job` to see how to handle cancel, retry, and detail actions

Related Content

- [Authenticate Remote Endpoints](#)
- [Enable Remote Endpoints for Custom Items](#)
- [Job Scheduler API](#)
- [Monitoring Hub](#)
- [Store Data in OneLake](#)
- [Access Fabric APIs](#)
- [Manifest Overview](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/04/2026

How-To: Enable remote endpoints for custom items



Summarize this article for me

Custom items in Fabric can leverage remote endpoints to execute job logic and receive item lifecycle notifications. This article explains how to implement and configure remote service endpoints for your custom workload.

Why use remote endpoints?

If you want to integrate complex business logic or need to run jobs, you need a remote endpoint where Fabric can call you. Remote endpoints enable custom items to execute jobs and receive lifecycle notifications outside of Fabric. You need to implement a remote endpoint that Fabric calls when:

- **A job is triggered** - Execute background processing, scheduled operations, or on-demand tasks (see [Define Jobs for Your Workload](#))
- **Item lifecycle events occur** - Respond to item creation, updates, deletion, or other lifecycle events (see [Enable Item Lifecycle Notifications](#))

This architecture gives you full control over where and how your logic executes, allowing you to:

- Choose the appropriate compute platform for your workload requirements
- Scale resources independently of Fabric
- Implement custom authentication and security policies
- Integrate with external systems and services
- Use any programming language or runtime environment

Remote endpoint requirements

When a job is triggered (either on-demand or on a schedule), or you have onboarded to lifecycle notifications, Fabric calls your remote endpoint with the details. You're responsible for implementing the job execution logic at this endpoint.

The remote endpoint can be:

- **An Endpoint Resolution Service** - Dynamically resolve endpoint URLs at runtime for flexible routing (see [Use Endpoint Resolution Service](#))
- **An Azure Function** - Simple, serverless execution for lightweight jobs

- **An Azure Container Instance** - For jobs requiring specific runtime environments
- **A custom web service** - For complex job orchestration and processing
- **Any compute service** - As long as it can expose an HTTP endpoint and handle the job execution

Swagger specification

Your remote endpoint must implement the Fabric job execution contract defined in the Swagger specification (link TBD). This contract specifies:

- Required HTTP endpoints for job execution
- Request and response payload formats
- Authentication and authorization requirements
- Status reporting mechanisms

Local development support

For local development and testing, the Fabric Extensibility Toolkit provides a built-in stub implementation in the development server. This allows you to try the job execution experience on your local machine without deploying to Azure.

How it works

When you run the local development server (see [Get started with Fabric Extensibility Toolkit](#) and [Register Local Web Server](#)), all jobs defined in your item manifest are automatically redirected to your local machine.

When a job is triggered in dev mode:

1. The job request is intercepted by the local development server
2. Log statements appear in your local console showing the job details
3. A sample implementation demonstrates how to interact with OneLake
4. You receive a token that can be used to call any Fabric operation

This stub implementation provides:

- **Immediate feedback** - See log statements in your console as jobs are triggered
- **Sample code** - Learn how to structure your job execution logic
- **OneLake integration examples** - See how to access OneLake data during job execution
- **Token handling** - Understand how to obtain and use Fabric authentication tokens

Using the development stub

To use the local development server for job testing:

1. Start your local development server following the [quickstart guide](#)
2. Register your local web server using the [registration steps](#)
3. Define jobs in your item manifest as described in [Define Jobs for Your Workload](#)
4. Trigger a job from your item in Fabric
5. Observe the log output in your local console
6. Review the sample code and token handling in the development server logs

This local development experience allows you to build and test your job logic before deploying to production endpoints.

Configuration steps

To enable remote endpoint execution for your custom items:

1. Implement the remote endpoint

Create a service that handles job execution requests. Your implementation must:

- **Accept job trigger requests** - Receive and validate incoming job requests from Fabric
- **Execute job logic** - Perform the actual work defined by your job type
- **Report progress** - Send status updates back to Fabric during execution
- **Handle errors** - Implement proper error handling and reporting
- **Update job status** - Notify Fabric when the job completes or fails

2. Define the remote service endpoint

Configure the endpoint in your WorkloadManifest. Add the remote service configuration to specify where Fabric should send job execution requests.

XML

```
<!-- WorkloadManifest configuration example -->
<!-- Detailed configuration schema TBD -->
```

3. Handle job lifecycle

Your remote endpoint receives job trigger requests and is responsible for:

- **Executing the job logic** - Run the actual processing defined for each job type
- **Reporting progress to Fabric** - Update the job status in Fabric's monitoring hub

- Handling errors and retries - Implement retry logic for transient failures
- Updating job status in the monitoring hub - Ensure users can track job execution

Implementation example

Here's a conceptual example of a remote endpoint implementation using Azure Functions:

C#

```
[FunctionName("ExecuteJob")]
public async Task<IActionResult> Run(
    [HttpTrigger(AuthorizationLevel.Function, "post", Route = "jobs/execute")]
    HttpRequest req,
    ILogger log)
{
    // Parse the job request from Fabric
    var jobRequest = await ParseJobRequest(req);

    // Validate authentication and authorization
    if (!await ValidateRequest(jobRequest))
    {
        return new UnauthorizedResult();
    }

    try
    {
        // Report job started
        await ReportJobStatus(jobRequest.JobId, "Running");

        // Execute the job logic based on job type
        var result = await ExecuteJobLogic(jobRequest);

        // Report job completed successfully
        await ReportJobStatus(jobRequest.JobId, "Succeeded", result);

        return new OkObjectResult(result);
    }
    catch (Exception ex)
    {
        log.LogError(ex, $"Job {jobRequest.JobId} failed");

        // Report job failure
        await ReportJobStatus(jobRequest.JobId, "Failed", ex.Message);

        return new StatusCodeResult(500);
    }
}
```

Security considerations

When implementing remote endpoints for Fabric integration:

Authentication and authorization

- Use secure authentication mechanisms (for example, OAuth 2.0, managed identities)
- Validate that requests originate from Fabric
- Implement proper authorization checks for job execution

Communication security

- Use HTTPS for all communication
- Validate and sanitize all input data
- Implement rate limiting to prevent abuse

Data protection

- Handle sensitive data securely
- Follow data residency and compliance requirements
- Implement proper logging without exposing sensitive information

Accessing Fabric and Other Entra-Secured Services

When Fabric calls your remote endpoint, it provides authentication tokens that you can use to access Fabric APIs and other Microsoft Entra-secured services. This enables your workload to call back to Fabric or access resources like OneLake on behalf of the user.

Authentication Token Format

Fabric uses the SubjectAndAppToken format when calling your remote endpoint:

```
SubjectAndAppToken1.0 subjectToken="<user-token>", appToken="<app-token>"
```

This dual-token format provides:

- **subjectToken** - A user-delegated token representing the user on whose behalf the operation is being performed
- **appToken** - An app-only token from Fabric, proving the request originated from Fabric

Token Exchange for Resource Access

To access Fabric APIs or other resources like OneLake, you need to exchange the user's token for resource-specific tokens using the OAuth 2.0 On-Behalf-Of (OBO) flow.

Accessing OneLake

JavaScript

```
// Exchange user token for OneLake access token
const oneLakeToken = await getOneLakeToken(
  authContext.subjectToken,
  authContext.tenantId
);

// Use the token to access OneLake data
const response = await axios.get(
  `https://onelake.dfs.fabric.microsoft.com/workspaces/${workspaceId}/items/${itemId}/files/data.parquet`,
  {
    headers: {
      'Authorization': `Bearer ${oneLakeToken}`
    }
  }
);
```

Accessing Fabric APIs

JavaScript

```
// Build composite token for Fabric API calls
const fabricAuthHeader = await buildCompositeToken(authContext);

// Call Fabric workload control API
const itemDetails = await axios.get(
  `https://api.fabric.microsoft.com/v1/workspaces/${workspaceId}/items/${itemId}`,
  {
    headers: {
      'Authorization': fabricAuthHeader
    }
  }
);
```

Accessing Other Entra-Secured Services

You can exchange the user token for any Entra-secured service:

JavaScript

```
// Exchange for Azure Resource Manager token
const armToken = await getTokenForScope(
  authContext.subjectToken,
  authContext.tenantId,
  'https://management.azure.com/.default'
);

// Exchange for Microsoft Graph token
const graphToken = await getTokenForScope(
  authContext.subjectToken,
  authContext.tenantId,
  'https://graph.microsoft.com/.default'
);
```

Token Validation and Security

Always validate incoming tokens from Fabric:

1. **Parse the SubjectAndAppToken format** - Extract both tokens from the Authorization header
2. **Validate token signatures** - Verify tokens against Azure AD signing keys
3. **Check token claims** - Ensure issuer, audience, and tenant match expected values
4. **Verify app token is from Fabric** - Confirm the appToken originates from the Fabric service

For complete authentication implementation details, including token validation, parsing, and exchange service examples, see [Authenticate Remote Endpoints](#).

Best practices

Reliability

- Implement retry logic for transient failures
- Use idempotent operations where possible
- Design for graceful handling of endpoint unavailability

Performance

- Choose appropriate compute resources for your remote endpoint
- Consider memory and compute requirements for job execution
- Implement proper resource cleanup after job completion

Monitoring

- Log all job execution requests and responses
- Track performance metrics (execution time, resource usage)
- Set up alerts for failures and anomalies
- Integrate with Azure Monitor or other monitoring solutions

Error handling

- Provide meaningful error messages that appear in the monitoring hub
- Differentiate between transient and permanent failures
- Implement proper timeout handling

Integration with Fabric monitoring

Your remote endpoint should report job status updates to Fabric so users can track execution in the [monitoring hub](#). Status updates should include:

- Job start time
- Progress updates (if applicable)
- Completion status (succeeded, failed, cancelled)
- Error messages and diagnostic information
- Execution duration and resource usage

Related content

- [Authenticate Remote Endpoints](#)
- [Use Endpoint Resolution Service](#)
- [Define Jobs for Your Workload](#)
- [Enable Item Lifecycle Notifications](#)
- [Get started with Fabric Extensibility Toolkit](#)
- [Register Local Web Server](#)
- [Job Scheduler API](#)
- [Monitoring Hub](#)
- [Manifest Overview](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

How-To: Use Endpoint Resolution Service



Summarize this article for me

The Endpoint Resolution Service provides dynamic endpoint routing for your custom workload, allowing you to configure where Fabric sends job execution requests and lifecycle notifications without hardcoding endpoint URLs in your manifest.

What is Endpoint Resolution Service?

The Endpoint Resolution Service is a flexible mechanism that resolves the actual endpoint URL at runtime based on your workload configuration. Instead of specifying a fixed endpoint URL in your WorkloadManifest, you can use the Endpoint Resolution Service to:

- Route requests to different endpoints based on environment (Dev, Test, Prod)
- Update endpoint URLs without redeploying your workload manifest
- Implement blue-green deployments or A/B testing
- Manage multiple backend services for different regions or tenants

When to use Endpoint Resolution Service

Use the Endpoint Resolution Service when you need:

- **Environment-specific routing** - Different endpoints for development, testing, and production
- **Dynamic endpoint updates** - Change backend services without updating the manifest
- **Multi-region deployments** - Route requests to region-specific endpoints
- **Flexible backend architecture** - Switch between different backend implementations

How it works

When Fabric needs to call your remote endpoint for job execution or lifecycle notifications:

1. Fabric queries the Endpoint Resolution Service with your workload identifier
2. The service returns the appropriate endpoint URL based on your configuration
3. Fabric sends the request to the resolved endpoint
4. Your backend service processes the request and responds

This indirection layer allows you to change your backend infrastructure without modifying the workload manifest.

Configuration

Define the Endpoint Resolution Service in WorkloadManifest

Configure the service endpoint with the Endpoint Resolution Service flag in your WorkloadManifest:

XML

```
<ServiceEndpoint>
  <Name>Backend</Name>
  <Url>https://your-resolution-service.azurewebsites.net</Url>
  <IsEndpointResolutionService>true</IsEndpointResolutionService>
</ServiceEndpoint>
```

When `<IsEndpointResolutionService>` is set to `true`, Fabric treats the URL as an endpoint resolution service rather than a direct backend endpoint. Fabric will call this service to resolve the actual endpoint URL for each request.

For comparison, a direct endpoint configuration would look like:

XML

```
<ServiceEndpoint>
  <Name>Backend</Name>
  <Url>https://your-backend.azurewebsites.net/api</Url>
  <IsEndpointResolutionService>false</IsEndpointResolutionService>
</ServiceEndpoint>
```

Implement the resolution service

Your Endpoint Resolution Service must implement the required contract to respond to resolution requests from Fabric. The service should:

1. **Authenticate requests** - Verify that requests come from Fabric
2. **Resolve endpoint URLs** - Return the appropriate backend endpoint URL
3. **Handle errors** - Provide fallback behavior for resolution failures
4. **Log resolutions** - Track which endpoints are being used

Example resolution service implementation:

JavaScript

```

const express = require('express');
const { authenticateControlPlaneCall } = require('./authentication');

const router = express.Router();

/**
 * POST /resolve-endpoint
 * Resolve endpoint based on context
 * Uses relaxed authentication: app token required, but subject token and tenant ID
header optional
 */
router.post('/resolve-endpoint', async (req, res) => {
  // Apply relaxed authentication (subject token not required for resolution)
  const authResult = await authenticateControlPlaneCall(req, res, {
    requireSubjectToken: false,
    requireTenantIdHeader: false
  });
  if (!authResult) return; // Auth failed, response already sent

  // Parse resolution request from Fabric
  const { workloadId, environment, requestType } = req.body;

  // Determine the appropriate endpoint based on configuration
  const endpoint = await resolveEndpoint(workloadId, environment, requestType);

  console.log(`Resolved endpoint: ${endpoint}`);

  const response = {
    url: endpoint,
    ttlInMinutes: 60 // Cache for 60 minutes
  };

  res.status(200).json(response);
});

async function resolveEndpoint(workloadId, environment, requestType) {
  // Implement your resolution logic here
  // This could query a database, configuration store, etc.

  if (environment === 'Production') {
    return 'https://prod-backend.azurewebsites.net/api';
  } else if (environment === 'Test') {
    return 'https://test-backend.azurewebsites.net/api';
  } else {
    return 'https://dev-backend.azurewebsites.net/api';
  }
}

module.exports = router;

```

Resolution caching

Fabric caches resolved endpoints to reduce the load on your resolution service. You can control the cache duration by returning a `tTlInMinutes` value in your resolution response:

JavaScript

```
const response = {  
  url: 'https://prod-backend.azurewebsites.net/api',  
  tTlInMinutes: 60 // Cache for 60 minutes  
};
```

Recommended TTL values:

- **Short TTL (5-15 minutes)** - Use for frequently changing endpoints or active deployments
- **Medium TTL (30-60 minutes)** - Use for stable environments with occasional updates
- **Long TTL (120+ minutes)** - Use for production environments that rarely change
- **No TTL or 0** - Resolve on every request (not recommended for production)

Best practices

Reliability

- Implement redundancy for your resolution service
- Provide default/fallback endpoints for resolution failures
- Monitor resolution service health and performance

Security

- Authenticate all resolution requests from Fabric
- Use HTTPS for all communication
- Validate and sanitize resolution inputs
- Don't expose sensitive configuration in resolution responses

Performance

- Use appropriate cache TTLs to balance freshness and performance
- Implement efficient resolution logic (avoid slow database queries)
- Consider geographic distribution of your resolution service

Monitoring

- Log all resolution requests and responses

- Track resolution failures and fallback usage
- Monitor cache hit rates and TTL effectiveness
- Set up alerts for resolution service failures

Example scenarios

Blue-green deployment

Use the Endpoint Resolution Service to switch between blue and green deployments:

JavaScript

```
async function resolveEndpoint(workloadId) {
  const activeDeployment = await getActiveDeployment(workloadId);

  if (activeDeployment === 'Blue') {
    return 'https://blue-backend.azurewebsites.net/api';
  } else {
    return 'https://green-backend.azurewebsites.net/api';
  }
}
```

Region-specific routing

Route requests to region-specific endpoints:

JavaScript

```
async function resolveEndpoint(workloadId, region) {
  switch (region) {
    case 'WestUS':
      return 'https://westus-backend.azurewebsites.net/api';
    case 'EastUS':
      return 'https://eastus-backend.azurewebsites.net/api';
    case 'WestEurope':
      return 'https://westeu-backend.azurewebsites.net/api';
    default:
      return 'https://global-backend.azurewebsites.net/api';
  }
}
```

Environment-based routing

Route based on environment:

```
async function resolveEndpoint(environment) {  
  if (environment === 'Production') {  
    return 'https://prod-backend.azurewebsites.net/api';  
  } else if (environment === 'Staging') {  
    return 'https://staging-backend.azurewebsites.net/api';  
  } else if (environment === 'Test') {  
    return 'https://test-backend.azurewebsites.net/api';  
  } else {  
    return 'https://dev-backend.azurewebsites.net/api';  
  }  
}
```

Troubleshooting

Resolution failures

If endpoint resolution fails:

1. Check resolution service logs for errors
2. Verify authentication configuration
3. Test resolution service endpoint directly
4. Check network connectivity from Fabric to your service
5. Verify SSL/TLS certificate validity

Incorrect endpoint routing

If requests go to the wrong endpoint:

1. Check resolution logic and configuration
2. Verify cache TTL settings
3. Check for stale cached entries
4. Review resolution service logs for the specific request

Related content

- [Enable Remote Endpoints for Custom Items](#)
- [Define Jobs for Your Workload](#)
- [Enable Item Lifecycle Notifications](#)
- [Manifest Overview](#)

Last updated on 03/04/2026

Authenticate Remote Endpoints



Summarize this article for me

Remote endpoints in Microsoft Fabric workloads use a specialized authentication mechanism that provides both user context and application identity. This article explains how to implement authentication for remote endpoints that handle jobs, lifecycle notifications, and other backend operations.

Authentication Flows Overview

Fabric workloads involve three primary authentication flows:

1. **Fabric to Workload** - Fabric calls your remote endpoint with SubjectAndAppToken format
2. **Workload to Fabric** - Your workload calls Fabric APIs with SubjectAndAppToken or Bearer tokens
3. **Workload Frontend to Backend** - Your frontend calls your backend with Bearer tokens

This article focuses on authenticating requests from Fabric to your workload and making authenticated calls back to Fabric.

SubjectAndAppToken Format

When Fabric calls your remote endpoint (for jobs, lifecycle notifications, or other operations), it uses a dual-token authentication format called SubjectAndAppToken.

Authorization Header Structure

```
SubjectAndAppToken1.0 subjectToken="<user-delegated-token>", appToken="<app-only-token>"
```

This format includes two distinct tokens:

- **subjectToken** - A delegated token representing the user on whose behalf the operation is being performed
- **appToken** - An app-only token from the Fabric application, proving the request originated from Fabric

 **Important**

The `subjectToken` is not always present. It may be missing in scenarios such as:

- **Service principal operations** - When a service principal interacts with Fabric public APIs without user context
- **System operations** - Operations like item deletion where no user is directly involved
- **Automated workflows** - CI/CD pipelines or scheduled operations that run without user interaction

Your remote endpoint must be designed to handle both cases: requests with user context (`subjectToken` present) and requests without user context (`subjectToken` absent).

Why Dual Tokens?

The dual-token approach provides three key benefits:

1. **Validation** - Verify that the request originated from Fabric by validating the `appToken`
2. **User Context** - The `subjectToken` provides user context for the action being performed
3. **Inter-Service Communication** - Use the `subjectToken` to acquire On-Behalf-Of (OBO) tokens for calling other services with user context

Parsing SubjectAndAppToken

Your remote endpoint must parse the Authorization header to extract both tokens.

JavaScript Example

JavaScript

```
/**
 * Parse SubjectAndAppToken from Authorization header
 * @param {string} authHeader - Authorization header value
 * @returns {object|null} Parsed tokens or null if invalid format
 */
function parseSubjectAndAppToken(authHeader) {
  if (!authHeader || !authHeader.startsWith('SubjectAndAppToken1.0 ')) {
    return null;
  }

  const tokenPart = authHeader.substring('SubjectAndAppToken1.0 '.length);
  const tokens = {};

  const parts = tokenPart.split(',');

  for (const part of parts) {
```



```

const [key, value] = part.split('=');
if (key && value) {
  // Remove surrounding quotes from the token value
  const cleanValue = value.trim().replace(/^"(.*)"$/, '$1');
  tokens[key.trim()] = cleanValue;
}
}

return {
  subjectToken: tokens.subjectToken || null,
  appToken: tokens.appToken || null
};
}

```

Usage Example

JavaScript

```

const authHeader = req.headers['authorization'];
const tokens = parseSubjectAndAppToken(authHeader);

if (!tokens || !tokens.appToken) {
  return res.status(401).json({ error: 'Invalid authorization header' });
}

// Now you have access to:
// - tokens.subjectToken (user-delegated token, may be null)
// - tokens.appToken (Fabric app token, always present)

// Check if user context is available
if (tokens.subjectToken) {
  console.log('Request has user context');
} else {
  console.log('Request is app-only (no user context)');
}

```

Token Validation

Both tokens must be validated to ensure authenticity and proper claims.

App Token Validation

The `appToken` is an app-only token from Fabric. Validate:

- **Token signature** - Verify against Azure AD signing keys
- **Token lifetime** - Check `nbf` (not before) and `exp` (expiration) claims
- **Audience** - Must match your workload's app registration

- **Issuer** - Must be from Azure AD with correct tenant
- **idtyp claim** - Must be "app" for app-only tokens
- **No scp claim** - App-only tokens don't have scope claims
- **Tenant ID** - Must match your publisher tenant ID

Sample appToken claims:

JSON

```
{
  "aud": "api://localdevinstance/aaaabbbb-0000-cccc-1111-dddd2222eeee/Fabric.WorkloadSample/123",
  "iss": "https://sts.windows.net/12345678-77f3-4fcc-bdaa-487b920cb7ee/",
  "iat": 1700047232,
  "nbf": 1700047232,
  "exp": 1700133932,
  "appid": "11112222-bbbb-3333-cccc-4444dddd5555",
  "appidacr": "2",
  "idtyp": "app",
  "oid": "aaaaaaaa-0000-1111-2222-bbbbbbbbbbbb",
  "tid": "bbbbsccc-1111-dddd-2222-eeee3333ffff",
  "ver": "1.0"
}
```

Subject Token Validation

The `subjectToken` is a user-delegated token. Validate:

- **Token signature** - Verify against Azure AD signing keys
- **Token lifetime** - Check `nbf` and `exp` claims
- **Audience** - Must match your workload's app registration
- **Issuer** - Must be from Azure AD with correct tenant
- **scp claim** - Must include "FabricWorkloadControl" scope
- **No idtyp claim** - Delegated tokens don't have this claim
- **appid must match** - Should be the same as in appToken

Sample subjectToken claims:

JSON

```
{
  "aud": "api://localdevinstance/aaaabbbb-0000-cccc-1111-dddd2222eeee/Fabric.WorkloadSample/123",
  "iss": "https://sts.windows.net/12345678-77f3-4fcc-bdaa-487b920cb7ee/",
  "iat": 1700050446,
  "nbf": 1700050446,
  "exp": 1700054558,
}
```

```
"appid": "11112222-bbbb-3333-cccc-4444dddd5555",
"scp": "FabricWorkloadControl",
"name": "john doe",
"oid": "bbbbbbbbb-1111-2222-3333-cccccccccccc",
"upn": "user1@contoso.com",
"tid": "bbbbsccc-1111-dddd-2222-eeee3333ffff",
"ver": "1.0"
}
```

JavaScript Token Validation Example

JavaScript

```
const jwt = require('jsonwebtoken');
const jwksClient = require('jwks-rsa');

/**
 * Validate Azure AD token
 * @param {string} token - JWT token to validate
 * @param {object} options - Validation options
 * @returns {Promise<object>} Validated token claims
 */
async function validateAadToken(token, options = {}) {
  const { isAppOnly = false, tenantId = null, audience = null } = options;

  // Decode token to get header and tenant
  const decoded = jwt.decode(token, { complete: true });
  if (!decoded) {
    throw new Error('Invalid token format');
  }

  const { header, payload } = decoded;
  const tokenTenantId = payload.tid;

  // Get signing key from Azure AD
  const client = jwksClient({
    jwksUri:
      `https://login.microsoftonline.com/${tokenTenantId}/discovery/v2.0/keys`,
    cache: true,
    cacheMaxAge: 86400000 // 24 hours
  });

  const signingKey = await new Promise((resolve, reject) => {
    client.getSigningKey(header.kid, (err, key) => {
      if (err) reject(err);
      else resolve(key.getPublicKey());
    });
  });

  // Verify token signature and claims
  const verifyOptions = {
    algorithms: ['RS256'],
  }
```

```

    audience: audience,
    clockTolerance: 60 // 1 minute clock skew
  };

  const verified = jwt.verify(token, signingKey, verifyOptions);

  // Validate app-only vs delegated token
  if (isAppOnly) {
    if (verified.idtyp !== 'app') {
      throw new Error('Expected app-only token');
    }
    if (verified.scp) {
      throw new Error('App-only token should not have scp claim');
    }
  } else {
    if (verified.idtyp) {
      throw new Error('Expected delegated token');
    }
    if (!verified.scp || !verified.scp.includes('FabricWorkloadControl')) {
      throw new Error('Missing required FabricWorkloadControl scope');
    }
  }

  // Validate tenant if required
  if (tenantId && verified.tid !== tenantId) {
    throw new Error(`Token tenant mismatch: expected ${tenantId}, got ${verified.tid}`);
  }

  return verified;
}

```

Authentication Middleware

Implement middleware to authenticate incoming requests from Fabric.

Complete Authentication Flow

JavaScript

```

/**
 * Authentication middleware that validates control plane calls from Fabric
 * Handles both user-delegated (with subjectToken) and app-only (without
subjectToken) scenarios
 * @param {object} req - Express request object
 * @param {object} res - Express response object
 * @param {object} options - Authentication options
 * @param {boolean} options.requireSubjectToken - Whether subject token is required
(default: false)
 * @returns {Promise<boolean>} True if authenticated successfully

```

```

*/
async function authenticateControlPlaneCall(req, res, options = {}) {
  const { requireSubjectToken = false } = options;
  try {
    // Extract and parse authorization header
    const authHeader = req.headers['authorization'];
    if (!authHeader) {
      return res.status(401).json({ error: 'Missing Authorization header' });
    }

    const tokens = parseSubjectAndAppToken(authHeader);
    if (!tokens || !tokens.appToken) {
      return res.status(401).json({ error: 'Invalid Authorization header format'
    });
  }

  // Get tenant ID from header
  const tenantId = req.headers['ms-client-tenant-id'];
  if (!tenantId) {
    return res.status(400).json({ error: 'Missing ms-client-tenant-id header' });
  }

  // Get configuration
  const publisherTenantId = process.env.TENANT_ID;
  const audience = process.env.BACKEND_AUDIENCE;
  const fabricBackendAppId = '00000009-0000-0000-c000-000000000000';

  // Validate app token (from Fabric)
  const appTokenClaims = await validateAadToken(tokens.appToken, {
    isAppOnly: true,
    audience: audience
  });

  // Verify app token is from Fabric
  const appTokenAppId = appTokenClaims.appid || appTokenClaims.azp;
  if (appTokenAppId !== fabricBackendAppId) {
    return res.status(401).json({ error: 'App token not from Fabric' });
  }

  // Verify app token is in publisher's tenant
  if (publisherTenantId && appTokenClaims.tid !== publisherTenantId) {
    return res.status(401).json({ error: 'App token tenant mismatch' });
  }

  // Validate subject token if present
  let subjectTokenClaims = null;
  if (tokens.subjectToken) {
    subjectTokenClaims = await validateAadToken(tokens.subjectToken, {
      isAppOnly: false,
      audience: audience,
      tenantId: tenantId
    });
  }

  // Verify subject token has same appid as app token
  const subjectAppId = subjectTokenClaims.appid || subjectTokenClaims.azp;

```

```

    if (subjectAppId !== appTokenAppId) {
      return res.status(401).json({ error: 'Token appid mismatch' });
    }
  } else if (requireSubjectToken) {
    // If subject token is required but missing, reject the request
    return res.status(401).json({
      error: 'Subject token required for this operation'
    });
  }

  // Store authentication context in request
  req.authContext = {
    subjectToken: tokens.subjectToken,
    appToken: tokens.appToken,
    tenantId: tenantId,
    hasSubjectContext: !!tokens.subjectToken,
    appTokenClaims: appTokenClaims,
    subjectTokenClaims: subjectTokenClaims,
    userId: subjectTokenClaims?.oid || subjectTokenClaims?.sub,
    userName: subjectTokenClaims?.name || subjectTokenClaims?.upn
  };

  return true; // Authentication successful
} catch (error) {
  console.error('Authentication failed:', error.message);
  res.status(401).json({ error: 'Authentication failed' });
  return false;
}
}

```

Using the Middleware

JavaScript

```

app.post('/api/jobs/execute', async (req, res) => {
  // Authenticate the request (subjectToken is optional)
  const authenticated = await authenticateControlPlaneCall(req, res);
  if (!authenticated) {
    return; // Response already sent by middleware
  }

  // Access authentication context
  const { hasSubjectContext, userId, userName, tenantId, subjectToken } =
    req.authContext;

  if (hasSubjectContext) {
    console.log(`Executing job for user: ${userName} (${userId})`);
  } else {
    console.log('Executing job in app-only context (no user)');
  }
}

```

```

    // Execute job logic...
  });

  // Example: Require user context for specific operations
  app.post('/api/lifecycle/create', async (req, res) => {
    // For create operations, we might require user context
    const authenticated = await authenticateControlPlaneCall(req, res, {
      requireSubjectToken: true
    });
    if (!authenticated) {
      return; // Response already sent by middleware
    }

    // User context is guaranteed to be present here
    const { userId, userName } = req.authContext;
    console.log(`Creating item for user: ${userName}`);

    // Handle creation...
  });

  // Example: Allow app-only context for delete operations
  app.post('/api/lifecycle/delete', async (req, res) => {
    // Delete operations might not have user context
    const authenticated = await authenticateControlPlaneCall(req, res, {
      requireSubjectToken: false // Default, but shown for clarity
    });
    if (!authenticated) {
      return;
    }

    const { hasSubjectContext, userName } = req.authContext;
    if (hasSubjectContext) {
      console.log(`Deleting item for user: ${userName}`);
    } else {
      console.log('Deleting item (system operation)');
    }

    // Handle deletion...
  });

```

Token Exchange - On-Behalf-Of (OBO) Flow

To call Fabric APIs or access OneLake from your remote endpoint, you need to exchange the user's subjectToken for resource-specific tokens using the OAuth 2.0 On-Behalf-Of flow.

Token Exchange Service

JavaScript

```

const https = require('https');
const { URL } = require('url');

const AAD_LOGIN_URL = 'https://login.microsoftonline.com';
const ONELAKE_SCOPE = 'https://storage.azure.com/.default';
const FABRIC_SCOPE = 'https://analysis.windows.net/powerbi/api/.default';

/**
 * Get token for any scope using OBO flow
 * @param {string} userToken - User's access token (subjectToken)
 * @param {string} tenantId - User's tenant ID
 * @param {string} scope - Target resource scope
 * @returns {Promise<string>} Access token for the requested scope
 * @throws {Error} Throws consent-related errors (AADSTS65001, AADSTS65005) that
should be propagated to UX
 */
async function getTokenForScope(userToken, tenantId, scope) {
    const clientId = process.env.BACKEND_APPID;
    const clientSecret = process.env.BACKEND_CLIENT_SECRET;

    if (!clientId || !clientSecret) {
        throw new Error('BACKEND_APPID and BACKEND_CLIENT_SECRET required');
    }

    const tokenEndpoint = `${AAD_LOGIN_URL}/${tenantId}/oauth2/v2.0/token`;
    const requestBody = new URLSearchParams({
        grant_type: 'urn:ietf:params:oauth:grant-type:jwt-bearer',
        client_id: clientId,
        client_secret: clientSecret,
        assertion: userToken,
        scope: scope,
        requested_token_use: 'on_behalf_of'
    }).toString();

    const response = await makeTokenRequest(tokenEndpoint, requestBody);

    if (!response.access_token) {
        throw new Error('Token exchange failed: missing access_token');
    }

    return response.access_token;
}

/**
 * Get OneLake access token
 * @param {string} userToken - User's access token from authContext
 * @param {string} tenantId - User's tenant ID
 * @returns {Promise<string>} OneLake access token
 */
async function getOneLakeToken(userToken, tenantId) {
    return getTokenForScope(userToken, tenantId, ONELAKE_SCOPE);
}

/**

```



```

* Get Fabric OBO token for calling Fabric APIs
* @param {string} userToken - User's access token
* @param {string} tenantId - User's tenant ID
* @returns {Promise<string>} Fabric OBO token
*/
async function getFabricOboToken(userToken, tenantId) {
  return getTokenForScope(userToken, tenantId, FABRIC_SCOPE);
}

/**
 * Make HTTPS request to token endpoint
 */
function makeTokenRequest(url, body) {
  return new Promise((resolve, reject) => {
    const parsedUrl = new URL(url);
    const options = {
      hostname: parsedUrl.hostname,
      port: 443,
      path: parsedUrl.pathname,
      method: 'POST',
      headers: {
        'Content-Type': 'application/x-www-form-urlencoded',
        'Content-Length': Buffer.byteLength(body)
      }
    };

    const req = https.request(options, (res) => {
      let responseBody = '';
      res.on('data', (chunk) => { responseBody += chunk; });
      res.on('end', () => {
        const parsed = JSON.parse(responseBody);
        if (res.statusCode >= 200 && res.statusCode < 300) {
          resolve(parsed);
        } else {
          const error = parsed.error_description || parsed.error || `HTTP
${res.statusCode}`;
          reject(new Error(error));
        }
      });
    });

    req.on('error', reject);
    req.write(body);
    req.end();
  });
}

```

Using Token Exchange

```
// In your job or lifecycle notification handler
async function handleJobExecution(req, res) {
  const authenticated = await authenticateControlPlaneCall(req, res);
  if (!authenticated) return;

  const { hasSubjectContext, subjectToken, tenantId } = req.authContext;

  try {
    // Only exchange token if user context is available
    if (hasSubjectContext) {
      // Get OneLake token to access item data on behalf of the user
      const oneLakeToken = await getOneLakeToken(subjectToken, tenantId);

      // Use OneLake token to read/write data
      const data = await readFromOneLake(oneLakeToken, workspaceId, itemId);

      // Process data...
    } else {
      // Handle app-only scenario
      // You may need to use different authentication or skip user-specific
      operations
      console.log('Processing job without user context');
      // Use workspace or item-level access instead
    }

    res.status(200).json({ status: 'completed' });
  } catch (error) {
    console.error('Job execution failed:', error);
    res.status(500).json({ error: 'Job execution failed' });
  }
}
```

Calling Fabric APIs

To call Fabric workload control APIs, you need to construct a SubjectAndAppToken with both OBO and S2S tokens.

Service-to-Service (S2S) Token

In addition to the OBO token, you need an app-only S2S token for the appToken part.

JavaScript

```
/**
 * Get S2S (Service-to-Service) token using client credentials flow
 * @param {string} tenantId - Publisher's tenant ID
 * @param {string} scope - Target resource scope
 * @returns {Promise<string>} S2S access token
 */
```

```

async function getS2STokenForScope(tenantId, scope) {
  const clientId = process.env.BACKEND_APPID;
  const clientSecret = process.env.BACKEND_CLIENT_SECRET;

  if (!clientId || !clientSecret) {
    throw new Error('BACKEND_APPID and BACKEND_CLIENT_SECRET required');
  }

  const tokenEndpoint = `${AAD_LOGIN_URL}/${tenantId}/oauth2/v2.0/token`;
  const requestBody = new URLSearchParams({
    grant_type: 'client_credentials',
    client_id: clientId,
    client_secret: clientSecret,
    scope: scope
  }).toString();

  const response = await makeTokenRequest(tokenEndpoint, requestBody);

  if (!response.access_token) {
    throw new Error('S2S token acquisition failed');
  }

  return response.access_token;
}

/**
 * Get Fabric S2S token
 * @param {string} publisherTenantId - Publisher's tenant ID
 * @returns {Promise<string>} Fabric S2S access token
 */
async function getFabricS2SToken(publisherTenantId) {
  return getS2STokenForScope(publisherTenantId, FABRIC_SCOPE);
}

```

Building Composite Token for Fabric APIs

JavaScript

```

/**
 * Build composite token for Fabric API calls
 * @param {object} authContext - Authentication context
 * @returns {Promise<string>} SubjectAndAppToken formatted header value
 */
async function buildCompositeToken(authContext) {
  const { subjectToken, tenantId } = authContext;
  const publisherTenantId = process.env.TENANT_ID;

  if (!subjectToken) {
    throw new Error('Subject token is required');
  }

  // Exchange user's subject token for Fabric OBO token

```

```

const fabricOboToken = await getFabricOboToken(subjectToken, tenantId);

// Acquire S2S token using publisher tenant
const fabricS2SToken = await getFabricS2SToken(publisherTenantId);

// Combine into SubjectAndAppToken format
return `SubjectAndAppToken1.0 subjectToken="${fabricOboToken}",
appToken="${fabricS2SToken}"`;
}

```

Calling Fabric APIs Example

JavaScript

```

const axios = require('axios');

/**
 * Call Fabric workload control API
 */
async function callFabricApi(authContext, workspaceId, itemId) {
  // Build composite token
  const authHeader = await buildCompositeToken(authContext);

  // Call Fabric API
  const response = await axios.get(
    `https://api.fabric.microsoft.com/v1/workspaces/${workspaceId}/items/${itemId}`,
    {
      headers: {
        'Authorization': authHeader,
        'Content-Type': 'application/json'
      }
    }
  );

  return response.data;
}

```

Complete Example: Job Execution with Authentication

Here's a complete example of a job execution endpoint with authentication:

JavaScript

```

const express = require('express');
const app = express();

```

```

app.post('/api/jobs/:jobType/instances/:instanceId', async (req, res) => {
  // Authenticate the request from Fabric
  const authenticated = await authenticateControlPlaneCall(req, res);
  if (!authenticated) {
    return; // Response already sent
  }

  const { jobType, instanceId } = req.params;
  const { workspaceId, itemId } = req.body;
  const { subjectToken, tenantId, userId } = req.authContext;

  console.log(`Starting job ${jobType} for user ${userId}`);

  try {
    // Get OneLake token to access item data
    const oneLakeToken = await getOneLakeToken(subjectToken, tenantId);

    // Read data from OneLake
    const itemData = await readItemData(oneLakeToken, workspaceId, itemId);

    // Process the job
    const result = await processJob(jobType, itemData);

    // Write results back to OneLake
    await writeResults(oneLakeToken, workspaceId, itemId, result);

    // Return success
    res.status(202).json({
      status: 'InProgress',
      instanceId: instanceId,
      message: 'Job started successfully'
    });
  } catch (error) {
    console.error(`Job ${instanceId} failed:`, error);
    res.status(500).json({
      status: 'Failed',
      instanceId: instanceId,
      error: error.message
    });
  }
});

```

Long-Running Operations

For long-running operations like jobs, tokens may expire before completion. Implement token refresh logic:

JavaScript

```
// Store refresh tokens securely
const tokenCache = new Map();

/**
 * Get cached token or acquire new one
 */
async function getOrRefreshToken(userToken, tenantId, scope, cacheKey) {
    const cached = tokenCache.get(cacheKey);

    // Check if cached token is still valid (with 5 min buffer)
    if (cached && cached.expiresAt > Date.now() + 300000) {
        return cached.token;
    }

    // Acquire new token
    const response = await getTokenForScopeWithRefresh(userToken, tenantId, scope);

    // Cache the token
    tokenCache.set(cacheKey, {
        token: response.access_token,
        expiresAt: Date.now() + (response.expires_in * 1000)
    });

    return response.access_token;
}
```

For more information about handling long-running OBO processes, see [Long-running OBO processes](#).

Security Best Practices

Token Storage

- Never log complete tokens - only log last 4 characters for debugging
- Store tokens securely in memory only
- Clear tokens from memory after use
- Don't persist tokens to disk or databases

Token Validation

- Always validate the appToken in SubjectAndAppToken format
- Validate the subjectToken only if present
- Verify token signatures against Azure AD keys
- Check token expiration and not-before times
- Validate issuer, audience, and tenant claims

- Verify required claims (scp, idtyp, etc.)

Handling Missing User Context

- Design your remote endpoint to handle both user-delegated and app-only scenarios
- Check `hasSubjectContext` in `authContext` before accessing user-specific properties
- For operations requiring user context, set `requireSubjectToken: true` in middleware options
- For delete operations or system operations, allow missing `subjectToken`
- Document which operations support app-only context and which require user context

Environment Configuration

JavaScript

```
// Required environment variables
const requiredEnvVars = [
  'BACKEND_APPID',           // Your workload's app registration ID
  'BACKEND_CLIENT_SECRET',  // Your app's client secret
  'TENANT_ID',              // Your publisher tenant ID
  'BACKEND_AUDIENCE'        // Expected token audience
];

// Validate on startup
requiredEnvVars.forEach(varName => {
  if (!process.env[varName]) {
    throw new Error(`Missing required environment variable: ${varName}`);
  }
});
```

Error Handling and Consent Propagation

When token exchange fails due to missing consent or scopes, your backend should propagate these errors to the frontend UX to prompt the user for consent. Common consent-related errors include:

- **AADSTS65001** - User or admin hasn't consented to use the application
- **AADSTS65005** - The application requires specific scopes that haven't been consented

JavaScript

```
/**
 * Handle token exchange with consent error propagation
 * Consent errors should be returned to the frontend to trigger consent flow
 */
```

```

async function handleTokenExchangeWithConsent(req, res) {
  const { subjectToken, tenantId } = req.authContext;
  const clientId = process.env.BACKEND_APPID;

  try {
    const token = await getTokenForScope(subjectToken, tenantId, ONELAKE_SCOPE);
    // Use token for OneLake operations...

  } catch (error) {
    // Check for consent-related errors
    if (error.message.includes('AADSTS65001') ||
error.message.includes('AADSTS65005')) {
      // Extract scope from error if available
      const requiredScope = extractScopeFromError(error.message) ||
'https://storage.azure.com/.default';

      // Build consent URL for the frontend to redirect the user
      const consentUrl = buildConsentUrl({
        clientId: clientId,
        tenantId: tenantId,
        scope: requiredScope,
        redirectUri: process.env.FRONTEND_URL || 'https://your-
frontend.azurewebsites.net'
      });

      // Return consent required response
      return res.status(403).json({
        error: 'ConsentRequired',
        errorCode: error.message.includes('AADSTS65001') ? 'AADSTS65001' :
'AADSTS65005',
        message: 'User consent is required to access this resource',
        consentUrl: consentUrl,
        requiredScope: requiredScope
      });
    }

    // Other errors should be handled normally
    throw error;
  }
}

/**
 * Build consent URL for user authorization
 * @param {object} options - Consent URL options
 * @returns {string} Formatted consent URL
 */
function buildConsentUrl(options) {
  const { clientId, tenantId, scope, redirectUri } = options;

  const params = new URLSearchParams({
    client_id: clientId,
    response_type: 'code',
    redirect_uri: redirectUri,
    response_mode: 'query',
    scope: scope,

```



```

    state: 'consent_required' // Your frontend can use this to handle the redirect
  });

  return `https://login.microsoftonline.com/${tenantId}/oauth2/v2.0/authorize?
${params.toString()}`;
}

/**
 * Extract scope from error message
 */
function extractScopeFromError(errorMessage) {
  // Azure AD error messages often include the missing scope
  const scopeMatch = errorMessage.match(/scope[s]?[:\s]+([^\s,]+)/);
  return scopeMatch ? scopeMatch[1] : null;
}

```

Frontend Consent Handling

When your frontend receives a `ConsentRequired` error, it should redirect the user to the consent URL:

JavaScript

```

// Frontend code to handle consent errors
async function callBackendApi(endpoint, data) {
  try {
    const response = await fetch(endpoint, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });

    if (response.status === 403) {
      const error = await response.json();

      if (error.error === 'ConsentRequired' && error.consentUrl) {
        // Redirect user to consent page
        window.location.href = error.consentUrl;
        return;
      }
    }

    return await response.json();
  } catch (error) {
    console.error('API call failed:', error);
    throw error;
  }
}

```

ⓘ Note

The [Fabric workload development sample](#) [↗] includes complete examples of consent error handling and propagation to the frontend UX. See the sample's authentication middleware and frontend error handling for production-ready implementations.

Additional Error Scenarios

JavaScript

```
// Handle various token exchange errors
try {
  const token = await getTokenForScope(subjectToken, tenantId, scope);
} catch (error) {
  const errorMessage = error.message;

  if (errorMessage.includes('AADSTS65001') || errorMessage.includes('AADSTS65005')) {
    // Consent required - propagate to frontend
    return propagateConsentError(res, error, tenantId);
  } else if (errorMessage.includes('AADSTS50013')) {
    // Invalid assertion - token may be expired
    return res.status(401).json({
      error: 'InvalidToken',
      message: 'The provided token is invalid or expired'
    });
  } else if (errorMessage.includes('AADSTS700016')) {
    // Application not found in tenant
    return res.status(400).json({
      error: 'ApplicationNotFound',
      message: 'Application is not configured in this tenant'
    });
  }

  // Unknown error
  throw error;
}
```

Troubleshooting

Common Authentication Issues

Error: "Invalid token format"

- Verify the Authorization header starts with `SubjectAndAppToken1.0`
- Check that both `subjectToken` and `appToken` are present
- Ensure tokens are properly quoted in the header

Error: "Token validation failed"

- Verify `BACKEND_APPID` matches your app registration
- Check that `BACKEND_AUDIENCE` is correctly configured
- Ensure tokens haven't expired
- Verify issuer matches your Azure AD tenant

Error: "Token exchange failed: AADSTS65001"

- User consent is required for the requested scope
- Ensure your app registration has the necessary API permissions
- Verify admin consent has been granted if required

Error: "App token not from Fabric"

- Verify the `appToken`'s `appid` claim matches Fabric's app ID
- Check that you're testing in the correct environment (dev/production)

Related Content

- [Enable Remote Endpoints](#)
- [Enable Remote Jobs](#)
- [Enable Item Lifecycle Notifications](#)
- [Use Endpoint Resolution Service](#)
- [Long-running OBO processes](#)
- [Fabric Workload Development Sample](#) [↗](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Enable Item Lifecycle Notifications

Important

Starting at the end of April 2026, Fabric will begin sending soft delete and restore notifications to workload endpoints. If your workload has `onDelete` enabled in the workload manifest, make sure your endpoints are ready to handle these new lifecycle events.

Item lifecycle notifications provide workload builders with the capability to receive notifications about lifecycle events for custom items, even when the workload UX is not in the loop. This setting enables your item to respond to item operations regardless of how they were initiated.

Overview

Item lifecycle notifications are essential for scenarios where items are created, updated, or deleted outside of your workload's user interface. This service notifies your workload whenever an item lifecycle event occurs, allowing you to:

- Set up or tear down infrastructure for new or deleted items
- Validate licenses or quotas before allowing operations
- Synchronize state with external systems
- Implement custom business logic for item operations
- Handle soft delete scenarios with custom cleanup logic

When Lifecycle Notifications Are Triggered

Lifecycle notifications are triggered for item operations that originate from multiple sources:

Public APIs

Items can be created, updated, or deleted through [Fabric's public REST APIs](#) without any interaction with your workload's UI. For example:

- Automation scripts using the Items API
- Third-party applications integrating with Fabric
- Administrative operations performed programmatically

CI/CD Pipelines

Items are deployed through [Fabric's CI/CD capabilities](#) as part of automated deployment pipelines. In this scenario:

- Items are created or updated as part of Git integration workflows
- Deployment pipelines promote items across environments
- No user interface interaction occurs during the deployment process

Platform-Controlled UX Flows

Even within the Fabric user interface, the platform controls certain creation and management flows. Your workload receives notifications for these operations to maintain consistency:

- Item creation through platform dialogs
- Workspace-level operations affecting items
- Cross-workspace copy or move operations

Lifecycle Events

Your workload can receive notifications for the following lifecycle events:

Create

Triggered when a new item is created. You can:

- Set up required infrastructure (databases, compute resources)
- Validate licenses or capacity quotas
- Initialize item-specific resources
- **Block the operation** if validation fails or prerequisites aren't met

Update

Triggered when an item's definition or properties are modified. You can:

- Update associated infrastructure
- Validate the new configuration
- Synchronize changes with external systems
- **Block the operation** if the update violates business rules

Delete

Triggered when an item is deleted. The request includes a `deleteType` field indicating whether it's a hard or soft delete:

- **Hard delete:** Permanent deletion - clean up all associated resources
- **Soft delete:** Item is marked for deletion but can be restored later - retain sufficient metadata and resources to support restoration

Your workload can choose different strategies for soft delete:

- Stop expensive compute resources while retaining data
- Archive critical data to lower-cost storage
- Retain full state for the recovery period

⚠ Note

The subject token may not be available during delete operations. Your endpoint must handle authentication accordingly.

Restore

Triggered when a previously soft-deleted item is restored. Your endpoint receives the item definition and should:

- Re-allocate resources that were freed during soft delete
- Restore item state and resume services
- Reinitialize compute or storage resources as needed

Request Format

Fabric calls a separate endpoint for each lifecycle event. Each request includes:

URL Path Parameters

The item's identifying information is provided in the URL path:

- `workspaceId` - The workspace ID (UUID)
- `itemType` - The item type (for example, `Contoso.FinanceAnalytics.Forecast`)
- `itemId` - The item ID (UUID)

Authentication Header

The `Authorization` header uses the `SubjectAndAppToken1.0` scheme, which contains both a delegated user token and an app-only token:

```
SubjectAndAppToken1.0 subjectToken="<delegated token>", appToken="<S2S token>"
```

This dual-token format allows your workload to validate the request origin, verify user context, and call other services. Extra required headers include `ActivityId`, `RequestId`, and `x-ms-client-tenant-id`.

For details on validating these tokens, see [Authenticate Remote Endpoints](#).

Request Body

The request body varies by operation:

- **Create, Update, and Restore:** The body contains an `ItemDefinition` object with a `parts` array. Each part has a `path`, `payload` (Base64-encoded), and `payloadType`.
- **Delete:** The body contains a `deleteType` field with a value of `Hard` or `Soft`.

The information in the request body allows you to:

- Extract configuration for infrastructure setup during create and restore
- Understand what changed in update operations
- Make informed decisions about blocking operations
- Choose the appropriate cleanup strategy for hard vs. soft deletes

Blocking Operations

For **create** and **update** operations, your workload can block the operation by returning an error response from the lifecycle endpoint. This is useful for:

- **License validation:** Prevent item creation if the user doesn't have required licenses
- **Quota enforcement:** Block creation if workspace or capacity limits are exceeded
- **Configuration validation:** Reject updates that would create invalid states
- **Security policies:** Enforce organizational policies on item configurations

When you block an operation:

1. Return an error status code (for example, 400 or 403) from your endpoint
2. Provide a clear error message explaining why the operation was blocked

3. The platform displays your error message to the user
4. The item operation isn't completed

Important

Delete operations (both hard and soft) can't be blocked. Your workload must handle cleanup regardless of the item state. Restore operations can be blocked by returning an error response.

Implementing Lifecycle Notification Handling

Lifecycle notifications defined in custom items can't be handled directly within Fabric. Instead, you need to implement a remote endpoint that Fabric calls when a lifecycle event occurs. This gives you full control over where and how your notification handling logic executes.

The remote endpoint can be:

- **An Azure Function** - Simple, serverless execution for lightweight notification handling
- **An Azure Container Instance** - For notifications requiring specific runtime environments
- **A custom web service** - For complex orchestration and processing
- **Any compute service** - As long as it can expose an HTTP endpoint and handle the notification

For detailed information about implementing and configuring remote endpoints, see [Enable Remote Endpoints](#).

Configuration

To enable lifecycle notifications for your custom item, configure the notifications in your item manifest.

Define Lifecycle Notifications in the Item Manifest

Add the lifecycle notification configuration to your item manifest:

XML

```
<Item>
  <Name>YourItemType</Name>
  <!-- Other item configuration -->
```



```
<LifecycleOperationsNotifications>
  <OnCreate>true</OnCreate>
  <OnUpdate>true</OnUpdate>
  <OnDelete>true</OnDelete>
</LifecycleOperationsNotifications>
</Item>
```

Configure only the events you need to handle. Set the value to `false` or omit the element if you don't need notifications for a specific event type.

Important

When `OnDelete` is enabled, your workload must also implement the `OnRestoreItem` endpoint. Fabric calls this endpoint when a soft-deleted item is restored, so your workload must be prepared to handle restore notifications if it handles delete notifications.

Endpoint Requirements

Your lifecycle endpoint must:

- Be publicly accessible over HTTPS
- Respond within a reasonable timeout (recommended: 30 seconds)
- Return appropriate HTTP status codes
- Handle authentication using the provided token

For create and update operations that you want to block:

- Return 4xx status code with error details
- Include a user-friendly error message in the response body

For operations that succeed, or for delete and restore operations:

- Return 200 OK status code
- Optionally include logging or tracking information

Local Development Support

For local development and testing, the Fabric Extensibility Toolkit provides a built-in stub implementation in the development server. This allows you to test item lifecycle notifications on your local machine without deploying to Azure.

How It Works

When you run the local development server (see [Get started with Fabric Extensibility Toolkit](#) and [Register Local Web Server](#)), all lifecycle notifications defined in your item manifest are automatically redirected to your local machine.

When a lifecycle event occurs in dev mode:

1. The notification is intercepted by the local development server
2. Log statements appear in your local console showing the event details
3. A sample implementation demonstrates how to interact with the item definition and OneLake storage
4. You receive a token that can be used to call any Fabric operation

This stub implementation provides:

- **Immediate feedback** - See log statements in your console as lifecycle events occur
- **Sample code** - Learn how to structure your notification handling logic
- **OneLake integration examples** - See how to access the item's OneLake storage during notifications
- **Token handling** - Understand how to obtain and use Fabric authentication tokens
- **Item definition access** - View the complete item definition payload

Using the Development Stub

To use the local development server for lifecycle notification testing:

1. Start your local development server following the [quickstart guide](#)
2. Register your local web server using the [registration steps](#)
3. Configure lifecycle notifications in your item manifest as described above
4. Perform item operations (create, update, delete) in Fabric
5. Observe the notification details in your local console
6. Review the sample code for OneLake interaction and token handling in the development server logs

OneLake Interaction Example

The development stub includes sample code showing how to interact with the item's OneLake storage:

```
JavaScript
```

```
// Example from the development stub
const tokenExchangeService = require('./tokenExchangeService');
const oneLakeClientService = require('./oneLakeClientService');

async function handleCreateWithOneLakeAccess(req) {
  const { workspaceId, itemId } = req.params;
  const { subjectToken, tenantId } = req.authContext;

  // Exchange user token for OneLake-scoped token
  const oneLakeToken = await tokenExchangeService.getTokenForScope(
    subjectToken, tenantId, oneLakeClientService.ONELAKE_SCOPE);

  // Write to the item's OneLake storage
  const filePath = oneLakeClientService.getOneLakeFilePath(
    workspaceId, itemId, 'config.json');
  const content = JSON.stringify(req.body, null, 2);
  await oneLakeClientService.writeToOneLakeFile(oneLakeToken, filePath, content);

  console.log(`Initialized item configuration in OneLake at ${filePath}`);
}
```

This local development experience allows you to build and test your lifecycle notification logic before deploying to production endpoints.

Implementation Example

The following example uses Node.js and Express, based on the [Fabric Extensibility Toolkit](#) reference implementation. Each lifecycle event has its own endpoint that Fabric calls with the appropriate request body. For full API details, see the [Item Lifecycle REST API reference](#).

JavaScript

```
const express = require('express');
const { authenticateControlPlaneCall } = require('./authentication');
const router = express.Router();

// POST /workspaces/{workspaceId}/items/{itemType}/{itemId}/onCreateItem
// Request body: { definition: { parts: [{ path, payload, payloadType }] } }
router.post('/workspaces/:workspaceId/items/:itemType/:itemId/onCreateItem',
  async (req, res) => {
    const authResult = await authenticateControlPlaneCall(req, res);
    if (!authResult) return;

    const { workspaceId, itemId, itemType } = req.params;
    const { definition } = req.body;

    // Validate license before allowing creation
    const hasLicense = await checkUserLicense(req.authContext.userId);
    if (!hasLicense) {
      return res.status(403).json({
```

```

        errorCode: 'LicenseRequired',
        message: 'User does not have the required license for this item type.',
        source: 'User',
        isPermanent: true
    });
}

// Set up infrastructure for the new item
await provisionResources(workspaceId, itemId, definition);

console.log(`Created item ${itemId} of type ${itemType}`);
res.status(200).json({});
}
);

// POST /workspaces/{workspaceId}/items/{itemType}/{itemId}/onUpdateItem
// Request body: { definition: { parts: [{ path, payload, payloadType }] } }
router.post('/workspaces/:workspaceId/items/:itemType/:itemId/onUpdateItem',
    async (req, res) => {
        const authResult = await authenticateControlPlaneCall(req, res);
        if (!authResult) return;

        const { workspaceId, itemId } = req.params;
        const { definition } = req.body;

        // Validate the updated definition
        if (!isValidConfiguration(definition)) {
            return res.status(400).json({
                errorCode: 'InvalidConfiguration',
                message: 'Invalid configuration: required settings are missing.',
                source: 'User',
                isPermanent: true
            });
        }

        await updateResources(workspaceId, itemId, definition);

        console.log(`Updated item ${itemId}`);
        res.status(200).json({});
    }
);

// POST /workspaces/{workspaceId}/items/{itemType}/{itemId}/onDeleteItem
// Request body: { deleteType: "Hard" | "Soft" }
// Note: Subject token may not be available during delete operations.
router.post('/workspaces/:workspaceId/items/:itemType/:itemId/onDeleteItem',
    async (req, res) => {
        const authResult = await authenticateControlPlaneCall(req, res,
            { requireSubjectToken: false });
        if (!authResult) return;

        const { workspaceId, itemId } = req.params;
        const { deleteType } = req.body;

        if (deleteType === 'Hard') {

```

```

    await deleteAllResources(workspaceId, itemId);
    console.log(`Hard deleted item ${itemId}`);
  } else if (deleteType === 'Soft') {
    // Retain metadata for recovery; optionally free expensive resources
    await stopComputeResources(workspaceId, itemId);
    console.log(`Soft deleted item ${itemId}`);
  }

  res.status(200).json({});
}
);

// POST /workspaces/{workspaceId}/items/{itemType}/{itemId}/onRestoreItem
// Request body: { definition: { parts: [{ path, payload, payloadType }] } }
// Called when a soft-deleted item is restored.
router.post('/workspaces/:workspaceId/items/:itemType/:itemId/onRestoreItem',
  async (req, res) => {
    const authResult = await authenticateControlPlaneCall(req, res,
      { requireSubjectToken: false });
    if (!authResult) return;

    const { workspaceId, itemId } = req.params;
    const { definition } = req.body;

    // Re-allocate resources freed during soft delete
    await restoreResources(workspaceId, itemId, definition);

    console.log(`Restored item ${itemId}`);
    res.status(200).json({});
  }
);

```

Use Cases

Infrastructure Management

Automatically provision and deprovision infrastructure based on item lifecycle:

JavaScript

```

// In onCreateItem handler: Provision resources
await createSqlDatabase(itemId);
await createStorageAccount(itemId);
await createComputeCluster(itemId);

// In onDeleteItem handler (deleteType === 'Hard'): Clean up resources
await deleteSqlDatabase(itemId);
await deleteStorageAccount(itemId);
await deleteComputeCluster(itemId);

```

License and Quota Validation

Enforce licensing and capacity requirements in the `onCreateItem` handler:

JavaScript

```
// Check user license before allowing creation
const userLicense = await getUserLicense(req.authContext.userId);
if (userLicense.tier < REQUIRED_TIER_PREMIUM) {
  return res.status(403).json({
    errorCode: 'LicenseRequired',
    message: 'Premium license required for this item type.',
    source: 'User',
    isPermanent: true
  });
}

// Check workspace capacity
const workspaceUsage = await getWorkspaceUsage(workspaceId);
if (workspaceUsage.itemCount >= workspaceUsage.maxItems) {
  return res.status(403).json({
    errorCode: 'QuotaExceeded',
    message: 'Workspace item limit reached.',
    source: 'User',
    isPermanent: false
  });
}
```

External System Synchronization

Keep external systems in sync with Fabric items:

JavaScript

```
// In onCreateItem handler: Sync item to external catalog
const { workspaceId, itemId, itemType } = req.params;
await externalCatalog.registerItem({
  id: itemId,
  type: itemType,
  workspaceId: workspaceId
});

// Update external monitoring
await monitoringSystem.trackItemCreated(workspaceId, itemId);
```

Delete and Restore Strategy

Implement different cleanup strategies based on `deleteType`, and handle restore:

JavaScript

```
// In onDeleteItem handler:
const { deleteType } = req.body;

if (deleteType === 'Soft') {
  // Free expensive resources but retain data for potential recovery
  await stopExpensiveCompute(itemId);
  await archiveDataToLowCostStorage(itemId);
}

if (deleteType === 'Hard') {
  // Permanent deletion - clean up everything
  await deleteAllResources(itemId);
}

// In onRestoreItem handler:
// Re-allocate resources freed during soft delete
await restartCompute(itemId);
await restoreDataFromArchive(itemId);
```

Best Practices

Respond Quickly

Lifecycle endpoints should respond as quickly as possible:

- Perform validation and quick setup operations synchronously
- Queue long-running infrastructure provisioning for background processing
- Return success/failure quickly to avoid timeouts

Handle Idempotency

Item operations may be retried, so your endpoint should be idempotent:

- Check if infrastructure already exists before creating
- Use unique identifiers to track operations
- Handle duplicate notifications gracefully

Provide Clear Error Messages

When blocking operations, provide helpful error messages:

JavaScript

```
// Good: Structured ErrorResponse with a clear message
return res.status(403).json({
  errorCode: 'LicenseRequired',
  message: 'Cannot create item: Premium license required. Upgrade your
subscription.',
  source: 'User',
  isPermanent: true
});

// Poor: Missing structured error information
return res.status(403).json({ message: 'Forbidden' });
```

Security

- Validate the authentication token on every request
- Use the token to verify the operation is legitimate
- Don't trust item IDs or workspace IDs without validation
- See [Authenticate Remote Endpoints](#) for security best practices

Monitoring and Logging

- Log all lifecycle notifications received
- Track infrastructure provisioning success/failure
- Monitor endpoint performance and errors
- Use the notification data for audit trails

Troubleshooting

Notifications Not Received

- Verify the endpoint URL is correct in the manifest
- Check that the endpoint is publicly accessible
- Ensure HTTPS is properly configured
- Review endpoint logs for errors

Operations Timing Out

- Optimize endpoint response time
- Move long-running operations to background jobs
- Increase endpoint timeout settings if needed
- Review infrastructure provisioning performance

Blocking Not Working

- Verify you're returning 4xx status codes for blocked operations
- Check that error messages are included in the response
- Ensure the endpoint responds within the timeout window
- Remember that delete operations cannot be blocked - the Fabric platform marks the item as deleted regardless of the response of your workload.

Related Content

- [Authenticate Remote Endpoints](#)
- [Enable Remote Endpoints](#)
- [Store Item Definition](#)
- [Define Jobs for Your Workload](#)
- [Get started with Fabric Extensibility Toolkit](#)
- [Register Local Web Server](#)
- [Fabric Item Management API](#)
- [Fabric CI/CD Overview](#)
- [Item Lifecycle REST API Reference](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 04/15/2026

How-To: Access Fabric APIs

To make it easy for Workload developers to access the Fabric Public APIs, we provide the [FabricPlatformAPIClient](#) client that abstracts the calls. This client primarily uses the **On-Behalf-Of (OBO)** flow to access Fabric APIs using the current user's credentials and permissions.

Primary Authentication Method: On-Behalf-Of (OBO) Flow

The extensibility toolkit is designed around the **On-Behalf-Of (OBO) flow**, which is the recommended and primary way to access Fabric APIs. This flow:

- **Uses user credentials:** Operates with the permissions of the currently signed-in user
- **Maintains security context:** Ensures users can only access resources they have permission to
- **Simplifies development:** No need to manage service principal credentials
- **Follows best practices:** Aligns with Microsoft Fabric's security model

Creating a FabricPlatformAPIClient

Using On-Behalf-Of flow (Recommended)

TypeScript

```
import { getWorkloadClient } from "../controller/WorkloadClient";
import { FabricPlatformAPIClient } from "../clients/FabricPlatformAPIClient";

// Create the client using the workload client (uses current user's credentials via OBO flow)
const client = FabricPlatformAPIClient.create(getWorkloadClient());
```

Using a service principal (Advanced scenarios only)

ⓘ Note

Service principal authentication is only needed for advanced scenarios where you need to access Fabric APIs without a user context, such as background processing or automated workflows. For most extensibility toolkit workloads, use the OBO flow instead.

TypeScript

```
import { FabricPlatformAPIClient } from "../clients/FabricPlatformAPIClient";

// Create with service principal authentication (advanced scenarios)
const client = FabricPlatformAPIClient.createWithServicePrincipal(
  "your-client-id",
  "your-client-secret",
  "your-tenant-id"
);
```

Example: Creating a new item

Here's an example of how to create a new Notebook item using the OBO flow (recommended):

TypeScript

```
import { FabricPlatformAPIClient } from "../clients/FabricPlatformAPIClient";
import { getWorkloadClient } from "../controller/WorkloadClient";

async function createNewNotebook() {
  try {
    // Create the client using OBO flow with current user's credentials
    const client = FabricPlatformAPIClient.create(getWorkloadClient());

    // Get the current workspace ID
    const workspace = await client.workspaces.getCurrentWorkspace();
    const workspaceId = workspace.id;

    // Create a new notebook item
    const newItem = await client.items.createItem(workspaceId, {
      displayName: "My New Notebook",
      description: "Created via FabricPlatformAPIClient",
      type: "Notebook", // Item type code
      // Optional: Specify folder ID if you want to create in a specific folder
      // folderId: "your-folder-id",

      // Optional: Add item-specific creation properties
      creationPayload: {
        notebookDefaultLanguage: "python"
      }
    });

    console.log("Created new item:", newItem);
    return newItem;
  } catch (error) {
    console.error("Error creating item:", error);
    throw error;
  }
}
```

Example: Creating an item with a definition

For more complex items that require a definition (like a Lakehouse), you can include the definition using the OBO flow:

TypeScript

```
async function createLakehouse() {
  // Create the client using OBO flow with current user's credentials
  const client = FabricPlatformAPIClient.create(getWorkloadClient());
  const workspace = await client.workspaces.getCurrentWorkspace();

  const newLakehouse = await client.items.createItem(workspace.id, {
    displayName: "My New Lakehouse",
    description: "Sample Lakehouse created programmatically",
    type: "Lakehouse",
    definition: {
      format: "json",
      parts: [
        {
          path: "definition/settings.json",
          content: JSON.stringify({
            schemaVersion: "1.0",
            // Lakehouse-specific settings
            settings: {
              defaultDatabase: "MyDatabase"
            }
          })
        }
      ]
    }
  });

  return newLakehouse;
}
```

Working with other Fabric resources

The FabricPlatformAPIClient provides access to various Fabric resources using the OBO flow:

TypeScript

```
// Create the client using OBO flow with current user's credentials
const client = FabricPlatformAPIClient.create(getWorkloadClient());

// List all workspaces the user has access to
const workspaces = await client.workspaces.listWorkspaces();

// Get workspace capacities
```

```
const capacities = await client.capacities.listCapacities();

// Work with OneLake (with user's permissions)
const tables = await client.oneLake.listTables(workspaceId, lakehouseId);

// Work with connections (scoped to user's access)
const connections = await client.connections.listConnections(workspaceId);
```

Key Benefits of Using FabricPlatformAPIClient with OBO Flow

- **Security:** All operations respect the current user's permissions and security context
- **Simplicity:** No need to manage service principal credentials or authentication flows
- **Consistency:** Works seamlessly with the extensibility toolkit's frontend-only architecture
- **Auditability:** All actions are performed and logged under the actual user's account
- **Best practices:** Follows Microsoft's recommended patterns for Fabric API access in extensibility scenarios

Last updated on 12/15/2025

How-To: Acquire Microsoft Entra tokens

This quick start guide shows you how to acquire Microsoft Entra tokens in your Fabric extension to access APIs. For comprehensive API reference and advanced scenarios, see the [Authentication JavaScript API](#) documentation.

Prerequisites

- A Fabric extension project set up with the Extensibility Toolkit
- Basic understanding of JavaScript/TypeScript

Basic token acquisition

The simplest way to get a token for Fabric APIs:

JavaScript

```
// Get a token for Fabric APIs
const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: [] });

// Use the token to call Fabric APIs
const response = await fetch('https://api.fabric.microsoft.com/v1/workspaces', {
  headers: {
    'Authorization': `Bearer ${token.token}`,
    'Content-Type': 'application/json'
  }
});
```

Get tokens for specific services

For Azure services or custom applications, specify the required scopes:

JavaScript

```
// Get token for Azure Storage
const storageToken = await workloadClient.auth.acquireFrontendAccessToken({
  scopes: ['https://storage.azure.com/user_impersonation']
});

// Get token for Microsoft Graph
const graphToken = await workloadClient.auth.acquireFrontendAccessToken({
  scopes: ['https://graph.microsoft.com/User.Read']
});
```

Error handling

Handle common authentication errors:

JavaScript

```
try {  
  const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: []  
});  
  // Use token...  
} catch (error) {  
  console.error('Authentication failed:', error.message);  
  // Handle error appropriately  
}
```

Next steps

- For comprehensive API reference and advanced scenarios, see [Authentication JavaScript API](#)
- Learn how to [Access Fabric APIs](#) with pre-built wrappers
- Review [Authentication guidelines](#) for best practices
- Explore the [Starter Kit examples](#) [↗](#) for complete implementations

Related content

- [Authentication overview](#)
- [Key concepts and features](#)

Last updated on 12/15/2025

How-To: Add custom settings to an item

Custom settings are already enabled in the Fabric Extensibility Toolkit. The worker automatically registers a custom settings tab for each item type with the route pattern `/{{ItemType}}Item-settings/{{itemId}}`.

Implementation steps

1. Create your settings component

Create a React component for your settings interface:

File: `Workload/app/items/[YourItem]Item/[YourItem]ItemSettings.tsx`

TypeScript

```
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { Field, Input, Button } from '@fluentui/react-components';
import { getWorkloadItem, saveWorkloadItem } from
'../../../../../controller/ItemCRUDController';

export function YourItemSettings(props) {
  const { workloadClient } = props;
  const { itemObjectId } = useParams();
  const [item, setItem] = useState();
  const [settings, setSettings] = useState({ apiEndpoint: '', maxRetries: 3 });

  // Load item and initialize settings
  useEffect(() => {
    // Load item data and populate form
  }, [itemObjectId]);

  const handleSave = async () => {
    // Update item definition with new settings
    const updatedItem = { ...item, definition: { ...item.definition, settings }
  };

  await saveWorkloadItem(workloadClient, updatedItem);
};

return (
  <div className="item-settings">
    <Field label="API Endpoint">
      <Input value={settings.apiEndpoint} onChange={(e, data) =>
        setSettings(prev => ({ ...prev, apiEndpoint: data.value })} />
    </Field>
    <Button appearance="primary" onClick={handleSave}>Save Settings</Button>
  </div>
```



```
);  
}
```

2. Add routing

Add the settings route to your app:

File: `Workload/app/App.tsx`

TypeScript

```
import { YourItemSettings } from './items/YourItemItem/YourItemSettings';  
  
// Add route  
<Route path="/YourItemItem-settings/:itemObjectId" element={<YourItemSettings  
  {...props} />} />
```

3. Update your data model

Extend your item definition to include settings:

File: `Workload/app/items/[YourItem]Item/[YourItem]ItemDefinition.ts`

TypeScript

```
export interface YourItemDefinition {  
  message?: string;  
  settings?: {  
    apiEndpoint: string;  
    maxRetries: number;  
  };  
}
```

4. Wire up the settings button

Ensure your ribbon opens the settings dialog:

TypeScript

```
import { createSettingsAction } from '../../controls/ItemEditor';  
import { callOpenSettings } from '../../controller/SettingsController';  
  
const handleSettings = () => callOpenSettings(workloadClient, item,  
  'customItemSettings');  
const actions = [createSettingsAction(handleSettings, 'Settings')];
```

Key points

- **Worker configuration:** Custom settings are enabled by default
- **Route pattern:** Follow `/{{ItemType}}Item-settings/{{itemId}}` pattern
- **Data persistence:** Use `saveWorkloadItem()` to save settings
- **UI consistency:** Use Fluent UI components for consistent styling

Related guides

- [Create Fabric Items](#)
- [Create Custom About Pages](#)
- [Store Item Definition](#)

Last updated on 12/15/2025

How To: Add a custom About page

Custom About pages provide users with item information, metadata, and version details through Fabric's standard settings interface. The About tab is the default tab in the settings dialog.

Implementation steps

1. Worker configuration (already set)

The About page route is already configured in the worker by default with the pattern

```
/{{ItemType}}Item-about/{{itemId}}.
```

2. Create About component

Create a React component for your custom About page:

File: `Workload/app/items/[YourItem]Item/[YourItem]ItemAbout.tsx`

TypeScript

```
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { useTranslation } from 'react-i18next';
import {
  Card, CardHeader, CardBody, Title2, Title3, Body1, Badge, Text, Spinner
} from '@fluentui/react-components';
import { InfoRegular, SettingsRegular } from '@fluentui/react-icons';
import { getWorkloadItem } from '../../controller/ItemCRUDController';

export function YourItemAbout(props) {
  const { workloadClient } = props;
  const { itemObjectId } = useParams();
  const { t } = useTranslation();

  const [item, setItem] = useState();
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadItem = async () => {
      if (!itemObjectId) return;
      try {
        const loadedItem = await getWorkloadItem(workloadClient,
itemObjectId);
        setItem(loadedItem);
      } catch (error) {
        console.error('Failed to load item:', error);
      }
    };
    loadItem();
  }, [itemObjectId, workloadClient]);
```

```

        } finally {
            setLoading(false);
        }
    };
    loadItem();
}, [workloadClient, itemObjectId]);

if (loading) {
    return <Spinner label="Loading item information..." />;
}

if (!item) {
    return <Text>Unable to load item information</Text>;
}

return (
    <div className="item-about">
        <Card className="about-section">
            <CardHeader>
                <div className="about-header">
                    <InfoRegular />
                    <Title2>About This Item</Title2>
                </div>
            </CardHeader>

            <CardBody>
                <div className="info-row">
                    <Text weight="semibold">Name:</Text>
                    <Text>{item.displayName}</Text>
                </div>
                <div className="info-row">
                    <Text weight="semibold">Type:</Text>
                    <Badge appearance="outline">{item.type}</Badge>
                </div>
                <div className="info-row">
                    <Text weight="semibold">Item ID:</Text>
                    <Text className="monospace">{item.id}</Text>
                </div>
            </CardBody>
        </Card>

        <Card className="about-section">
            <CardHeader>
                <div className="about-header">
                    <SettingsRegular />
                    <Title3>Configuration</Title3>
                </div>
            </CardHeader>

            <CardBody>
                <div className="info-row">
                    <Text weight="semibold">Status:</Text>
                    <Badge appearance="filled" color={item.definition ? "brand"
: "warning"}>
                        {item.definition ? "Configured" : "Not Configured"}

```

```

        </Badge>
      </div>
      <div className="info-row">
        <Text weight="semibold">Version:</Text>
        <Text>1.0.0</Text>
      </div>
    </CardBody>
  </Card>
</div>
);
}

```

3. Add routing

Register the About component route in your app:

File: `Workload/app/App.tsx`

TypeScript

```

import { YourItemAbout } from './items/YourItemItem/YourItemAbout';

// Add route
<Route path="/YourItemItem-about/:itemObjectId" element={<YourItemAbout {...props}>
/>} />

```

4. Add basic styling

File: `Workload/app/items/[YourItem]Item/[YourItem].scss`

SCSS

```

.item-about {
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 20px;

  .about-section .about-header {
    display: flex;
    align-items: center;
    gap: 8px;
  }

  .info-row {
    display: flex;
    gap: 8px;
    align-items: center;
    margin-bottom: 12px;
  }
}

```

```

    > *:first-child {
      min-width: 120px;
    }
  }

.monospace {
  font-family: 'Consolas', monospace;
  background: var(--colorNeutralBackground2);
  padding: 2px 4px;
  border-radius: 4px;
}
}

```

Common patterns

Health status display

TypeScript

```

const [healthStatus, setHealthStatus] = useState('healthy');

// In JSX
<Badge color={healthStatus === 'healthy' ? 'brand' : 'warning'}>
  {healthStatus}
</Badge>

```

Version information

TypeScript

```

const workloadVersion = process.env.WORKLOAD_VERSION || '1.0.0';

// Display
<Text weight="semibold">Workload Version:</Text>
<Badge appearance="outline">{workloadVersion}</Badge>

```

Resource links

TypeScript

```

<div className="resource-links">
  <Link href="https://docs.microsoft.com/fabric" target="_blank">
    Documentation
  </Link>
  <Link href="https://community.fabric.microsoft.com" target="_blank">
    Community Support

```

```
</Link>  
</div>
```

Key points

- **Worker routes:** About page routes are pre-configured by the worker
- **Default tab:** About is the default tab in Fabric's settings dialog
- **Load patterns:** Use `getWorkloadItem()` to load item data
- **UI consistency:** Use Fluent UI components and standard patterns
- **Information hierarchy:** Use cards to group related information

Related guides

- [Create Custom Settings Dialogs](#)
- [Create Fabric Items](#)
- [Store Item Definition](#)

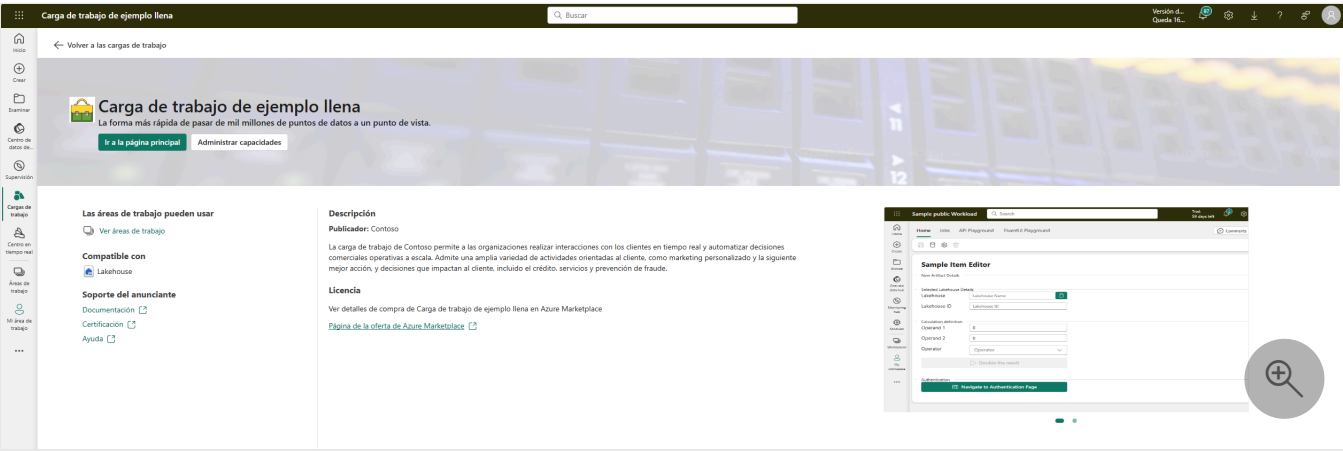
Last updated on 12/15/2025

Localize your workload in Microsoft Fabric

Localization is the process of adapting a product to serve other markets by changing the language or the content. Localizing your workload enables you to reach a wider audience and provide your customers with the best, most personalized experience. For example, a localized workload is more attractive to a user whose default language isn't English in the Microsoft Fabric workload hub.

The Fabric Extensibility Toolkit comes pre-configured with localization support, including i18next setup and example translations in English and Spanish.

The following image shows an example of a localized workload in the workload hub.



For a Fabric workload built with the Extensibility Toolkit, localization involves two distinct aspects:

- **Manifest localization:** Translating strings that are part of the Fabric shell experience (workload name, item names, descriptions, etc.). These are uploaded to Fabric as part of the manifest package and displayed in Fabric's UI outside the iframe.
- **App localization:** Translating text within your workload's iframe (your app's UI), which runs independently in the iframe using [i18next configuration](#).

These two localization systems work independently but together provide a complete localized experience.

Supported languages

Fabric currently supports the following 44 languages:

 Expand table

Language code	Language
ar	العربية (Arabic)
bg	български (Bulgarian)
ca	català (Catalan)
cs	čeština (Czech)
da	dansk (Danish)
de	Deutsche (German)
el	ελληνικά (Greek)
en-US	English (English)
es	español (Spanish)
et	eesti (Estonian)
eu	Euskal (Basque)
fi	suomi (Finnish)
fr	français (French)
gl	galego (Galician)
he	עברית (Hebrew)
hi	हिन्दी (Hindi)
hr	hrvatski (Croatian)
hu	magyar (Hungarian)
id	Bahasa Indonesia (Indonesian)
it	italiano (Italian)
ja	日本の (Japanese)
kk	Қазақ (Kazakh)
ko	한국의 (Korean)
lt	Lietuvos (Lithuanian)
lv	Latvijas (Latvian)
ms	Bahasa Melayu (Malay)

Language code	Language
nb	norsk (Norwegian)
nl	Nederlands (Dutch)
pl	polski (Polish)
pt-BR	português (Portuguese - Brazil)
pt-PT	português (Portuguese - Portugal)
ro	românesc (Romanian)
ru	русский (Russian)
sk	slovenský (Slovak)
sl	slovenski (Slovenian)
sr-Cyrl	српски (Serbian - Cyrillic)
sr-Latn	srpski (Serbian - Latin)
sv	svenska (Swedish)
th	ไทย (Thai)
tr	Türk (Turkish)
uk	український (Ukrainian)
vi	tiếng Việt (Vietnamese)
zh-Hans	中国 (Chinese - Simplified)
zh-Hant	中國 (Chinese - Traditional)

You can choose to provide some or all of these languages. The only language that's required is English (US), because it's the default language of Fabric.

Package structure

The Fabric Extensibility Toolkit uses two separate localization systems with different file locations:

Manifest localization (uploaded to Fabric)

All strings in the manifest files must be replaced with keys. These translations are uploaded to Fabric as part of the manifest package and control how your workload appears in Fabric's shell UI (outside the iframe).

For example, if your `Product.json` manifest previously started as follows:

JSON

```
{
  "name": "Product",
  "displayName": "Hello Fabric!",
  "fullDisplayName": "Hello Fabric!",
  "description": "See how to build, customize, and publish your first Fabric workload in minutes.",
  "favicon": "assets/images/HelloFabricWorkloadIcon.png",
  "icon": {
    "name": "assets/images/HelloFabricWorkloadIcon.png"
  },
  "homePage": {
    ...
  }
}
```

Your `Product.json` manifest should now be something like this example:

JSON

```
{
  "name": "Product",
  "displayName": "Workload_Display_Name",
  "fullDisplayName": "Workload_Full_Display_Name",
  "description": "Workload_Description",
  "favicon": "assets/images/HelloFabricWorkloadIcon.png",
  "icon": {
    "name": "assets/images/HelloFabricWorkloadIcon.png"
  },
  "homePage": {
    ...
  }
}
```

App localization (deployed with your workload)

Your workload app's UI strings are handled separately using i18next and are deployed with your workload application that runs in the iframe.

To support both localization systems in the Fabric Extensibility Toolkit, your package must have the following structure:

text

```

Workload/
├── Manifest/
│   ├── assets/
│   │   ├── images/
│   │   │   └── # Images and icons uploaded to Fabric with the manifest
│   │   └── locales/
│   │       ├── en-US/
│   │       │   └── translations.json # Manifest strings (uploaded to Fabric)
│   │       ├── es/
│   │       │   └── translations.json
│   │       └── # More locales for manifest
└── app/
    ├── i18n.js # i18next configuration for app UI
    ├── assets/
    │   └── locales/
    │       ├── en-US/
    │       │   └── translation.json # App UI strings (deployed with app)
    │       ├── es/
    │       │   └── translation.json
    │       └── # More locales for app UI

```

Manifest translation files

Each locale has its own folder under `Manifest/assets/locales`. Each locale contains a single file, `translations.json`. This file contains a dictionary of key/value pairs for strings that appear in Fabric's shell UI (workload hub, create experience, etc.).

For example, the `translations.json` file for English could contain:

JSON

```

{
  "Workload_Display_Name": "Hello Fabric!",
  "Workload_Full_Display_Name": "Hello Fabric!",
  "Workload_Description": "See how to build, customize, and publish your first Fabric workload in minutes.",
  "Workload_Hub_Workload_Slogan": "The fastest way to build, test, and publish your first Fabric workload.",
  "HelloWorldItem_DisplayName": "Hello World",
  "HelloWorldItem_DisplayName_Plural": "Hello Worlds",
  "HelloWorldItem_Description": "A Hello World Item to start your Dev Experience"
}

```

Whereas the `translations.json` file for Spanish could contain:

JSON

```

{
  "Workload_Display_Name": "¡Hola Fabric!",

```

```
"Workload_Full_Display_Name": "¡Hola Fabric!",
"Workload_Description": "Vea cómo crear, personalizar y publicar su primera
carga de trabajo de Fabric en minutos.",
"Workload_Hub_Workload_Slogan": "La forma más rápida de crear, probar y publicar
su primera carga de trabajo de Fabric.",
"HelloWorldItem_DisplayName": "Hola Mundo",
"HelloWorldItem_DisplayName_Plural": "Hola Mundos",
"HelloWorldItem_Description": "Un artículo Hello World para comenzar su
experiencia de desarrollo"
}
```

App translation files

Your workload app uses separate translation files located at `app/assets/locales/[language]/translation.json`. These files contain strings for your app's UI that runs inside the iframe and are loaded dynamically by i18next.

Default behavior

Manifest localization behavior

- The only required language for manifest localization is English.
- If a workload manifest is localized, *all* of the translatable strings must be provided as keys in the `translations.json` file for English.
- Other provided languages aren't required to translate all keys. Any keys that are left untranslated default to the provided English translation. For example, if your workload name should always be in English, you don't need to provide a translation in any other language.
- Fabric supports both localized and unlocalized workloads. If any image files are *directly* in the `Manifest/assets` folder, the assumption is that the workload is unlocalized.
- For a workload to be treated as localized, it must have only two subdirectories in the `Manifest/assets` folder: `images` and `locales`. No other files can be in the `Manifest/assets` folder.

App localization behavior

- Your app's i18next configuration determines which languages are supported for the iframe UI.
- App translations are loaded dynamically when your workload app runs in the iframe.
- The app can detect the Fabric portal language and adjust accordingly, but it operates independently from manifest localization.

Language of the Fabric portal

The `workloadClient` code [provides an API](#) to retrieve the workload settings, which contain the current language of the Fabric portal. For example:

JavaScript

```
export async function callLanguageGet(workloadClient: WorkloadClientAPI):
Promise<string> {
  const settings = await workloadClient.settings.get();
  return settings.currentLanguageLocale;
}
```

App-level localization with i18next

For localizing your workload's user interface, the Fabric Extensibility Toolkit uses [i18next](#). The configuration is already set up in `Workload/app/i18n.js`:

JavaScript

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import HttpApi from 'i18next-http-backend';

i18n
  .use(HttpApi)
  .use(initReactI18next)
  .init({
    fallbackLng: 'en-US',
    supportedLngs: ['en-US', 'es', 'he'],
    debug: false,
    useSuspense: false,
    backend: {
      loadPath: "/assets/locales/{{lng}}/translation.json"
    }
  });

export default i18n;
```

Adding new languages

To add support for additional languages:

1. **Update the i18n configuration:** Add your language code to the `supportedLngs` array in `app/i18n.js`

- 2. **Create manifest translation files:** Add `translations.json` files in `Workload/Manifest/assets/locales/[language-code]/` for Fabric shell strings
- 3. **Create app translation files:** Add `translation.json` files in `Workload/app/assets/locales/[language-code]/` for your app's UI strings

Important: The manifest translations are uploaded to Fabric with your manifest package, while the app translations are deployed with your workload application and loaded dynamically in the iframe.

Using translations in React components

TypeScript

```
import { useTranslation } from 'react-i18next';

function MyComponent() {
  const { t } = useTranslation();

  return (
    <div>
      <h1>{t('Welcome_Title')}</h1>
      <p>{t('Welcome_Description')}</p>
    </div>
  );
}
```

With the provided `currentLanguageLocale` from the workload settings, you can dynamically change the language or use it to customize your workload's behavior based on the user's preferred language.

Key differences summary

Understanding the two localization systems is crucial for proper implementation:

 Expand table

Aspect	Manifest Localization	App Localization
Location	Workload/Manifest/assets/locales/	Workload/app/assets/locales/
File name	translations.json	translation.json
Upload/Deploy	Uploaded to Fabric with manifest package	Deployed with workload app to hosting location

Aspect	Manifest Localization	App Localization
Controls	Fabric shell UI (workload hub, create experience)	Your app's iframe UI
Loading	Loaded by Fabric when displaying manifest info	Loaded dynamically by i18next in iframe
Examples	Workload names, item descriptions, create cards	App buttons, forms, error messages, tooltips

Last updated on 12/04/2025

How-To: Relax the iFrame

This article describes how you can enable additional iFrame attributes for your editor.

Prerequisite

When you enable sandbox relaxation in your manifest, the following happens:

1. **AAD Consent Scopes:** Your workload requests two scopes:
 - **Basic Fabric scope** - The standard scope required for any workload to function
 - **Fabric relaxation scope** - Another scope specifically for sandbox relaxation capabilities
2. **User Consent Flow:** When a user first accesses your workload with sandbox relaxation enabled, they're prompted to consent to both scopes. If they deny, the iFrame won't load.
3. **Additional iFrame Capabilities:** Once consent is granted, your iFrames receive these other sandbox attributes:
 - `allow-downloads` - Enables file downloads from your workload
 - `allow-forms` - Enables form submissions to external services
 - `allow-popups` - Enables opening new windows or tabs

Default sandbox (without relaxation): `allow-same-origin allow-scripts` Relaxed sandbox (with consent): `allow-same-origin allow-scripts allow-downloads allow-forms allow-popups`

Best practices

Only request sandbox relaxation if necessary, since each relaxed permission introduces potential security risks.

Enable in manifest

Add the `enableSandboxRelaxation` setting to your workload manifest:

XML

```
<RemoteServiceConfiguration>
  <CloudServiceConfiguration>
    <Cloud>Public</Cloud>
    <AADFEApp>
```

```
<AppId>00000000-0000-0000-0000-000000000000</AppId>
</AADFEApp>
<EnableSandboxRelaxation>true</EnableSandboxRelaxation>
```

Important

The line `<EnableSandboxRelaxation>true</EnableSandboxRelaxation>` must be included to enable this feature.

Development mode

For local development, you can use sandbox relaxation and bypass consent using the dev override. [Add to devParameters](#) 

TypeScript

```
const devParameters = {
  name: process.env.WORKLOAD_NAME,
  url: "http://127.0.0.1:60006",
  devAADFEAppConfig: {
    appId: process.env.DEV_AAD_CONFIG_FE_APPID,
  },
  devSandboxRelaxation: true
};
```

Note

The line `devSandboxRelaxation: true` enables sandbox relaxation in development mode without requiring user consent.

Last updated on 12/15/2025

Publishing overview

This article provides a comprehensive overview of how to publish a workload in Microsoft Fabric. There are two distinct publishing scenarios based on your organization type and distribution goals:

Publishing Scenarios

- **Internal publishing:** Deploy to your own tenant for organizational use
- **Cross-tenant publishing:** Distribute through Workload Hub to other tenants

ⓘ Note

For Organizations: Follow the overview and general requirements for internal publishing **For Cross-tenant distribution:** Complete all sections including cross-tenant specific requirements

Scenario Details

1. **Organizational scenario:** Publish to your own tenant for internal use
2. **ISV/Software Development Company scenario:** Publish to the Workload Hub for external distribution

General requirements

All workloads must meet fundamental requirements regardless of publishing scenario. These include Microsoft Entra custom domain verification, workload development completion, manifest configuration, and authentication setup.

ⓘ Note

These general requirements apply to both internal and cross-tenant publishing scenarios

 **General Publishing Requirements** - Complete guide to infrastructure, hosting, authentication, and configuration requirements that apply to all workloads.

Organizational scenario: Internal tenant publishing

This scenario is designed for organizations that want to deploy workloads for internal use within their own Fabric tenant.

Steps for organizational publishing

1. **Prepare your manifest package** - See [Manifest overview](#) for detailed instructions.
2. **Upload the package** - Upload your `.nupkg` file through **OneLake catalog > Govern > Configurations > Workloads**. For detailed upload steps, see [Upload a workload](#). Before uploading, ensure the `WorkloadName` field in your [workload manifest](#) is set to the intended Workload Name in the format `Org.[WorkloadName]` (for example, `Org.SalesInsights`). Organizational publishing doesn't require separate registration.
3. **Set up tenant configuration** - Ensure tenant settings and capacity are configured so users can create and use your items.

Outcome: Your workload is available to users in your tenant through the Workload Hub and behaves like a native experience.

Cross-tenant publishing: Workload Hub distribution

This scenario is for Independent Software Vendors (ISVs) and Software Development Companies that want to make their workloads available to all Fabric customers through the Workload Hub.

Prerequisites for Workload Hub distribution

Before you begin publishing to the Workload Hub, ensure you have:

- **Publishing tenant:** Define the tenant that manages your workload lifecycle.
- **Unique Workload Name:** Choose a unique identifier in the form `[Publisher].[Workload]` (for example, `Contoso.SalesInsights`). The Workload portion cannot exceed 32 characters. Set this value in your [workload manifest](#) before uploading.

Steps for Workload Hub distribution

1. **Prepare your manifest package** - See [Manifest overview](#) for detailed instructions. Make sure the `WorkloadName` field uses the `[Publisher].[Workload]` format.

2. **Upload the package** - Upload your `.nupkg` file through **OneLake catalog > Govern > Configurations > Workloads**. For detailed upload steps, see [Upload a workload](#). On the first upload, a **Confirm workload Name** dialog appears. The Workload Name comes from your manifest and you can't change it in this dialog. Confirm the Workload Name to begin registration and select **Confirm**.

 Important

The Workload Name is permanently reserved in your tenant after confirmation and can't be changed.

3. **Test with other tenants** - After the Workload Name is registered, select your workload in **OneLake catalog > Govern > Configurations > Workloads**, and go to the **Publish** tab. Under **Publish to**, select **Selected tenants** and add up to 20 tenant IDs to test your workload with before broader distribution. Select a version and then select **Publish**.

 Note

It might take up to 10 minutes after publishing before the workload is visible in target tenants while the Workload Name propagates across Fabric clusters. Target tenants can only see and use your workload if their admin has enabled the **Users can see and work with additional workloads not validated by Microsoft** [tenant setting](#).

4. **Meet publishing requirements** - Review comprehensive requirements that cover workload functionality, compliance, and user experience standards.
5. **Submit publishing request** - Use the [Publishing Request Form](#)  to move to Preview
6. **Address validation feedback** - Work with Microsoft to address any requirements and validation feedback
7. **Preview listing** - Once accepted, your workload is listed as Preview to all tenants
8. **General Availability** - When ready for GA, submit the publishing request again to remove the preview badge

 **Publishing Requirements** - Complete guide to all requirements, validation tools, and processes

Important

Publishing requirements are only required for ISV/Software Development Company scenarios that distribute workloads through the Workload Hub. Organizations publishing workloads for internal tenant use do not need to meet these requirements.

Outcome: Your workload is available to all Fabric tenants, with controls for preview audiences and general availability.

Troubleshooting

Common issues and solutions during the publishing process:

- **Registration issues:** Confirm publishing tenant and unique Workload Name
- **Upload failures:** Verify manifest schema and required fields
- **iFrame load issues:** Verify front-end endpoint, HTTPS/CSP, and host integration
- **Authentication issues:** Confirm Microsoft Entra registration and scopes

Related content

- [Publishing Requirements](#)
- [Manifest overview](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 09/29/2026

Monetize your workload in Microsoft Fabric

Implement monetization for your Fabric workload using Azure Marketplace SaaS offers or external monetization solutions with user experiences defined in [Fabric templates](#) .

Monetization options

Your workload can monetize through:

- **Azure Marketplace SaaS:** Leverage Microsoft's billing infrastructure
- **External solutions:** Use your own billing and payment systems
- **Hybrid approach:** Combine marketplace and external monetization

Azure Marketplace integration

SaaS offer setup

Create an Azure Marketplace SaaS offer for your workload:

1. **Publisher Center:** Configure your SaaS offer in Partner Center
2. **Landing page:** Implement subscription management UI
3. **Webhook endpoint:** Handle subscription lifecycle events
4. **Billing integration:** Connect to Azure Marketplace billing APIs

Template-defined experience

The [Fabric templates](#) define user experiences for Azure Marketplace integration:

TypeScript

```
// marketplace-integration.ts
export const handleMarketplaceSubscription = (subscriptionData) => {
  // Follow Fabric templates pattern for marketplace UX
  openSubscriptionDialog({
    templatePattern: 'azure-marketplace-saas',
    subscriptionId: subscriptionData.id,
    planId: subscriptionData.planId
  });
};
```

Marketplace APIs

Fabric Extensibility toolkit does not provide additional tools for Azure Marketplace integration. Reference the official APIs:

- [Azure Marketplace SaaS APIs](#)
- [SaaS fulfillment Subscription APIs v2](#)
- [Metering service APIs](#)

External monetization

Custom billing solutions

Implement your own monetization outside Azure Marketplace:

TypeScript

```
// external-billing.ts
export const handleCustomBilling = (userData) => {
  // Follow Fabric templates pattern for external billing UX
  openBillingDialog({
    templatePattern: 'external-monetization',
    userId: userData.id,
    subscriptionTier: userData.tier
  });
};
```

Template support

[Fabric templates](#) provide UX patterns for both marketplace and external monetization approaches.

Implementation considerations

User experience alignment

- **Template consistency:** Follow [Fabric templates](#) for monetization UX
- **Seamless integration:** Embed billing flows naturally in your workload
- **Clear pricing:** Present subscription options transparently

Technical integration

- **Authentication:** Ensure proper user identity management

- **Subscription state:** Track user subscription status across sessions
- **Feature gating:** Control access based on subscription level
- **Usage tracking:** Monitor consumption for metered billing

Best practices

- **Template alignment:** Use [Fabric templates](#) [↗] for consistent UX
- **Marketplace APIs:** Leverage official Azure Marketplace documentation
- **External flexibility:** Design for both marketplace and external billing
- **User transparency:** Clearly communicate pricing and billing terms

Next steps

- [Azure Marketplace SaaS documentation](#)
- [Fabric templates](#) [↗]
- [Publisher monetization guide](#)

Last updated on 12/15/2025

General publishing requirements

Important

These requirements apply to ALL publishing scenarios (both internal and cross-tenant publishing)

General requirements for all Microsoft Fabric Extensibility Toolkit workloads:

- **Microsoft Entra custom domain verification:** Required for Entra App verification and proper resource ID configuration
- **Complete workload development:** Your workload must be fully developed and tested
- **Manifest configuration:** Prepare your workload manifest according to Fabric specifications
- **Authentication setup:** Configure Microsoft Entra application registration for your workload
- **Infrastructure and hosting:** Meet hosting, security, and performance requirements

Microsoft Entra Requirements

Custom Domain Verification

You must have a verified custom domain in your Entra tenant for Entra App verification and resource ID configuration.

- **Requirement:** Verified custom domain (e.g., `contoso.com`)
- **Frontend domain:** Must be a subdomain of your verified Entra domain
- **Restrictions:** No `*.onmicrosoft` subdomains

Resource ID Format

HTTP

`https://<verified-domain>/<frontend>/<backend>/<workload-id>/<optional>`

Example: `https://datafactory.contoso.com/feserver/beserver/Fabric.WorkloadSample/123`

Application Registration

- **Type:** Web application

- **Tenant support:** Multitenant (required for cross-tenant publishing)
- **Redirect URI:** `{frontend-url}/close`
- **Application ID URI:** Must match verified domain
- **Permissions:** Microsoft Fabric APIs

Domain Configuration

Important

The frontend domain must **always** be a subdomain of your verified Entra domain.

Frontend and backend URLs must be subdomains of the `resourceId` value:

- **Frontend domain requirement:** Must be a subdomain of your verified Entra domain
- **Domain relationship:** Maximum of one extra segment beyond the verified domain
- **Reply URL domain:** Must match the frontend host domain
- **HTTPS requirement:** All endpoints must use HTTPS

Example Configuration:

- Resource ID:
`https://datafactory.contoso.com/feserver/beserver/Fabric.WorkloadSample/123`
- Frontend: `https://feserver.datafactory.contoso.com`
- Backend: `https://beserver.datafactory.contoso.com/workload`
- Redirect URI: `https://feserver.datafactory.contoso.com/close`

Workload Development Requirements

Complete Development and Testing

- **Functional completeness:** All planned features implemented and working
- **Integration testing:** Tested within Fabric environment using the dev gateway
- **Error handling:** Proper error handling and user feedback mechanisms

Manifest Configuration

Endpoint values in your manifest are typically set using placeholders such as `{{FRONTEND_URL}}`, which are replaced with values from your `.env` file during the build process. Update the `.env` file to set the correct URLs for your deployment environment.

Configure the workload endpoints in your manifest:

XML

```
<CloudServiceConfiguration>
  <Cloud>Public</Cloud>
  <Endpoints>
    <ServiceEndpoint>
      <Name>Frontend</Name>
      <Url>https://feserver.datafactory.contoso.com</Url>
    </ServiceEndpoint>
  </Endpoints>
</CloudServiceConfiguration>
```

Under the `AADApp` section in `WorkloadManifest.xml`:

- **AppId**: Your Microsoft Entra application ID
- **redirectUri**: Frontend URL with `/close` suffix
- **ResourceId**: The properly formatted resource ID URI

Infrastructure Requirements

Hosting Requirements

- **HTTPS**: All endpoints must use HTTPS with valid SSL certificates
- **Availability**: Minimum 99.9% uptime SLA
- **Performance**: Page load times under 3 seconds
- **CORS**: Configure CORS to allow Fabric domains (`*.powerbi.com`, `*.fabric.microsoft.com`)

Content Security Policy (CSP)

Configure CSP headers to ensure compatibility with Fabric:

- **Frame ancestors**: Allow Fabric domains to embed your workload
- **Script and style sources**: Configure safe sources for your application

Configuration

Frontend Configuration:

- **WORKLOAD_NAME**: Your workload identifier
- **WORKLOAD_VERSION**: The version of your workload

- **FRONTEND_URL**: The frontend URL for your workload (typically set as `{{FRONTEND_URL}}` in the manifest and replaced from the .env file)
- **FRONTEND_APPID**: The frontend application ID (typically set as `{{FRONTEND_APPID}}` in the manifest and replaced from the .env file)

Backend Configuration (if applicable): Backend configurations are currently not supported.

Security Requirements

Authentication and Authorization

- **OAuth 2.0 compliance**: Use standard OAuth 2.0 flows
- **Token validation**: Proper validation of access tokens
- **Permission validation**: Verify user permissions before allowing access

Data Protection

- **Data encryption**: Encrypt data in transit and at rest
- **Input validation**: Validate and sanitize all user inputs
- **Security headers**: Implement appropriate security headers

Next steps

- [Publishing Overview](#)
- [Authentication Guidelines](#)
- [Tutorial: Host Workload in Azure](#)

Last updated on 01/08/2026

Publishing requirements for Microsoft Fabric workloads

Important

This section applies ONLY to cross-tenant publishing (Workload Hub distribution). Organizations publishing workloads for internal tenant use do not need to meet these requirements.

This article provides a comprehensive overview of all requirements for publishing workloads to Microsoft Fabric, including workload registration, item requirements, attestation documentation, and validation tools.

Prerequisites

Before you can publish your workload to the Workload Hub, you must meet these fundamental requirements:

Workload Name Registration

Every workload requires a unique **Workload Name** that follows the format `[Publisher].[Workload]` (for example, `Contoso.SalesInsights`).

- **Character Limit:** The Workload portion cannot exceed 32 characters
- **Automatic registration:** The workload name is automatically registered when you confirm it during the first upload in **OneLake catalog > Govern > Configurations > Workloads**.
- **Fixed Value:** Once registered, the Workload name cannot be changed
- **Manifest Configuration:** Set the Workload name in your [workload manifest](#)
- **First Upload Confirmation:** When you upload a workload for the first time, a **Confirm workload Name** dialog appears. The workload name is permanently reserved in your tenant after confirmation.

For detailed information about workload name requirements and the registration process, see [Publish your workload](#).

Microsoft Entra App Verification

All workloads must use a **verified Microsoft Entra application** for authentication and API access.

- **Verification Required:** Your Entra application must complete the Microsoft verification process
- **Publisher Verification:** Applications must have verified publisher status
- **Compliance Standards:** Verified apps meet Microsoft's security and compliance requirements
- **Publishing Prerequisite:** App verification must be completed before workload publishing

For detailed information about the verification process and requirements, see [Microsoft Entra verified publisher documentation](#).

Comprehensive Requirements

Publishing to Microsoft Fabric involves meeting requirements across three main categories:

General Requirements

General requirements apply to all workloads regardless of publishing scenario and cover fundamental infrastructure, authentication, and configuration needs.

[General Publishing Requirements](#)

Key areas include:

- Microsoft Entra custom domain verification and application registration
- Resource ID format and domain configuration requirements
- Infrastructure hosting, HTTPS, and security standards
- Manifest configuration and endpoint setup
- Basic authentication and authorization patterns

Workload-Level Requirements

Workload requirements cover the overall workload functionality, infrastructure, and compliance standards specific to Workload Hub distribution.

[Publishing requirements for workloads](#)

Key areas include:

- Infrastructure and hosting requirements
- Security and authentication standards

- Performance and reliability criteria
- Documentation and support requirements
- Compliance and certification standards

Item-Level Requirements

Item requirements focus on the specific user-facing components and experiences within your workload.

[Publishing requirements for items](#)

Key areas include:

- User interface and experience standards
- Data handling and processing requirements
- Integration with Fabric platform features
- Accessibility and usability standards
- Item lifecycle management

Attestation Documentation

All workloads must provide **attestation documentation** that certifies compliance with Microsoft Fabric standards.

Attestation Requirements

- **Template Compliance:** Follow the provided attestation document template
- **Public Accessibility:** Host the document at a publicly accessible HTTPS URL
- **Manifest Integration:** Include the attestation document URL in your workload manifest
- **Content Validation:** Ensure all information is accurate and up-to-date
- **Regular Updates:** Keep attestation documentation current with any changes

Attestation Document Template

The attestation document template provides the required structure and content areas for your compliance certification.

 [Publishing Vendor Attestation Template](#) - Use this template to create your attestation document

Required in Manifest: Your workload manifest must include the `AttestationUrl` property pointing to your published attestation document.

Validation Tools

Microsoft Fabric provides validation tools to help partners ensure their workloads meet publishing requirements before submission.

Publishing Validation Tool

Use the official validation tool to check your workload against publishing requirements:

 [How to validate your workload for publishing](#)

The validation tool provides:

- **Automated Checks:** Scans your workload for common compliance issues
- **Requirement Coverage:** Validates against both workload and item requirements
- **Detailed Reports:** Provides specific feedback on areas that need attention
- **Pre-Submission Validation:** Catch issues before submitting for review

Validation Process

1. **Complete Development:** Ensure your workload is feature-complete
2. **Run Validation Tool:** Use the publishing validation tool to identify issues
3. **Address Findings:** Fix any compliance gaps identified by the tool
4. **Re-validate:** Run the tool again to confirm all issues are resolved
5. **Submit for Review:** Proceed with the publishing request form

Publishing Process Overview

Once you meet all requirements, follow this process:

1. **Workload name registration:** Register your workload name during the first upload in **OneLake catalog > Govern > Configurations > Workloads**.
2. **Requirements Compliance:** Meet all workload and item requirements
3. **Attestation Documentation:** Create and publish your attestation document
4. **Validation:** Use validation tools to verify compliance
5. **Manifest Preparation:** Include Workload name and attestation URL in manifest
6. **Publishing Request:** Submit the [Publishing Request Form](#) 

7. **Review Process:** Microsoft reviews your submission against requirements
8. **Approval:** Once approved, your workload is published to the Workload Hub

Support and Resources

Getting Help

- **Documentation:** Review all linked requirement documents thoroughly
- **Validation Tools:** Use provided tools to identify and fix issues early
- **Partner Support:** Contact your Microsoft partner representative for guidance
- **Publishing Form:** Use the official publishing request form for submissions

Related Content

- [Publishing Overview](#) - High-level publishing process
- [Publish your workload](#) - Detailed publishing workflow
- [Manifest overview](#) - Workload manifest configuration
- [Publishing requirements for workloads](#)
- [Publishing requirements for items](#)
- [How to validate your workload for publishing](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 09/29/2026

Microsoft Fabric Workload Publishing Requirements

 Summarize this article for me

This document outlines all publishing requirements for Microsoft Fabric workloads.

1. Business Requirements

Building applications in Fabric is more like creating documents in office than provisioning resources in Azure. While we don't expect every user to be able to take advantage of every workload in Fabric, each of the workloads must appeal to the broadest set of users.

Fabric is designed to support the largest Tier 1 projects and data sizes. The Fabric platform provides a common set of services that the workload must not replace with their own proprietary services. Consistent use of the platform services across all the workloads ensures a uniform experience and functionality throughout the system.

This document provides a comprehensive overview of all the different components that you need to take into account to publish a workload.

1.1 - Value To Customers

Workload must clearly articulate its value proposition, benefits, and use cases to help customers understand how it solves their business problems

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

1.2.1 - Term of use link in Workload page is correct

Workload must provide a valid, accessible HTTPS link to terms of use in the manifest

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Governance overview and guidance](#)
- [Product Manifest](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field `product.productDetail.supportLink.terms.url`.

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.terms.url` must exist and contain a valid URL
- URL redirects are allowed and will be followed

1.2.2 - Privacy Policy link in workload hub is correct

Workload must provide a valid, accessible HTTPS link to privacy policy in the manifest

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Governance overview and guidance](#)
- [Product Manifest](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field `product.productDetail.supportLink.privacy.url`.

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.privacy.url` must exist and contain a valid URL
- URL redirects are allowed and will be followed

1.3.1 - Marketplace Offer has been published

Workload must have a published marketplace offer with a valid, accessible license URL pointing to Azure Marketplace or AppSource

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Creating a SaaS offer](#)
- [Commercial marketplace certification policies](#)
- [Publishing Overview](#)
- [Publish Workload Tutorial](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field `product.productDetail.supportLink.license.url`.

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.license.url` must exist and contain a valid URL
- URL domain must be one of: <https://azuremarketplace.microsoft.com/> , <https://appsource.microsoft.com/> , https://marketplace.microsoft.com
- URL redirects are allowed and will be followed

1.3.2 - Marketplace Offer Link in workload page is correct

The offer link needs to point to Azure Marketplace or AppSource.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Documentation](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field `product.productDetail.supportLink.license.url`.

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.license.url` must exist and contain a valid URL

- URL domain must be one of: <https://azuremarketplace.microsoft.com> , <https://appsource.microsoft.com/> , <https://marketplace.microsoft.com> 
- URL redirects are allowed and will be followed

1.3.3 - Workload publisher name is clear

Workload must display a clear, professional publisher name that helps customers identify the vendor

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Documentation](#)

1.3.4 - Workload publisher name is aligned with marketplace

Publisher name must be consistent between the workload manifest and Azure Marketplace listing to avoid customer confusion

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Documentation](#)

1.4.1 - Attestation Link in workload page is correct

Workload must provide a valid, accessible HTTPS link to certification attestation documentation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Vendor Attestation Document Template](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field

```
product.productDetail.supportLink.certification.url.
```

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.certification.url` must exist and contain a valid URL
- URL redirects are allowed and will be followed

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
 - Document must be downloadable and accessible
 - Document must be in a supported format (Website, PDF)
 - Document must follow the template format
 - Document content must be extractable and readable
-

1.4.2 - Getting started material exists

Workload must provide learning materials or getting started guides to help new users onboard successfully

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)

1.4.3 - At a glance section includes image or a video

Workload must include visual content (images or videos) in the 'At a glance' section to demonstrate functionality

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to configure at a glance section](#)
- [Product Manifest](#)
- [Video guidelines](#)

1.4.4 - Learning Material must land on static pages

Learning material links must direct users to stable, static documentation pages rather than dynamic or temporary content

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)

1.4.5 - Documentation link in Workload page is correct

Workload must provide a valid, accessible HTTPS link to comprehensive product documentation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field

```
product.productDetail.supportLink.documentation.url.
```

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed
- Manifest field `product.productDetail.supportLink.documentation.url` must exist and contain a valid URL
- URL redirects are allowed and will be followed

2. Technical Requirements

Technical implementation requirements including authentication, APIs, and integration

2.1.1 - Verified Microsoft Entra App ID

Workload must register a valid Microsoft Entra application with proper GUID format and configuration

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Authentication Overview](#)
- [Documentation](#)
- [App Registration Guide](#)
- [Verified Microsoft Entra App ID](#)

2.1.2 - Dependency on Fabric.Extend scope

The workload must take a static dependency on Fabric.Extend scope and get user consent for it. Workload Development Kit infrastructure helps to implement the consent flow, but the responsibility is with the workload owner to take dependency on Fabric.Extend scope and ensure consent is granted. Fabric reserves the rights to block the integration with a workload for which the consent isn't granted.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Entra Scopes](#)

2.1.3 - Microsoft Entra Redirect URL must be correct

The redirect must land in Fabric for The Extensibility Authentication redirect URL must return a minimal HTML page that only executes windows.close() JavaScript without other content

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Authentication Overview](#)
- [How to Acquire Entra Token](#)

2.1.4 - Microsoft Entra Scopes are kept to a limit

Workload should request only essential Microsoft Entra scopes to minimize permissions and follow principle of least privilege

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Entra Scopes](#)
- [Fabric Scopes](#)
- [Authentication Overview](#)
- [Authentication Guidelines](#)

2.2 - OneLake

OneLake is where data is stored in Fabric. Workloads need to integrate with it to store data in the standard formats supported by the platform so that other services can take advantage of it.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [OneLake Overview](#)
- [How to Store Data in OneLake](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.3 - Microsoft Entra Conditional Access

Workload must support Microsoft Entra Conditional Access policies for enterprise security requirements

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.4 - Customer Facing Monitoring & Diagnostic

Workload must provide monitoring and diagnostic capabilities for customers to troubleshoot and track usage

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.5 - B2B

Fabric sharing strategy is focused on allowing customers to collaborate with their business partners, customers, vendors, subsidiaries, etc. It also means users from other tenants can potentially be granted access to items partners are creating.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.6 - Business Continuity and disaster recovery

Fabric commits to a seamless service availability to customers. We recognize the importance of planning for unforeseen events in a world of uncertainties. We ask partners providing workloads to customers within Fabric to define Business Continuity and Disaster Recovery (BCDR) plans designed to tackle unplanned disasters and recovery steps.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Vendor Attestation Document Template](#)
- [Reliability in Microsoft Fabric](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.7 - Performance

Performance is an important requirement for Fabric customers. We recommend our partners to think about performance in the context of their workload and take measures to test and track performance of their Items.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.8 - Presence

To ensure that you can meet customer expectations independent of their home or capacity region, partners need to think about how they want to align with Fabric regions and clouds. Availability in certain regions also impacts your Data Residency commitments.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

2.9 - Accessibility

The workload and all Item types the partner provides as part of it need to comply with the Fabric UX guidelines.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Fabric UX System](#) 

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format

- Document content must be extractable and readable

2.10 - World Readiness

English has to be the default language. Content can be localized if you decide to do it. In this case add the supported languages to your attestation page.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

3. Design/UX Requirements

Get familiar with the design requirements for your Fabric workload, review the Fabric UX System (<https://aka.ms/fabricux> ). The Fabric UX System provides an overview of the patterns and components that are available in the Fabric UX to accelerate your work. It also includes Fabric UX React and Angular wrapper libraries that integrate with Fluent web components.

Fabric users expect a fast loading UX. Independent of the technical implementation and regional hosting, your workload should load fast within all Fabric regions.

3.1.1 - Workload has a clear name

Workload must have a clear, descriptive name that helps users understand its purpose at a glance. You can't use generic terms (for example AI, Agent, ..) without prefix or suffix to differentiate from others on the platform.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
-

3.1.2 - Workload icons, and images are clear

Workload must provide custom, professional icons, and images that are distinct from default placeholders and represent the brand

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
 - [Fabric UX System](#) 
 - [Microsoft Fabric visuals kit](#) 
-

3.1.2.1 - Product icon is clear

Product icon must meet dimension requirements (240x240) and be visually clear and recognizable

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

Guidelines:

IMAGE:

Make sure the image in the manifest is aligned with the requirements

Requirements:

- Image file must exist and be accessible
- Image file must not be corrupted or invalid
- Image must be of the following types: image/png, image/jpeg, image/svg+xml
- Image must be aligned with one of the formats: 16x16, 20x20, 24x24, 32x32, 40x40, 48x48, 64x64
- Image should be optimized for web delivery

3.1.2.2 - Favicon icon is clear

Favicon must be provided in the manifest with appropriate dimensions and clarity for browser tab display

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

Guidelines:

IMAGE:

Make sure the image in the manifest is aligned with the requirements

Requirements:

- Image file must exist and be accessible
- Image file must not be corrupted or invalid
- Image must be of the following types: image/png, image/jpeg, image/svg+xml
- Image must be aligned with one of the formats: 16x16, 20x20, 24x24, 32x32, 40x40, 48x48, 64x64
- Image should be optimized for web delivery

3.1.2.3 - Custom actions Icon validation

Custom action icons must meet dimension requirements and be clear, recognizable representations of their functions

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

3.1.2.4 - Create experience Icon validation

Create experience icons must meet dimension requirements and provide clear visual representation for item creation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

3.1.2.5 - Create experience Icon small validation

Small create experience icons must meet dimension requirements and remain clear at reduced sizes

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

3.1.2.6 - Tabs Icon validation

Tab icons must meet dimension requirements and be visually distinct to aid navigation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

3.1.2.7 - Tabs Icon active validation

Active tab icons must meet dimension requirements and provide clear visual indication of selected state

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

3.1.3 - Workload has a clear sub title

Workload must provide a clear, concise slogan/subtitle that complements the name and communicates key value

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 

3.1.4 - Workload has a clear description

Workload must provide a comprehensive description that explains features, capabilities, and use cases

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)

3.1.4.1 - Workload has a clear description in the Workload Hub Card

Workload Hub card must include a concise description optimized for the card display format

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)

3.1.5 - Workload info page has banner at size 1920X240

Workload must provide a banner image with exact dimensions of 1920x240 pixels for the info page header

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Product Manifest](#)
- [Fabric UX System](#) 

Guidelines:

IMAGE:

Make sure the image in the manifest is aligned with the requirements

Requirements:

- Image file must exist and be accessible
- Image file must not be corrupted or invalid
- Image must be of the following types: image/png, image/jpeg, image/svg+xml
- Image must be aligned with one of the formats: 16x16, 20x20, 24x24, 32x32, 40x40, 48x48, 64x64
- Image should be optimized for web delivery

3.1.6 - Gallery videos aren't allowed to show non product related ads

Gallery videos must focus exclusively on product features without displaying third-party advertisements or unrelated content

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Video guidelines](#)

4. Security & Compliance Requirements

Security, privacy, and compliance requirements for data protection and regulatory adherence

4.1 - Security general

Microsoft customers entrust Fabric with their most sensitive data. As partners implementing workloads can have access to this data, they also have a responsibility to protect that data. We request workloads to go through a security assessment, a security review, and attest that they completed it and remediated any issues discovered in the process.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest

- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.1.1 - Publisher must obtain user Microsoft Entra ID token from the Fabric host before calling Fabric JS functions

Workload must obtain Microsoft Entra ID tokens exclusively through the Fabric host before invoking any Fabric JavaScript functions

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Authentication Guidelines](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.1.2 - Publisher must only obtain any Microsoft Entra token using the JavaScript APIs provided by the Fabric Workload

Client SDK

Workload must use only the official Fabric Workload Client SDK JavaScript APIs for obtaining Microsoft Entra tokens, no custom implementations

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Authentication Guidelines](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.1.3 - Publisher must not use third party cookies as part of their solution

Workload must not rely on third-party cookies; only essential HTTP-only cookies after authentication are permitted

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Cookie compliance](#) 

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.1.4 - Microsoft Entra App Name and publisher need to be aligned with the Workload

Microsoft Entra application name and publisher must align with the workload name to ensure clear identity and avoid user confusion

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Authentication Guidelines](#)
 - [How to register an Entra app](#)
-

4.2 - Privacy

Microsoft customers entrust Fabric with their most sensitive data. As such, partners that build workloads also have a responsibility to protect that data when they access it. To that end we request that every workload goes through a privacy assessment and a privacy review.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.2.1 - Publisher using only essential HTTP-only cookies after positively authenticating the user

Workload owners can only use essential HTTP-only cookies. Workload can use them only after positively authenticating the user. Only use same-origin cookies

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#) 

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.2.2 - Third-party cookie policy

Partner workloads aren't to use, write, or rely on third-party cookies

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.2.3 - Microsoft Entra token policy

Publisher must only obtain any Microsoft Entra token using the JavaScript APIs provided by the Fabric Workload Client SDK

Stage Requirements:

 **Expand table**

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
 - Document must be downloadable and accessible
 - Document must be in a supported format (Website, PDF)
 - Document must follow the template format
 - Document content must be extractable and readable
-

4.3 - Data Residency

Fabric is making an Enterprise Promise around data not leaving the geography of the tenant for stored data and data in transit. As a workload you're showing up in Fabric directly and users need to be aware what your commitments to Data Residency are. In the attestation, you need to define what our commitments are to the Data Residency of customer data.

Stage Requirements:

[Expand table](#)

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#) 

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

4.4 - Compliance attestation

Within the Attestation Document you can show customers how your app handles security, data, and compliance. In this self-assessment, the Workload developer describes the Workload's security attributes and data-handling practices. The publisher attestation document should be hosted on the partner website. If applicable to your customers, align with more Fabric certifications.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

5. Support Requirements

Support and documentation requirements for customer assistance and troubleshooting

5.1 - Livesite

Partner workloads are becoming an integral part of Fabric therefore our support teams need to be aware how you want to be contacted in case customers are reaching out to us directly. Partners need to provide the contact details as part of the publishing process to us.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

5.2 - Supportability

Partners are responsible to define and document their support parameters (Service level agreement, contact methods, etc.). This information needs to be linked from the Workload page and should always be accessible to customers. In addition, the Marketplace criteria need to be taken into account for the listing of the SaaS offer.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

5.2.1 - Help Link in Workload Page is correct

Workload must provide a valid, accessible HTTPS link to help resources and support documentation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)

Guidelines:

LINK:

This validation extracts a URL from the manifest and verifies it is reachable via HTTP request. The URL is extracted from the manifest field `product.productDetail.supportLink.help.url`.

Requirements:

- URL must use HTTPS protocol for security
- URL must be reachable and return HTTP status 200-399
- URL must respond within reasonable timeout period
- URL format must be valid and well-formed

- Manifest field `product.productDetail.supportLink.help.url` must exist and contain a valid URL
- URL redirects are allowed and will be followed

5.2.2 - Support contact is listed in the product details page

Product details page must clearly display support contact information for customer assistance

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

5.3 - Service Health & Availability

Partners need to host a website that shows their service health and availability to customers. This information can be included in the Supportability page.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6. Fabric Features

Integration with Microsoft Fabric platform features and capabilities

6.1 - Application lifecycle management (ALM)

Microsoft Fabric's lifecycle management tools enable efficient product development, continuous updates, fast releases, and ongoing feature enhancements.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6.1.1 - CICD section is present in the manifest

Workload manifest should include a CICD section to declare support for deployment pipelines and git integration, enabling seamless application lifecycle management within Microsoft Fabric.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Documentation](#)
- [Product Manifest](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6.2 - Private Links

In Fabric, you can configure and use an endpoint that allows your organization to access Fabric privately.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
 - Document must be downloadable and accessible
 - Document must be in a supported format (Website, PDF)
 - Document must follow the template format
 - Document content must be extractable and readable
-

6.3 - Data Hub

The OneLake data hub makes it easy to find, explore, and use the Fabric data items in your organization that you have access to. It provides information about the items and entry points for working with them. If you're implementing a Data Item, show up in the Data Hub as well.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to integrate with OneLake catalog](#)
- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6.4 - Data lineage

In modern business intelligence (BI) projects, understanding the flow of data from the data source to its destination can be a challenge. The challenge is even bigger if you built advanced analytical projects spanning multiple data sources, data items, and dependencies. Questions like "What happens if I change this data?" or "Why isn't this report up to date?" can be hard to answer.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6.5 - Sensitivity Labels

Sensitivity labels from Microsoft Purview Information Protection on items can guard your sensitive content against unauthorized data access and leakage. They're a key component in helping your organization meet its governance and compliance requirements. Labeling your data correctly with sensitivity labels ensures that only authorized people can access your data.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
- Document must be downloadable and accessible
- Document must be in a supported format (Website, PDF)
- Document must follow the template format
- Document content must be extractable and readable

6.5.1 - Sensitivity Labels - Exports

For partners that are using Export functionality within their Item they need to follow the guidelines.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Vendor Attestation Document Template](#)
- [Documentation](#)

Guidelines:

ATTESTATION:

Make sure the information in the Attestation document is valid, can be reached and follows the template.

Requirements:

- Attestation document URL must be present in manifest
 - Document must be downloadable and accessible
 - Document must be in a supported format (Website, PDF)
 - Document must follow the template format
 - Document content must be extractable and readable
-
-

Additional Information

Version History

This document is automatically generated.

Generated by Microsoft Fabric Workload Validation System
© 2026 Microsoft Corporation. All rights reserved.

Last updated on 02/24/2026

Microsoft Fabric Item Publishing Requirements

 Summarize this article for me

This document outlines all publishing requirements for Microsoft Fabric items (workload-specific artifacts).

General Requirements

General requirements for item naming, icons, and basic metadata

1.1 - Item has a clear name

Item must have a clear, descriptive display name that helps users understand its purpose and type. You can't use generic names (for example, AI, Agent,..) without any pre or suffix indicating the specialty, your company, or product.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Manifest](#)
- [How to Create Item](#)
- [Fabric UX System](#) 

1.2 - Item has a clear icon

Item must provide a clear, recognizable icon that visually represents the item type across the Fabric interface

Stage Requirements:

Preview	General Availability
 Required	 Required

References:

- [Item Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

Guidelines:

UX:

Filetype icons are comprised of a container and an interior Fluent system icon. There are 3 generic shapes: vertical, horizontal, and square. Some interior icons work better in specific shapes, so explore which shape is best.

This grid template displays the sizes we use: 20 / 24 / 32 / 40 / 48 / 64 For the workspace, workload L2 page and cards we use the 24x24px size.

The interior icon should be centered within the container. If an icon is composed of only/mostly strokes, and it needs more weight, you can adjust to 1.5px weight. General reference for interior icon sizing inside container:

- 12px in 20px container
- 14px in 24px container
- 20px in 32px container
- 24px in 40px container
- 28px in 48px container
- 36px in 64px container

Make sure interior icons are as centered as possible. All icons are drawn with 1px stroke unless otherwise stated. Item icon must follow the icon creation guidelines for consistency across the Fabric platform.

1.2.1 - Item has a clear active icon

Item must provide a distinct active state icon to indicate when the item is selected or in focus.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Manifest](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Fabric UX System](#) 

1.3 - Item has a clear title in create menu

Item must display a clear title in the create menu that matches the item's display name for consistency

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Manifest](#)
- [How to Create Item](#)

1.4 - Item has a clear subtitle in create menu

Item must provide a concise, informative subtitle in the create menu that describes the item's purpose or key features

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Manifest](#)
- [Fabric UX System](#) 

1.5 - Item can be favorited in the "new" menu

Item must support favoriting functionality in the 'new' menu to allow users to quickly access frequently used items

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Create Item](#)
- [Item Manifest](#)
- [Fabric UX System](#) 

Item Creation Flow

Requirements for the item creation experience and workspace integration

2.1 - Item Creation Experience

Item creation flow must follow Fabric design guidelines providing a consistent, intuitive experience for users

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Create Item](#)
- [Item Manifest](#)
- [Item Concept](#)
- [Fabric UX System](#) 

2.2 - Item can be created in Creation Hub

Item must be available for creation through the Fabric Creation Hub for centralized item creation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Create Item](#)
- [Item Manifest](#)
- [Item Concept](#)
- [Fabric UX System](#) 

2.3 - Item can be created in the Workspace menu

Item must be available for creation through the workspace '+New' menu for contextual item creation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Create Item](#)
- [Item Manifest](#)
- [Item Concept](#)
- [Fabric UX System](#) 

2.4 - Platform create experience is configured

Item creation must use Fabric's default naming experience (createItemDialogConfig in Product.json). Custom item creation flows are deprecated, if used they should avoid generating default names to prevent workspace clutter

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Create Item](#)
- [Item Manifest](#)
- [Item Concept](#)
- [Fabric UX System](#) 

2.5 - Item is persisted in the Workspace

Created items must be immediately persisted and visible in the workspace after creation completes

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [How to store item definition](#)

2.6 - Item is in the multitasking menu and supports the behavior

Items must appear in the multitasking menu and support switching between multiple open items seamlessly

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Item Manifest](#)
- [Fabric UX System](#) 

2.7 - Item creation works under *.powerbi.com and *.fabric.microsoft.com

Item creation must function correctly on both powerbi.com and fabric.microsoft.com domains for cross-platform support

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

Ribbon & Toolbar

Requirements for the item editor ribbon and toolbar following design guidelines

3.1 - Item editor ribbon exists

Item editor must implement with one ribbon interface for consistent user experience across Fabric. This ribbon should include all actions and controls necessary for item editing and it must be sticky on top above the canvas. The ribbon tabs can't be used to switch the view.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Fabric Templates](#) 

Guidelines:

UX:

Item editor must implement with one ribbon interface for consistent user experience across Fabric. This ribbon should include all actions and controls necessary for item editing and it must be sticky on top above the canvas. The ribbon tabs can't be used to switch the view.

3.2 - Item Editor ribbon actions are aligned with style guide

Item Editor ribbon actions must use the fabric button component in subtle state. Any other component should be max height 32 px

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Item Editor ribbon actions must use the fabric button component in subtle state. Any other component should be max height 32 px.

3.2.10 - Item Editor ribbon Tooltips - follows style guide

Ribbon tooltips must follow Fabric guidelines for content, timing, and placement to aid user understanding

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

3.3 - Item editor ribbon has a "Home" tab

Item Editor ribbon must include Home tab as the primary editor page. It also must be called "Home" and be the first tab in the ribbon. This is the first place users land when opening the item editor. Tabs are only switching the ribbon control and can't be used to switch the canvas.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Item Editor ribbon must include Home tab as the primary editor page, it's also must called "home" and be the first tab in the ribbon. this is the first palace users will land when oppening the item editor.

3.4 - Item Editor ribbon implements a save button / AutoSave

Item editor should provide either explicit save functionality or AutoSave to prevent data loss

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Item editor should implement either a save button or autosave functionality for user data protection. button is icon only.

3.6 - Item Editor ribbon Color - follows style guide

Ribbon colors must follow Fabric style guidelines using approved color tokens and theme support. Example: Table changes from gray to red when it becomes Table Dismiss.

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

3.7 - Item Editor ribbon Typography - follows style guide

Ribbon typography must use Fabric-approved fonts, sizes, and weights for consistent text display

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

Guidelines:

UX:

3.8 - Item Editor ribbon Elevation - follows style guide

Ribbon elevation (shadow/depth) must follow Fabric guidelines to maintain proper visual layering

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

3.9 - Item Editor ribbon Border radius - follows style guide

Ribbon border radius must follow Fabric guidelines for consistent rounded corner styling

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

3.10 - Item Editor ribbon Spacing and layouts - follows style guide

Ribbon spacing and layout must use Fabric-approved padding, margins, and grid systems for visual consistency

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Ribbon spacing and layout must use Fabric-approved padding, margins, and grid systems for visual consistency. Ribbon spacing and layouts of the buttons: if placing buttons next to each other than use 0px spacing. When adding border line between buttons use 4px spacing on the left and right sides. If placing different components e.g. dropdown use 4px spacing between them.

Canvas & Editor

Requirements for the item canvas and editor UI components

4.1 - Canvas border - follows style guide

Canvas border styling must follow Fabric guidelines for color, width, and border radius

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Canvas border styling must follow Fabric guidelines for color, width, and border radius. Canvas must implement proper border styling according to Fabric design standards.

4.2 - Colors - follows style guide

Canvas colors should follow Fabric style guidelines using semantic color tokens and theme support

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 
- [Fabric Templates](#) 

Guidelines:

UX:

Canvas colors should follow Fabric style guidelines using semantic color tokens and theme support. Canvas colors must follow Fabric design system color guidelines for consistency.

4.3 - Empty states - follows style guide

If Empty state exists (recommended), it should follow Fabric guidelines providing helpful messaging and clear actions for users

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 

- [Documentation](#) 

Guidelines:

UX:

If Empty state exists (recommended), it should follow Fabric guidelines providing helpful messaging and clear actions for users. Canvas must properly implement empty states to guide users when no content exists.

4.4 - Left drawer - follows style guide

If left drawer panel exists it should follow Fabric guidelines for width, animation, and content organization

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

If left drawer panel exists it should follow Fabric guidelines for width, animation, and content organization. Canvas may implement a left drawer for additional navigation or tools.

4.5 - Bottom Drawer - follows style guide

If bottom drawer panel exists it should follow Fabric guidelines for height, animation, and content organization

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

If bottom drawer panel exists it should follow Fabric guidelines for height, animation, and content organization. Canvas may implement a bottom drawer for additional functionality.

4.6 - Tabs - follows style guide

Tab components should follow Fabric guidelines for styling, interaction, and active state indication

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Tab components should follow Fabric guidelines for styling, interaction, and active state indication. Canvas may implement tabs for organizing content within the editor.

4.7 - Switch controls - follows style guide

If Switch/toggle controls exist they should follow Fabric guidelines for size, color, and interaction behavior

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

If Switch/toggle controls exist they should follow Fabric guidelines for size, color, and interaction behavior. Canvas may implement switch controls for toggling functionality.

4.8 - Canvas Center control - follows style guide

Center drawer panel should follow Fabric guidelines for width, animation, and content organization

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Navigation

Requirements for navigation between item views and levels

5.1 - Multi-Level navigation

If multi level navigation between canvas levels (L1 to L2) is used it must follow Fabric guidelines for transitions, breadcrumbs, and back navigation

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Popup Dialogs

6.1 - Popup dialog - follows style guide

If popup dialogs are used they must follow Fabric style guidelines for modal behavior, backdrop, and dismissal patterns

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

6.1.5 - Border radius - follows style guide

Dialog border radius must follow Fabric guidelines for consistent rounded corner styling across modals

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)

- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

6.2 - Color - follows style guide

Dialog colors must follow Fabric guidelines using semantic color tokens for backgrounds, text, and borders

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

6.3 - Typography - follows style guide

Dialog typography must use Fabric-approved fonts, sizes, and hierarchy for headers, body text, and actions

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

6.4 - Elevation - follows style guide

Dialog elevation must follow Fabric guidelines to ensure proper visual layering above page content

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

6.6 - Spacing and layouts - follows style guide

Dialog spacing and layout must use Fabric-approved padding, margins, and content organization patterns

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [Item Concept](#)
- [Fabric UX System](#) 
- [Microsoft Fabric visuals kit](#) 
- [Documentation](#) 

Guidelines:

UX:

Other UX Requirements

Other requirements including monitoring, settings, and context menus

7.1 - Monitoring Hub integration

Item should optionally integrate with Fabric Monitoring Hub to expose job execution and performance data

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Item Concept](#)

7.2 - Actions in Workspace need to work

If Actions in the workspace are used, all item actions (rename, delete, share, etc.) must function correctly

Stage Requirements:

 Expand table

Preview	General Availability
 Required	 Required

References:

- [How to Add Custom Item Action](#)
- [Item Manifest](#)

7.3 - Trial experience

Item should provide a sample or trial experience to help user explore features before committing to purchase.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [How to add a trial experience](#)
- [Fabric Templates](#) 

Guidelines:

UX:

7.4 - Monetization experience

Item should provide a sample or trial experience to help users explore features before committing

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [How to monetize the workload](#)
- [Fabric Templates](#) 

Guidelines:

UX:

7.5 - Jobs to be done

Item should define job types in the manifest to enable job tracking and monitoring integration

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [How to add Jobs to be done](#)
 - [Item Manifest](#)
-

7.6 - Item Settings are used

Item should implement settings panel for user configuration following Fabric patterns

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [Item Manifest](#)
- [How to Add Custom Settings](#)
- [Item Concept](#)

7.7 - Custom Item Actions

Item should define custom actions for the item in the workspace. All Actions need to work if defined.

Stage Requirements:

 Expand table

Preview	General Availability
 Optional	 Optional

References:

- [How to Add Custom Item Action](#)
- [Item Manifest](#)

Additional Information

Version History

This document is automatically generated.

Vendor Attestation Document Template

Executive Summary

This Vendor Self-Attestation document is designed for vendors to formally declare their compliance with the requirements outlined by Microsoft for publishing workloads using the Microsoft Workload Development Kit (WDK). It includes an attestation of adherence to standards in areas such as functional compatibility, security, performance, reliability, supportability, and legal compliance. Additionally, an attestation checklist is included to provide further details regarding each requirement and identify any exceptions, variances, or specific notes.

Key Terms Defined

Vendor / Independent Software Vendor (ISV) / Partner: The company or individual responsible for developing, distributing, and maintaining the workload using Microsoft Fabric's Workload Development Kit. Vendors may also provide value-added services, support, or integration solutions to extend the capabilities of Microsoft workloads and assist customers in their deployment and management. In this document, the vendor attests to their compliance with the requirements defined by Microsoft.

Customer: The end user or organization that uses the workload developed by the vendor. Customers benefit from the functionalities provided by the workload, and their needs drive the requirements for reliability, performance, and supportability.

Workload: A software component or solution designed to perform specific tasks on the Microsoft Fabric platform. Workloads typically interact with various services and require compatibility, reliability, and security measures to ensure seamless operation.

Microsoft Fabric Workload Development Kit (WDK): A toolkit provided by Microsoft that includes tools, guidelines, and best practices for developing and publishing workloads to run efficiently on the Microsoft Fabric platform.

Publish Workload Requirements: A set of requirements and standards specified by Microsoft that each workload must meet to be published and deployed within the Microsoft environment. These requirements include functional, security, performance, and compliance standards.

The document serves as an assurance from vendors to Microsoft and its Customers, ensuring that all requirements are met for the safe and efficient operation of workloads within the Microsoft

ecosystem.

The appendix allows vendors to provide detailed information about each requirement, specifying whether it's supported, and detailing any exceptions or additional information for clarity.

Process

The document is composed of three sections. All sections are to be provided to Microsoft as Vendor’s formal attestation, while ONLY Section III, which details all of the specifics on the attestation are to be hosted on the partners website for customers to reference.

Microsoft must be notified promptly prior to releasing any changes if the changes materially impact the attestation especially regarding security, compliance, privacy and/or if the change has significant variance from the design / UX guidelines.

Section I

ISV Information

Vendor Information

 Expand table

Description	Required or Optional
Company Name:	(Required)
Company Website:	(Required)
Address:	(Required)
City:	(Required)
State:	(Required)
Postal Code:	(Required)
County:	(Required)
Phone:	(Optional)

Primary Contact

 Expand table

Description	Required or Optional
Name:	(Required)
Title:	(Required)
Email:	(Required / Email Alias OK)

Section II

Attestation

To: Microsoft Corporation

Subject: Vendor Self-Attestation for Compliance with Microsoft Workload Development Requirements

We, the undersigned, [Vendor Name], hereby confirm and attest that we have reviewed, understood, and complied with all applicable requirements as outlined in the Microsoft Extensibility toolkit documentation, specifically the [Publish Workload Requirements](#).

Section III

Publish Workload Requirements Attestation Checklist

We, the vendor, [Vendor Name], confirm and attest to reviewing, meeting, and complying with the requirements outlined in the Microsoft Fabric Extensibility Toolkit specifically the [Publish Workload Requirements](#)

The following sections documents details, exceptions, or variances regarding the attestation of adherence to the Publish Workload Requirements.

Workload Information

 Expand table

Description	Required or Optional
Workload Version:	(Required)
Workload Name:	(Required)
Release Date:	(Required / Email Alias OK)

Business Requirements

Value To Customers

The workload provides the following value to customers –

ⓘ Note

Short paragraph here with text and / or bulleted items

Trial

We provide an easy and fast trial experience. The trial is available to the customer without waiting time (less than 5 seconds), and provides a free and easy way to explore the offered workload for a limited time in accordance with Microsoft guidelines for Trials

☐ Yes

☐ No

ⓘ Note

Description of the trial and the limits around the trial

Monetization

The workload is available on the marketplace for the customer to procure with or without a trial in accordance with the monetization guidelines

☐ Yes

☐ No

ⓘ Note

Link to the workloads here

Technical Requirements

Microsoft Entra Access

The workloads use Microsoft Entra authentication and authorization.

☐ No other authentication and authorization mechanisms are used

☐ Different authentication and authorization mechanisms are used for stored data In Fabric

ⓘ Note

Provide details here: Provide the endpoint / tenant ID for the Microsoft Entra If other Non Microsoft Entra Authentication and Authorization Mechanisms are required, outline details.

One Lake

Workloads integrate with One Lake to store data in the standard formats supported by the Fabric platform so that other services can take advantage of it.

☐ All data and metadata is stored in One Lake or Fabric Data Stores

☐ Not all data and metadata is stored in One Lake or Fabric Data Stores

ⓘ Note

Clarify what data or metadata is stored outside of One Lake and/or Fabric

Microsoft Entra Conditional Access

Enterprise customers require centralized control and management of the identities and credentials used to access their resources and data and via Microsoft Entra to further secure their environment via conditional access.

☐ The service works in its entirety with even if customers enable this functionality

☐ The service works in with limitations if customers enable this functionality

☐ The service doesn't work Microsoft Entra Conditional Access

ⓘ Note

Clarify limitations, constraints, exceptions if applicable

Admin REST API

Admin REST APIs are an integral part of Fabric admin and governance process. These APIs help Fabric admins in discovering workspaces and items, and enforcing governance such as performing access reviews, etc. Basic functionality is supported as part of the Workload Development Kit and doesn't need any work from Partners.

☐ Microsoft Fabric Admin APIs are being used (/admin/*)

☐ No Microsoft Fabric Admin APIs are being used

Customer Facing Monitoring & Diagnostic

Health and telemetry data needs to be stored for a minimum for 30 days including activity ID for customer support purposes, including Trials.

☐ Minimum 30 days requirement is adhered to

☐ Vendor stores the data for __ days beyond the minimum requirement

B2B

The implementation of the workload is in line with Microsoft Fabric's sharing strategy focused on allowing customers to collaborate with their business partners, customers, vendors, subsidiaries, etc. It also means users from other tenants can potentially be granted access to items partners are creating.

☐ Cross tenant B2B collaboration supported

☐ Workload Item Access only within the tenant

Note

Clarify limitations, constraints, exceptions if applicable

Business Continuity and disaster recovery

The vendor has a comprehensive Business Continuity and Disaster Recovery (BCDR) plans designed to tackle unplanned disasters and recovery steps.

Note

Either provide a link to the BCD plan or TOS (Terms of Service) for their offering here along with a summary

Performance

The Workload implementation takes measures to test and track performance of their Items

[] Performance Metrics on workload performance are available via the monitoring hub

[] Workload includes a separate monitoring UI to test and track performance

[] Performance tracking isn't currently available to the end user however vendor support personnel can monitor, test, track performance via their internal instrumentation and monitoring systems

ⓘ Note

Notes here

Presence

To ensure that customer expectations independent of their home or capacity region are met, vendors need to align with fabric regions and clouds. Availability in certain restrictions also impacts your Data Residency commitments.

[] Service availability and colocation/alignment in the following fabric regions

ⓘ Note

Notes here

[] All or part of the service doesn't reside in Azure

ⓘ Note

Notes here

Public APIs

Fabric Public APIs are the backbone of automation, enabling seamless communication and integration for both customers and partners within the Fabric ecosystem. Fabric Public API empowers users to build innovative solutions, enhance scalability, and streamline workflows.

[X] The workload uses Fabric Public APIs

Design / UX Requirements

Common UX

The workload and all item types the partner provides as part of it comply with the Fabric UX guidelines.

[] The following variance and/or exceptions have been granted by Microsoft

 Note

Exception and reason for variance and/or exception

Item Creation Experience

The item creation experience is in accordance with the Fabric UX System.

[] Yes

[] No

Monitoring Hub All Long running operations need to integrate with Fabric Monitoring Hub

[] Yes

[] No

Trial Experience

The workload provides a Trial Experience for users as outlined in the design guidelines

[] Trial Supported

[] Trial Not Supported

Monetization Experience

The monetization experience is in line with the design guidelines provided

☐ The monetization experience is integrated with the market place and compliant with the guidelines

☐ Bring Your Own License (BYOL)

☐ Free / Freemium

☐ Other

Note

Limitations here, outline details for Other and Freemium models link to Terms of Service if applicable

Accessibility

The user experience is in compliance with the Fabric UX design guidelines for Accessibility

☐ The user experience is compliant with the guidelines

☐ The following limitations exist

Note

Limitations here

World Readiness / Internationalization

English is supported as the default language. Localization through optional, should be considered.

☐ English is the only supported language

☐ The following languages are supported

Note

List of Languages

Item Settings

Item settings are implemented as a part of the ribbon as outlined in the UX guidelines

☐ Yes

☐ No

ⓘ Note

Clarify any variances/exceptions

Samples

Samples are optionally provided that preconfigure items of their type to help customers get started more easily.

☐ Samples not provided

☐ Samples for preconfiguration of items provided

Custom Actions

Custom actions can be optionally provided as a part of the item editor.

☐ Custom Actions aren't implemented

☐ Custom Actions implemented as part of Workload

ⓘ Note

List of custom actions

Workspace settings

Workspace settings provide a way that workloads can be configured on a workspace level.

☐ Supported

☒ Not Supported

Global Search

Searching for items in Fabric is supported through the top search bar.

☐ Supported

☒ Not supported

Security / Compliance Requirements

Security general

Protection of customer data and metadata is of paramount importance. Workloads must go through a security review and assessment. Vendor attests that the security review and assessment was completed and will be periodically performed as enhancements and changes are made. Security issues discovered which could have a detrimental impact on the customer should be addressed promptly and customers notified where applicable.

Note

Please outline all the security and compliance tests, attestations performed. The more detailed this is the easier it's for customers/Microsoft to review. This should include references to the latest security assessment reports

Privacy

Partners that build workloads also have a responsibility to protect that data when they access it. Every workload goes through a privacy assessment and a privacy review. Vendor attests that privacy review was completed and is periodically performed as enhancements and changes are made.

****Extra Requirements:**

☐ Publisher attests that only essential HTTP-only cookies are used by the Workload and only after positively authenticating the user.

☐ Publisher attests that it's not using or relying on third-party cookies as part of their solution.

☐ Publisher attests that's obtaining any Microsoft Entra token using the JavaScript APIs provided by the Fabric Workload Client SDK

Note

Outline all the privacy assessment and reviews performed. Links to latest reports and privacy policy provided here

Data Residency

Microsoft Fabric is making an Enterprise Promise around data not leaving the geography of the tenant for stored data and data in transit. As a workload in Fabric directly and users need to be aware what your commitments to Data Residency are. Define what your commitments are to the Data Residency of customer data.

ⓘ Note

Summarize your commitments. For details on Data Residency and compliance, provide an existing link to your Data Residency and compliance support>

Compliance

The publisher attests to the following security, data, and compliance regulations and standards

ⓘ Note

Summarize how your app handles security, data, and compliance. Describe the Workload's security attributes and data-handling practices. If applicable to your customers, align with more Fabric certifications. List all standards compliance / reports/ certifications here ISO, SOC2, HIPAA, and others For more details reference a link to your security, data, and compliance information.

Support

Live site

Partner workloads are an integral part of Fabric that requires that the Microsoft support teams are aware of how to contact you in case customers are reaching out to us directly.

Microsoft direct vendor outreach:

| Contact Name/Team: | (Vendor Primary contact Name / Team Here) | | Number | (Optional Phone # here) | | Email alias | (Email alias here) | | Self Service portal | (Link to website) |

Supportability

Vendors are responsible for defining and documenting their support parameters (Service level agreement, contact methods, ...). This information needs to be linked from the Workload page and should always be accessible to customers. In addition, the Marketplace criteria, need to be taken into account for the listing of the SaaS offer.

☐ Vendor attests that support information is published to the marketplace offering and available to user/customers directly via the workload

Service Health and Availability

Vendors need to host a service health dashboard that shows their service health and availability to customers. This information can be included on the Supportability page.

Service health dashboard can be found here:

Note

URL to service health dashboard

Fabric Features

Application Life Cycle Management (ALM)

Microsoft Fabric's lifecycle management tools enable efficient product development, continuous updates, fast releases, and ongoing feature enhancements.

☐ Supported

☒ Not Supported

CI/CD (Continuous Integration / Continuous Deployment)

The workload supports CI/CD pipelines to enable automated deployment and integration workflows within Microsoft Fabric.

☐ CI/CD is supported via the Fabric CICD manifest section

☐ CI/CD is not supported

ⓘ Note

Describe any limitations or details about the CI/CD integration

Item Definition Portability

Items can be exported with their full definition and restored in other workspaces, enabling lifecycle management and cross-workspace deployment.

☐ Items can be restored with their definition in other workspaces

☐ Items cannot be restored in other workspaces

ⓘ Note

Describe any limitations or constraints around item portability across workspaces

Private Links

In Fabric, you can configure and use an endpoint that allows your organization to access Fabric privately.

☐ Supported

☒ Not Supported

Data Hub

The OneLake data hub makes it easy to find, explore, and use the Fabric data items in your organization that you have access to. It provides information about the items and entry points for working with them. If you're implementing a Data Item, show up in the Data Hub as well.

☐ Supported

☒ Not Supported

Data Lineage

In modern business intelligence (BI) projects, understanding the flow of data from the data source to its destination can be a challenge. The challenge is even bigger if you built advanced analytical projects spanning multiple data sources, data items, and dependencies. Questions like

"What happens if I change this data?" or "Why isn't this report up to date?" can be hard to answer.

[] Supported

[X] Not Supported

Sensitivity labels

Sensitivity labels from Microsoft Purview Information Protection on items can guard your sensitive content against unauthorized data access and leakage. They're a key component in helping your organization meet its governance and compliance requirements. Labeling your data correctly with sensitivity labels ensures that only authorized people can access your data.

Extra requirements:

For partners that are using Export functionality within their Item they need to follow the guidelines.

[] Supported

[X] Not Supported

Extra Notes

Use this section to provide any further explanations, references, or notes that may be relevant to your attestation:

ⓘ Note

Additional Notes by Vendor

References

ⓘ Note

Consolidate all relevant reference here for completeness, e.g. support policy, compliance, service health, etc. might seem redundant but useful to have it in a single place

How to consume workloads in Microsoft Fabric

This guide explains how to add workloads in Microsoft Fabric through the Workload Hub at different organizational levels.

Overview

Microsoft Fabric allows you to extend its capabilities by adding workloads at three different organizational levels. All workload assignments and management are done through the **Workload Hub**, where customers can discover, evaluate, and manage workloads:

- **Tenant level:** Available to all users across the organization
- **Capacity level:** Available to all workspaces assigned to specific capacities
- **Workspace level:** Available only within specific workspaces

ⓘ Note

All workload consumption and management activities are centralized in the Workload Hub, providing a unified experience regardless of the organizational level.

Prerequisites

Before consuming workloads, ensure you have:

- Appropriate permissions at the intended level (tenant admin, capacity admin, or workspace admin)
- Access to Microsoft Fabric with the necessary licensing

Adding workloads through the Workload Hub

Tenant-level assignment

Who can do this: Tenant administrators

1. Navigate to the Fabric Workload Hub
2. Browse workloads and select "Add to Tenant"
3. Review permissions and confirm enablement for the entire organization

Result: Workload becomes available to all users across the organization.

Capacity-level assignment

Who can do this: Capacity administrators

1. Access the Workload Hub with capacity administrator credentials
2. Select "Add to Capacity" for your chosen workload
3. Choose the target capacity from your managed capacities

Result: Workload becomes available to all workspaces assigned to that capacity.

Workspace-level assignment

Who can do this: Workspace administrators

1. Navigate to the Workload Hub from your workspace
2. Select the desired workload and choose "Add to Workspace"
3. Follow the installation and configuration process

Result: Workload becomes available only within that specific workspace.

Finding workloads

The Workload Hub provides a central marketplace where you can:

- **Browse by category:** Data engineering, analytics, visualization, industry-specific solutions
- **Search and filter:** Find workloads by functionality, vendor, or requirements
- **Review details:** Capabilities, documentation, ratings, and vendor information

Managing workloads

Monitoring usage

- Track workload performance and resource consumption
- Review usage patterns and user adoption

Updates and maintenance

- Enable automatic updates where appropriate
- Test updates in non-production environments first

Removing workloads

- Review dependencies before removal
- Communicate changes to affected users

Best practices

- **Start small:** Begin with workspace-level assignments to test workloads
- **Plan capacity:** Consider resource requirements during capacity planning
- **Govern access:** Establish clear policies for workload approval
- **Monitor performance:** Track resource consumption and user adoption
- **Document usage:** Maintain records of workload purposes and owners

Troubleshooting

Workload not available: Verify permissions and check if enabled at higher levels (tenant/capacity)

Permission errors: Ensure you have administrator rights at the appropriate level

Performance issues: Review resource consumption and consider capacity scaling

Next steps

Once you've added workloads:

- Explore workload-specific documentation and tutorials
- Train your team on new capabilities
- Set up monitoring and governance processes

For custom workload development, see [Getting Started with Fabric Extensibility Toolkit](#).

FET Authentication overview

Fabric Extensibility Toolkit workloads rely on integration with [Microsoft Entra ID](#) for authentication and authorization. The toolkit uses a frontend-only architecture that simplifies the authentication flow compared to traditional workloads with separate backend components.

All interactions between your workload frontend and Fabric, Azure, or external services must be accompanied by proper authentication support. The toolkit handles token acquisition and management through JavaScript APIs that integrate seamlessly with the Fabric platform.

It's recommended that you become familiar with the [Microsoft identity platform](#) before starting to work with Fabric workloads. It's also recommended to go over [Microsoft identity platform best practices and recommendations](#).

Frontend-only authentication flow

The Extensibility Toolkit uses a simplified authentication model where your frontend application directly authenticates with various services:

- **Frontend to Fabric APIs:** Your workload frontend uses tokens to call Fabric REST APIs directly
- **Frontend to Azure services:** Use the same tokens to authenticate against Azure services like storage, Key Vault, and other Entra-secured applications
- **Frontend to external services:** Authenticate against any Entra-secured application using the acquired tokens

Token versatility

The tokens acquired through the Extensibility Toolkit can be used to authenticate against:

- **Fabric APIs:** Access workspaces, items, and platform features
- **Azure services:** Storage accounts, Key Vault, databases, and other Azure resources
- **Custom applications:** Any Entra-secured application you own or have been granted access to
- **Third-party services:** External services that support Entra ID authentication

Scope abstraction

The Extensibility Toolkit provides abstracted authentication through client libraries:

- **Fabric scopes:** Required scopes for Fabric APIs are automatically managed by the toolkit's client libraries

- **Azure scopes:** Standard Azure service scopes are handled transparently
- **Custom scopes:** You can specify additional scopes for your own applications

Authentication JavaScript API

The Fabric frontend offers a JavaScript API for Extensibility Toolkit workloads to acquire tokens for authentication. The API handles:

- Token acquisition for your workload
- Consent management for required permissions
- Token refresh and expiration handling
- Integration with Fabric's security context

For detailed information about using the authentication API, see [Authentication JavaScript API](#).

To understand why consents are required, review [User and admin consent in Microsoft Entra ID](#).

How consents work in Extensibility Toolkit workloads

The Extensibility Toolkit simplifies consent management:

1. **Automatic consent prompting:** When your workload needs access to a service, the platform automatically prompts for consent
2. **Redirect handling:** The toolkit manages consent redirects and popup closure automatically
3. **Consent caching:** Once granted, consents are cached and reused for subsequent requests
4. **Granular permissions:** You can request specific scopes for different services as needed

Authentication setup

Before using authentication in your workload:

- **Register your application** in Microsoft Entra ID
- **Configure redirect URIs** for your workload
- **Set required permissions** for Fabric and any external services
- **Test authentication flow** in your development environment

For step-by-step authentication setup instructions, see [Authentication guidelines](#).

Related content

- [Authentication JavaScript API](#)
 - [Authentication guidelines](#)
-

Last updated on 12/15/2025

Authentication guidelines for Microsoft Fabric Extensibility Toolkit

This article provides guidelines on how to work with authentication when building Microsoft Fabric extensibility toolkit workloads. It includes information about working with tokens, consents, and accessing various services from your frontend application.

Before you begin, make sure you're familiar with the concepts in [Authentication overview](#).

Frontend-only authentication model

The Extensibility Toolkit uses a frontend-only architecture that simplifies authentication compared to traditional workloads:

- **Direct API calls:** Your frontend directly calls Fabric APIs, Azure services, and external applications
- **Token reuse:** A single token can be used to authenticate against multiple Entra-secured services
- **Simplified consent:** Consent management is handled by the platform with automatic prompting

Microsoft Entra application configuration

Expose an API tab

Configure scopes for your workload application:

- **Fabric integration scopes:** Preauthorize Microsoft Power BI with application ID `871c010f-5e61-4fb1-83ac-98610a7e9110`
- **Custom API scopes:** Add scopes for any custom APIs your workload exposes
- **Granular permissions:** Use different scopes for read vs. write operations

For example, if your workload exposes data APIs:

- Add `data.read` scope for read operations
- Add `data.write` scope for write operations
- Validate appropriate scopes in your API handlers

API permissions tab

Configure permissions for external services your workload needs to access:

- **Required:** `Fabric.Extend` under Power BI Service (mandatory for Fabric integration)
- **Azure services:** Add scopes for Azure services like `https://storage.azure.com/user_impersonation`
- **Custom applications:** Add scopes for your own Entra-secured applications
- **Third-party services:** Include any external services that support Entra authentication

Token usage patterns

Using tokens for multiple services

The frontend token acquired through the Extensibility Toolkit can be used to authenticate against:

Fabric APIs

TypeScript

```
// Token automatically includes required Fabric scopes
const response = await fetch('https://api.fabric.microsoft.com/v1/workspaces', {
  headers: {
    'Authorization': `Bearer ${token.accessToken}`
  }
});
```

Azure services

TypeScript

```
// Same token works for Azure services
const response = await fetch('https://management.azure.com/subscriptions', {
  headers: {
    'Authorization': `Bearer ${token.accessToken}`
  }
});
```

Custom applications

TypeScript

```
// Token works for your own Entra-secured applications
const response = await fetch('https://myapp.contoso.com/api/data', {
```



```
headers: {  
  'Authorization': `Bearer ${token.accessToken}`  
}  
});
```

Scope management

The Extensibility Toolkit abstracts scope management for common scenarios:

- **Fabric client libraries:** Automatically include required Fabric scopes
- **Azure client libraries:** Handle Azure service scopes transparently
- **Custom scopes:** Specify additional scopes when needed

Working with consents

Initial token acquisition

Start by acquiring a token to establish the authentication context:

TypeScript

```
const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: [] });
```

This call may result in:

- **Consent prompt:** If the user hasn't consented to your application
- **Silent acquisition:** If consent was previously granted

Handling additional service access

When your workload needs to access additional services, specify the required scopes:

TypeScript

```
try {  
  // Request token with specific scopes for Azure Storage  
  const token = await workloadClient.auth.acquireFrontendAccessToken({  
    scopes: ['https://storage.azure.com/user_impersonation']  
  });  
  
  // Use token to access Azure Storage  
  const response = await fetch('https://mystorageaccount.blob.core.windows.net/', {  
    headers: { 'Authorization': `Bearer ${token.token}` }  
  });  
} catch (error) {
```

```
// Handle authentication or authorization errors
console.error('Access failed:', error.message);
}
```

Example scenarios

Scenario 1: Accessing Fabric and Azure services

Your workload needs to:

- List Fabric workspaces
- Read from Azure Storage
- Write to Azure Key Vault

Implementation:

1. Configure API permissions for required services
2. Acquire initial token
3. Use token for all service calls
4. Handle consent prompts as needed

Scenario 2: Custom application integration

Your workload integrates with your own backend service:

1. **Configure your backend:** Ensure it accepts Entra tokens
2. **Add API permissions:** Include your backend's scopes in the workload application
3. **Use standard authentication:** The same token pattern works for your custom services

Scenario 3: Third-party service integration

Integrating with external Entra-enabled services:

1. **Service registration:** Register your workload with the third-party service
2. **Scope configuration:** Add the service's scopes to your API permissions
3. **Token usage:** Use the same authentication pattern for external services

Error handling and troubleshooting

Common authentication errors

- **Consent required:** User hasn't granted permission for a specific scope
- **Conditional access:** Additional authentication requirements (e.g., MFA)
- **Token expiration:** Token has expired and needs refresh
- **Invalid scope:** Requested scope isn't configured or available

Error handling patterns

TypeScript

```
async function handleAuthenticatedRequest(url: string, requiredScopes: string[] = []) {
  try {
    const token = await workloadClient.auth.acquireFrontendAccessToken({
      scopes: requiredScopes
    });
    return await makeRequest(url, token);
  } catch (error) {
    if (error.code === 'consent_required') {
      // User needs to grant consent for the requested scopes
      console.error('Consent required for scopes:', requiredScopes);
    }
    throw error;
  }
}
```

Redirect URI handling

The extensibility toolkit includes built-in redirect URI handling for authentication consent popups. This is implemented in the main `index.ts` file and handles consent redirects automatically.

The toolkit handles:

- **Automatic window closure:** Consent popups close automatically after user interaction
- **Error handling:** Specific error codes are detected and handled appropriately
- **Error display:** Failed consent attempts show user-friendly error messages

Current implementation in the toolkit:

TypeScript

```
const redirectUriPath = '/close';
const url = new URL(window.location.href);
if (url.pathname?.startsWith(redirectUriPath)) {
  // Handle errors
  if (url?.hash?.includes("error")) {
    if (url.hash.includes("AADSTS650052")) {
```

```
// Handle missing service principal error
printFormattedAADErrorMessage(url?.hash);
} else if (url.hash.includes("AADSTS65004")) {
  // Handle user declined consent error
  printFormattedAADErrorMessage(url?.hash);
} else {
  window.close();
}
} else {
  // Close window on successful consent
  window.close();
}
}
```

ⓘ Note

The redirect URI handling is automatically included in the extensibility toolkit template. You don't need to implement this yourself unless you want to customize the error handling behavior.

Best practices

Token management

- **Cache tokens:** Reuse tokens until they expire
- **Handle refresh:** Implement automatic token refresh logic
- **Secure storage:** Store tokens securely in browser memory

Consent management

- **Minimal permissions:** Request only the scopes you actually need
- **Progressive consent:** Request additional permissions as features are used
- **Clear messaging:** Explain to users why permissions are needed

Error handling

- **Graceful degradation:** Provide fallback functionality when possible
- **User feedback:** Clearly communicate authentication requirements
- **Retry logic:** Implement appropriate retry mechanisms for transient failures

Related content

- [Authentication overview](#)
 - [Authentication JavaScript API](#)
-

Last updated on 12/15/2025

Authentication JavaScript API reference

The Fabric Extensibility Toolkit provides a JavaScript API for acquiring authentication tokens that can be used to access Fabric APIs, Azure services, and any Entra-secured application. This article provides comprehensive API reference and usage examples.

Tip

For a quick start guide, see [Acquire Microsoft Entra tokens](#).

API reference

JavaScript

```
acquireFrontendAccessToken(params: AcquireFrontendAccessTokenParams):  
Promise<AccessToken>;  
  
export interface AcquireFrontendAccessTokenParams {  
    scopes: string[];  
}  
  
export interface AccessToken {  
    token: string;  
}
```

Note

The current extensibility toolkit implementation supports basic token acquisition with scopes. Advanced features like full consent prompting and conditional access handling are not yet available but may be added in future releases.

The API returns an `AccessToken` object that contains:

- **token**: The JWT token string to use in Authorization headers

Basic usage

Simple token acquisition

JavaScript

```
// Acquire a token with default Fabric permissions
const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: [] });

// Use the token in API calls
const response = await fetch('https://api.fabric.microsoft.com/v1/workspaces', {
  headers: {
    'Authorization': `Bearer ${token.token}`,
    'Content-Type': 'application/json'
  }
});
```

Token with specific scopes

JavaScript

```
// Request specific scopes for Azure Storage
const token = await workloadClient.auth.acquireFrontendAccessToken({
  scopes: ['https://storage.azure.com/user_impersonation']
});
```

Token usage examples

Fabric API access

The token can be used directly with Fabric REST APIs:

JavaScript

```
async function listWorkspaces() {
  const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: [] });

  const response = await fetch('https://api.fabric.microsoft.com/v1/workspaces', {
    headers: {
      'Authorization': `Bearer ${token.token}`
    }
  });

  return await response.json();
}
```

Azure service access

Use scopes to specify the Azure services you need access to:

JavaScript

```
async function readFromStorage(accountName, containerName, blobName) {
  const token = await workloadClient.auth.acquireFrontendAccessToken({
    scopes: ['https://storage.azure.com/user_impersonation']
  });

  const url =
`https://${accountName}.blob.core.windows.net/${containerName}/${blobName}`;
  const response = await fetch(url, {
    headers: {
      'Authorization': `Bearer ${token.token}`,
      'x-ms-version': '2021-12-02'
    }
  });

  return await response.text();
}
```

Custom application access

Access your own Entra-secured applications:

JavaScript

```
async function callCustomAPI() {
  const token = await workloadClient.auth.acquireFrontendAccessToken({
    scopes: ['https://myapp.contoso.com/data.read']
  });

  const response = await fetch('https://myapp.contoso.com/api/data', {
    headers: {
      'Authorization': `Bearer ${token.token}`
    }
  });

  return await response.json();
}
```

Parameter reference

scopes

An array of scope strings that specify the permissions your token needs.

Common Azure service scopes:

- `https://storage.azure.com/user_impersonation` - Azure Storage

- https://vault.azure.net/user_impersonation - Azure Key Vault
- https://management.azure.com/user_impersonation - Azure Resource Manager
- <https://graph.microsoft.com/User.Read> - Microsoft Graph

Usage example:

JavaScript

```
const token = await workloadClient.auth.acquireFrontendAccessToken({
  scopes: [
    'https://storage.azure.com/user_impersonation'
  ]
});
```

Empty scopes array: Use an empty array to get a token with default Fabric permissions:

JavaScript

```
const token = await workloadClient.auth.acquireFrontendAccessToken({ scopes: [] });
```

Consent management

Automatic consent flow

The Extensibility Toolkit automatically handles consent workflows:

- **Initial request:** If consent is missing, a popup window opens
- **User interaction:** User grants or denies permissions
- **Automatic closure:** Popup closes automatically after user action
- **Token delivery:** If successful, the token is returned to your application

Consent popup handling

The toolkit manages consent popups automatically, but you can customize the redirect URI behavior. Create a redirect page that handles the consent response:

JavaScript

```
// redirect.js - Handle consent redirect
const redirectUriPath = '/close';
const url = new URL(window.location.href);

if (url.pathname?.startsWith(redirectUriPath)) {
  // Handle consent errors
}
```

```

if (url?.hash?.includes("error")) {
  // Extract error information
  const errorMatch = url.hash.match(/error=([^&]+)/);
  const errorDescription = url.hash.match(/error_description=([^&]+)/);

  // Handle specific errors
  if (url.hash.includes("AADSTS650052")) {
    console.error("Service principal not configured");
  } else if (url.hash.includes("AADSTS65004")) {
    console.error("User declined consent");
  }
}

// Always close the popup immediately
window.close();
}

```

Cross-tenant consent

For accessing resources across different tenants:

JavaScript

```

// Request consent for cross-tenant access
const token = await workloadClient.auth.acquireAccessToken({
  additionalScopesToConsent: ['https://api.partner-app.com/data.read']
});

```

Error handling

Common error scenarios

JavaScript

```

async function robustTokenAcquisition() {
  try {
    return await workloadClient.auth.acquireAccessToken();
  } catch (error) {
    switch (error.code) {
      case 'user_cancelled':
        console.log('User cancelled the consent dialog');
        break;
      case 'consent_required':
        console.log('Additional consent required');
        break;
      case 'interaction_required':
        console.log('User interaction required');
        break;
    }
  }
}

```

```
    default:
      console.error('Authentication error:', error.message);
    }
    throw error;
  }
}
```

Token expiration handling

JavaScript

```
class TokenManager {
  private currentToken: AccessToken | null = null;

  async getValidToken(): Promise<AccessToken> {
    if (!this.currentToken || this.isTokenExpired(this.currentToken)) {
      this.currentToken = await workloadClient.auth.acquireAccessToken();
    }
    return this.currentToken;
  }

  private isTokenExpired(token: AccessToken): boolean {
    // Add buffer time to prevent requests with almost-expired tokens
    const bufferMinutes = 5;
    const expirationWithBuffer = new Date(token.expiresOn.getTime() - (bufferMinutes
    * 60 * 1000));
    return new Date() >= expirationWithBuffer;
  }
}
```

Best practices

Token caching and reuse

- **Cache tokens:** Store tokens in memory until expiration
- **Automatic refresh:** Implement automatic token refresh before expiration
- **Secure storage:** Never persist tokens to local storage or cookies

Scope management

- **Minimal scopes:** Request only the permissions you need
- **Progressive consent:** Request additional scopes as features are used
- **Scope validation:** Verify tokens include required scopes before API calls

Advanced error handling

- **Graceful degradation:** Provide fallback functionality when authentication fails
- **User messaging:** Clearly explain why permissions are needed
- **Retry logic:** Implement appropriate retry mechanisms for transient failures

Performance optimization

- **Parallel requests:** Acquire tokens for multiple services in parallel when possible
- **Batch operations:** Group API calls to minimize token acquisition overhead
- **Cache management:** Implement efficient token caching strategies

Related content

- [Quick start: Acquire Microsoft Entra tokens](#) - Simple getting started guide
- [Authentication overview](#) - High-level authentication concepts
- [Authentication guidelines](#) - Best practices and recommendations
- [Access Fabric APIs](#) - Pre-built API wrappers

Last updated on 12/15/2025

Manifest overview

A NuGet-style package is used to define Fabric Workloads. This manifest is the contract between your workload and Fabric—it tells the host what your workload is, which capabilities it exposes, how and where to load it, and what items it contributes.

What's in the manifest package

The manifest package is composed of several parts that work together:

- Workload manifest: Declares identity, routes, entry points, permissions, and capabilities. See [Workload manifest](#).
- Product JSON: Provides product-level metadata used for discovery and publishing. See [Product manifest](#).
- Item manifests and JSON files: Define each item type your workload contributes, including creation, editing, viewing, and behaviors. See [Items](#).

Package structure

The workload is packaged as a NuGet package, combining backend and frontend components. The structure adheres to specific naming conventions and is enforced by Fabric for consistency across upload scenarios.

Backend structure

The backend segment is in the `BE` folder and comprises .xml files that define the workload and its associated items:

- `WorkloadManifest.xml` - The workload configuration file, required to have this exact name for Fabric's verification.
- `Item1.xml`, `Item2.xml`, ... - Manifests for individual items with flexible naming, following the XML format.

Frontend structure

The frontend section is in the `FE` folder and contains .json files detailing the product and items for the frontend, along with an `assets` directory for icons:

- `Product.json` - The main manifest for your product's frontend, which must be named precisely for Fabric's verification.
- `Item1.json`, `Item2.json`, ... - Manifests for individual items with flexible naming, following the JSON format. Each json corresponds to a backend manifest (for example, `Item1.json` to `Item1.xml`).
- `assets` folder - Stores all icons and localized resources used by the frontend.

Package limits

The following limits apply to all types of NuGet packages, both in development mode and cloud mode:

- Only `BE` and `FE` subfolders are permitted. Any other subfolders or files located outside these folders result in an upload error.
- The `BE` folder accepts only `.xml` files. Any other file type result in an upload error.
- A maximum of 10 item files is allowed, meaning the `BE` folder can contain one `WorkloadManifest.xml` and up to 10 `Item.xml` files. Having more than 10 item files in the folder result in an upload error.
- The `Assets` subfolder must reside under the `FE` folder. It can contain up to 15 files, with each file being no larger than 1.5 MB.
- Only the following file types are permitted in the `Assets` subfolder: `.jpeg`, `.jpg`, `.png`.
- The `FE` folder can contain a maximum of 10 item files plus one `product.json` file.
- The size of the `product.json` must not exceed 50 KB.
- Each asset within the `Assets` folder must be referenced within the item files. Any asset referenced from an item file that is missing in the `Assets` folder will result in an upload error.
- Filenames for items must be unique. Duplicate filenames result in an upload error.
- Filenames must contain alphanumeric (English) characters or hyphens only and cannot exceed a length of 32 characters. Using other characters or exceeding this length result in an upload error.
- The total package size must not exceed 20 MB.

Why the manifest matters

- Development: The manifest enables local registration and host bootstrapping so you can run and debug your workload inside Fabric.
- Publishing: The same manifest package is used to validate, certify, and publish your workload to the Workload Hub or tenant environments.

Scripts and automation in the Starter-Kit

The [Starter-Kit](#) repository includes scripts that automatically build the manifest package for you during setup and development. It reduces manual steps and keeps the contract in sync with your code.

How it's used in dev and production

- DevGateway: Uses the manifest to register your local development instance with Fabric so your app loads in an iFrame during development. See [DevGateway](#).
- Workloads: For testing and production, upload the manifest package through **OneLake catalog > Govern > Configurations > Workloads** as part of your publishing flow. See [Publish your workload](#).

Related content

- [Workload manifest](#)
- [Product manifest](#)
- [Items](#)
- [Getting started guide](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 09/29/2026

Workload manifest in Extensibility Toolkit



Summarize this article for me

The workload manifest (WorkloadManifest.xml) is the workload-level configuration for your workload. It tells Fabric who your workload is and how to reach your front end so the host can bootstrap it securely. It complements, but doesn't replace, the product and item definitions:

- Product metadata (for discovery and publishing) lives in Product.json.
- Item types and their behaviors live in the item manifests and JSON files.

What the workload manifest defines

- Workload identity: `WorkloadName` ([Organization].[WorkloadId]) and `Version` (semantic version). The WorkloadId portion cannot exceed 32 characters.
- Hosting model: `HostingType` (use `FERemote`)
- Front-end Microsoft Entra app: `AADFEApp` > `AppId`
- Front-end endpoints: `ServiceEndpoint` entries with `Name` (for example, `Frontend`), `Url` (localhost in dev; production domain must be a subdomain of your verified Entra domain), and `IsEndpointResolutionService`. See [General Publishing Requirements](#) for domain restrictions.
- Optional sandbox relaxation: `EnableSandboxRelaxation` only when special iFrame capabilities are required (for example, initiating file downloads)

Best practices

- Keep versions consistent across your app and manifest
- Declare the minimum permissions needed (principle of least privilege)
- Validate the manifest as part of CI

Structure

Key elements in the Manifest and what they mean:

- Root element with a schema version (for example, `SchemaVersion="2.0.0"`).
- `Workload` node with attributes such as:
 - `WorkloadName` — unique identifier in the form `[Organization].[WorkloadId]` (for example, `Org.MyWorkload`). The WorkloadId portion cannot exceed 32 characters. If you don't intend to publish to other tenants, you can use `Org.[WorkloadId]`. For publishing across tenants, register a full WorkloadName with Fabric.

- `HostingType` — indicates workload hosting; use `FERemote`.
- `Version` node — semantic version of your workload package.
- `RemoteServiceConfiguration` > `CloudServiceConfiguration` containing:
 - `Cloud` (for example, `Public`).
 - `AADFEApp` > `AppId` — contains the frontend Microsoft Entra application ID.
 - `EnableSandboxRelaxation` — set it to `true` if you require special iFrame capabilities (for example, initiating file downloads). Keep `false` by default for security.
 - `Endpoints` > `ServiceEndpoint` entries with:
 - `Name` (for example, `Frontend`).
 - `Url` — where the frontend is hosted. Use `https://localhost:port` for development. In production, the domain must be a subdomain of your verified Entra domain. See [General Publishing Requirements](#) for complete domain restrictions.
 - `IsEndpointResolutionService`.

Build output and placeholders

In the Starter-Kit, populates the placeholders every time the Manifest is created based on the configuration in the environment file. The final finalized manifest is created in the `build/Manifest` directory. Using placeholders and populating them from the environments allows you to build different environments without changing the files or the scripts. The build output is used by:

- DevGateway: Register your local development instance with Fabric so your app can load inside the Fabric portal during development.
- DevServer: Which is providing the information to the Fabric
- Admin Portal: for test and production, upload the manifest package through the Fabric Admin Portal as part of publishing. See [Publish your workload](#).

Backend folder limits

The backend (`BE`) folder has the following constraints:

- Accepts only `.xml` files. Any other file type result in an upload error.
- Can contain one `WorkloadManifest.xml` file (required).
- Can contain up to 10 `Item.xml` files. Having more than 10 item files result in an upload error.

See [Manifest overview](#) for complete package limits.

Learn more

- [Architecture](#)

Related content

- [Getting started guide](#)
- [Setup guide](#)
- [Fabric Public REST APIs](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/04/2026

Product manifest



Summarize this article for me

The `Product.json` describes the front-end behavior and product metadata for your workload. Fabric uses it to render the workload information in the portal and to surface information such as display names, descriptions, icons, documentation links, and attestation.

What `Product.json` defines

- Front-end behavior hints for your workload in the Fabric portal
- Product metadata for discovery and presentation (names, descriptions, categories)
- Links to your documentation support, and attestation pages
- Icons and other visual assets referenced by the product
- Internationalized strings resolved from the `assets` folder

Internationalization and assets

Use the `assets` folder to store localized string resources and images that `Product.json` references. Strings are resolved via the internationalization model defined in the repository, so you can present localized names and descriptions across the Fabric portal.

Frontend folder and `Product.json` limits

The frontend (`FE`) folder has the following constraints:

- Can contain a maximum of 10 item files plus one `product.json` file.
- The `product.json` file must not exceed 50 KB in size.
- The `Assets` subfolder must reside under the `FE` folder and can contain up to 15 files.
- Each file in the `Assets` folder must be no larger than 1.5 MB.
- Only `.jpeg`, `.jpg`, and `.png` file types are permitted in the `Assets` subfolder.
- Each asset within the `Assets` folder must be referenced within the item files. Any asset referenced from an item file that is missing in the `Assets` folder will result in an upload error.

See [Manifest overview](#) for complete package limits.

How it fits with other manifests

- Workload manifest (WorkloadManifest.xml): runtime configuration for identity, hosting, endpoints. See [Workload manifest](#).
- Item manifests and JSON files: per-item definitions and behaviors. See [Items](#).

Related content

- [Manifest overview](#)
- [Workload manifest](#)
- [Items](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/04/2026

Item manifest



Summarize this article for me

Each item contributed by your workload is defined by two files that work together and align with the workload and product manifests:

- `{ItemName}.xml` — Defines the item type itself and core parameters.
- `{ItemName}.json` — Defines front-end behavior (where items can be created, display names, icons, and UI hints).

What the XML defines

Use the XML file to declare the item type and important parameters that Fabric needs to recognize and manage the item. This includes identity and behaviors that align with your workload configuration.

What the JSON defines

Use the JSON file to describe front-end behavior and presentation, such as:

- Where items can be created in the Fabric experience
- Display names, descriptions, and icons for the item
- Other UI hints that the Fabric portal uses when rendering your item

Naming and generation

Important

Item names must be unique within a single workload. Choose distinct names to avoid collisions across your item manifests.

In the [Starter-Kit](#), PowerShell scripts read values from the environment file and populate placeholders in newly created item XML files and during manifest builds. This keeps item identifiers and metadata consistent across XML/JSON and the workload/product manifests.

Item filename constraints

Item filenames must meet the following requirements:

- Filenames must be unique within the workload. Duplicate filenames result in an upload error.
- Filenames must contain alphanumeric (English) characters or hyphens only.
- Filenames cannot exceed a length of 32 characters.
- Using other characters or exceeding the length limit result in an upload error.

See [Manifest overview](#) for complete package limits.

How item manifests fit with other manifests

- Workload: runtime configuration for identity, hosting, and endpoints. See [Workload manifest](#).
- Product: portal-facing metadata and links. See [Product manifest](#).

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/04/2026

Fabric Item Overview

Fabric extensions are built around the fundamental concept of **items**. Every workload in Microsoft Fabric consists of one or more item types that users can create, configure, and interact with through standardized editors.

Item-based architecture

All Fabric workloads follow a consistent item-based structure:

- **Items** are the core building blocks that users create and manage in workspaces
- **Editors** provide the interface for users to configure and work with items
- **Components** are the standardized building blocks that make up editors

Item structure

Each Fabric item consists of five key components:

- Item Model (`ItemModel.ts`)

Defines the data structure and properties that represent the item's configuration and state that is stored in Fabric.

- Item Editor (`ItemEditor.tsx`)

The main interface where users interact with the item. Built using default Fabric components for consistency.

- Empty View (`ItemEditorEmpty.tsx`)

The initial experience when users first create an item, providing onboarding and setup guidance.

- Detail View (`ItemEditorDetail.tsx`)

Provides drill-down navigation and detailed views for complex item configurations and hierarchical content.

- Ribbon Actions (`ItemEditorRibbon.tsx`)

Toolbar actions that users can perform on the item, such as save, settings, and custom operations.

Default editor components

Fabric provides standardized components that ensure consistency across all workloads:

Core editor components

- **ItemEditor**: Main container with automatic layout management
- **ItemEditorDefaultView**: Two-panel layouts with resizable splitters
- **ItemEditorDetailView**: Navigation for drill-down scenarios
- **ItemEditorEmptyView**: Standardized first-run experience

Panel system

- **Left panel**: Navigation, file explorers, OneLake views
- **Center panel**: Main content area with automatic scrolling
- **Collapsible panels**: Responsive layouts with accessibility support

Ribbon system

- **Home toolbar**: Standard actions like save and settings
- **Additional toolbars**: Custom actions specific to your item type
- **Tooltip integration**: Consistent user experience patterns

Benefits of the item structure

Consistency: Users get a familiar experience across all Fabric workloads

Accessibility: Built-in support for keyboard navigation and screen readers

Responsive design: Automatic layout adjustments for different screen sizes

Integration: Seamless connection with Fabric platform features like OneLake, lineage, and permissions

Development approach

When building Fabric extensions:

- **Start with the item model** - Define what data your item will store
- **Use default components** - Leverage Fabric's standardized editor components
- **Follow established patterns** - Implement ribbon actions, panel layouts, and navigation consistently
- **Customize content** - Fill the standardized structure with your workload-specific functionality

Key principles

- **Component reuse:** Use Fabric's default components instead of creating custom interfaces
- **Standard patterns:** Follow established navigation, layout, and interaction patterns
- **Platform integration:** Leverage built-in Fabric capabilities for storage, security, and management
- **User familiarity:** Maintain consistency with the broader Fabric user experience

Next steps

To implement items in your workload:

- [Create Custom Fabric Items](#)
- [Define Item Creation](#)
- [Store Item Definition](#)

For detailed component documentation, see the [ItemEditor documentation](#) [↗](#).

Last updated on 12/15/2025

DevGateway with Microsoft Fabric

The DevGateway is a small helper you run during development to register your local workload instance with Fabric. It tells the Fabric service where your frontend is hosted (your DevServer on localhost) and which items are available, based on your manifests. With the DevGateway running, Fabric can load your UI in an iFrame and call your local DevServer to read product and item metadata—without publishing a package to your tenant.

What the DevGateway does

- Registers a local workload instance in a specific workspace for the current signed-in developer.
- Informs Fabric about your workload identity and entry point using your manifests.
- Routes Fabric host calls to your local environment so the platform can load your frontend and discover items while you iterate.
- Works hand-in-hand with the [DevServer](#), which actually hosts your frontend and exposes manifest endpoints.

ⓘ Note

The DevGateway doesn't serve your web app or manifests itself. Instead, it registers your local instance and points Fabric to your DevServer and manifests.

How it uses manifests in development

Your workload is manifest-driven in both development and production. During development:

- The DevGateway uses your local configuration to advertise the workload's identity and entry points (as defined in the [Workload manifest](#)).
- Fabric then calls your [DevServer](#) to fetch the [Product manifest](#) and [Item manifests](#) via local endpoints (for example, the template's `/manifests` route), so the service can render navigation, tiles, and creation experiences.

This mirrors what happens at publish time, but everything stays local for fast iteration.

Typical development flow

1. Start your DevServer to host the frontend on localhost.
2. Start the DevGateway and sign in with a user who is an admin of the target workspace.

3. Open your Fabric workspace and use the workload entry point; Fabric loads your UI in an iFrame and discovers items via your manifests.
4. Edit UI and manifest files, refresh to see changes immediately.

See the [Getting Started tutorial](#) for step-by-step setup and start instructions.

Configuration inputs

Depending on your template, the DevGateway reads a local config that includes:

- Target workspace for registration
- Local frontend endpoint (your DevServer URL)
- Paths or references to your manifest files/assets used during development

Your repository's setup/build scripts generate or update this config automatically so the DevGateway and DevServer stay in sync.

Troubleshooting tips

- If the UI doesn't open in Fabric, ensure you started both the DevGateway and the DevServer, and that you're signed in to a workspace where you have the required permissions.
- If Fabric can't discover items, confirm your DevServer's manifests endpoint returns valid JSON and your DevGateway is pointing the service to the correct localhost URL.
- If icons or strings are missing, verify the assets paths in your manifests and that your DevServer serves them.

See also

- [DevServer](#)
- [Manifest overview](#)
- [Workload manifest](#)
- [Product manifest](#)
- [Item manifest](#)

DevServer with Microsoft Fabric



Summarize this article for me

The DevServer is the local web server you run while developing a Fabric workload. It serves your frontend (SPA) on localhost and provides a small set of HTTP endpoints that Fabric calls during development to retrieve your Product and Item manifests. When combined with the DevGateway, Fabric can load your workload UI in an iFrame and read your manifest data without publishing anything to your tenant.

What the DevServer does

- Hosts your workload frontend over HTTP on localhost (for example, <http://localhost:60006>) so Fabric can load it in an iFrame.
- Serves static assets referenced by your manifests (icons, localized strings, images).
- Exposes local JSON endpoints Fabric uses to read your manifests during development.
- Enables rapid edit-refresh cycles with hot reload in most setups.

Important

The DevServer works together with the [DevGateway](#). The DevGateway registers your local workload instance with Fabric so the service knows to talk to your DevServer endpoints while you develop.

Where Fabric calls the DevServer

When you enable development mode and start both your DevGateway and DevServer:

- Fabric navigates to your frontend via the frontend endpoint defined by your workload manifest (see [Workload manifest](#)). In development, this usually points to a localhost URL exposed by the DevServer.
- Fabric queries the DevServer for your product-facing metadata so it can render navigation, tiles, and other UX for your workload. This allows you to iterate on `Product.json` and item manifests without rebuilding and uploading a package.

Local endpoints the DevServer provides

Exact routes can vary by template, but the sample repository exposes a small set of predictable endpoints:

- GET / — returns your web app (the UI Fabric loads into an iFrame).
- GET /manifests — returns a JSON payload that aggregates your Product manifest and Item manifests used by the frontend. This mirrors the structure Fabric expects at publish time (see [Product manifest](#) and [Item manifest](#)).
- GET /assets/... — serves icons, images, and localized strings referenced by your manifests.

ⓘ Note

- CORS and headers are preconfigured in the sample DevServer so the app can be embedded and communicate with the host.
- The route names above follow the current sample; consult your template's README if your project uses a different path for the manifests endpoint.

Typical development flow

1. Start the DevServer from the sample repository to host your frontend on localhost.
2. Start the DevGateway to register your local workload with Fabric.
3. Open your Fabric workspace and launch the workload entry point; Fabric loads your app in an iFrame and calls your DevServer endpoints to read manifest data.
4. Edit UI or manifest files and refresh; changes take effect immediately without repackaging.

For how to start each process, see the [Getting Started tutorial](#) and the [Setup guide](#).

Relationship to published manifests

In production, your workload's manifests are packaged and uploaded as part of your workload's NuGet package (see [Manifest overview](#)). During development, the DevServer's local endpoints act as a lightweight stand-in for those packaged files so you can iterate quickly:

- Schema and rules are the same as for published manifests.
- The DevServer only affects local development; it doesn't change how publishing works.

Troubleshooting tips

- If the iFrame shows a blank page, confirm the DevServer is running and the frontend endpoint in your manifest points to the correct localhost URL.
- If icons or strings are missing, check the `assets` paths and that the DevServer is serving those files under `/assets`.

- If Fabric can't find your manifests, verify the `/manifests` route exists in your template and returns valid JSON.

Remote endpoint stubs for local development

The DevServer includes built-in stub implementations for remote endpoints, allowing you to test jobs and lifecycle notifications locally without deploying to Azure or other cloud services. These stubs intercept calls that would normally go to your production remote endpoints and handle them on your local machine.

What are remote endpoint stubs?

Remote endpoint stubs are local implementations of the remote endpoint contract that:

- Receive job execution requests and lifecycle notifications from Fabric
- Log request details to your console for immediate visibility
- Provide sample implementations showing how to interact with OneLake and other Fabric services
- Demonstrate token handling and authentication patterns
- Enable rapid iteration without cloud deployment

Features supported by stubs

The DevServer stubs support:

Job execution

- On-demand and scheduled job triggers
- Job lifecycle management (start, status, cancel)
- Sample implementations showing data processing patterns
- OneLake integration examples

Lifecycle notifications

- Item creation notifications
- Item update notifications
- Item deletion and soft delete notifications
- Sample infrastructure provisioning patterns

How it works

When you run the DevServer with remote endpoints configured:

1. **Automatic redirection** - All remote endpoint calls defined in your item manifest are automatically redirected to your local machine
2. **Console logging** - Request details, tokens, and payloads appear in your DevServer console
3. **Sample execution** - Stub implementations execute showing best practices
4. **Token access** - You receive actual Fabric tokens that can be used to call Fabric APIs and access OneLake

Using remote endpoint stubs

To use the remote endpoint stubs for local testing:

1. **Configure your item manifest** - Define jobs or lifecycle notifications as you would for production (see [Enable Remote Jobs](#) and [Enable Item Lifecycle Notifications](#))
2. **Start your DevServer** - Launch the development server following the [quickstart guide](#)
3. **Register your local workload** - Use the DevGateway to register your workload with Fabric
4. **Trigger operations in Fabric** - Perform actions that trigger jobs or lifecycle notifications:
 - Create, update, or delete items for lifecycle notifications
 - Trigger jobs manually or via schedules
5. **Observe stub execution** - Watch your DevServer console for:
 - Incoming request details
 - Authentication token information
 - OneLake access examples
 - Job execution patterns

Example console output

When a job is triggered, you see output like:

```
🔔 Job execution request received
Job Type: MyWorkload.DataProcessor.RefreshData
Instance ID: abc123-def456
Workspace ID: 11111111-2222-3333-4444-555555555555
Item ID: aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee

🔑 Authentication tokens received
Subject Token: eyJ0eXAiOiJKV1QiLCJhbGc...
App Token: eyJ0eXAiOiJKV1QiLCJhbGc...
```

Tenant ID: bbbbcccc-1111-dddd-2222-eeee3333ffff



OneLake access example

Getting OneLake token for scope: <https://storage.azure.com/.default>

Reading from: `/workspaces/{workspaceId}/items/{itemId}/files/`



Job execution completed

Benefits of local stub testing

- **No Azure deployment required** - Test remote endpoint logic without cloud resources
- **Fast iteration** - See results immediately in your console
- **Token handling examples** - Learn authentication patterns with real tokens
- **OneLake integration** - Test data access patterns safely
- **Sample code** - See working implementations to guide your production code

Moving to production

The stub implementations provide sample code you can adapt for production:

1. Review the stub implementation in your DevServer codebase
2. Implement production endpoints using the patterns demonstrated
3. Deploy endpoints to Azure Functions, Container Instances, or other hosting
4. Update manifest configuration to point to production endpoints (see [Enable Remote Endpoints](#))

The stubs use the same authentication patterns and API contracts as production, making the transition straightforward.

See also

- [DevGateway](#)
 - [Enable Remote Endpoints](#)
 - [Enable Remote Jobs](#)
 - [Enable Item Lifecycle Notifications](#)
 - [Authenticate Remote Endpoints](#)
 - [Workload manifest](#)
 - [Product manifest](#)
 - [Item manifest](#)
 - [Manifest overview](#)
-

Microsoft Fabric Extensibility Toolkit Validator

The Microsoft Fabric Extensibility Toolkit Validator is a comprehensive validation tool for Microsoft Fabric workloads. It helps publishers validate their workloads against Microsoft Fabric requirements before submitting for official certification.

This tool is the **public validation tool** for external partners and customers developing Fabric workloads.

Overview

The validation tool allows workload creators to self-validate their Microsoft Fabric workloads. It's designed to help identify potential issues early in the development process, making the official validation process more likely to succeed.

Important

Successful self-validation is no guarantee for passing official validation, but it significantly increases the likelihood of success by catching common issues early.

Prerequisites

- **Node.js** (version 14 or higher)
- **Microsoft Fabric Account** with access to Fabric Workload Hub
- **Chrome/Chromium Browser** (for automated manifest download)
- **Workload Package** built using the [Microsoft Fabric Extensibility Toolkit](#) 
- **Published Workload** - Your workload must be published to a tenant before validation can begin.

Tip

Use a different tenant for validation than the one used for publishing. This allows you to experience what end users will see and provides a more realistic validation environment.

Get started

1. Build your workload

First, create your workload using the official starter kit:

1. Clone the [Fabric Extensibility Toolkit \(starter kit\)](#) [↗].
2. Follow the toolkit documentation to build your workload.

2. Install dependencies

Navigate to the validator directory and install the dependencies:

Bash

```
cd fabric-extensibility-toolkit-validator/validator
npm install
```

3. Basic validation

Run a complete validation for your workload:

Basic validation command

Bash

```
node index.js --workload-name "YourPublisher.YourWorkload" --workload-stage  
"Preview"
```

Example with real workload

Bash

```
node index.js --workload-name "Contoso.DataProcessor" --workload-stage "Preview"
```

For GA stage workloads

Bash

```
node index.js --workload-name "Contoso.DataProcessor" --workload-stage "GA"
```

ⓘ Note

Make sure you're logged into Microsoft Fabric in your browser before running validation, and ensure your workload is published and accessible in the target tenant.

Command line options

 Expand table

Option	Description	Example
<code>--workload-name</code>	Full workload name (Publisher.Product)	<code>"Contoso.DataProcessor"</code>
<code>--workload-stage</code>	Validation stage	<code>"Preview"</code> or <code>"GA"</code>
<code>--help</code>	Show help information	<i>(flag only)</i>

Validation process

The validator runs through several stages:

1. **Manifest Download** - Automatically downloads your workload manifest from Fabric Workload Hub.
2. **Test Discovery** - Identifies applicable test cases based on your workload type.
3. **Test Execution** - Runs validation tests using the configured validators (both automated and manual).
4. **Report Generation** - Creates comprehensive reports in multiple formats.

Output structure

After validation, you'll find results in the `Results` directory:

text

```
Results/
├── YourPublisher.YourWorkload/
│   ├── Preview/
│   │   └── [ValidationID]/
│   │       ├── YourPublisher.YourWorkload_Metadata.json
│   │       ├── YourPublisher.YourWorkload_Manifest.json
│   │       ├── YourPublisher.YourWorkload_Workload_Tests.json
│   │       ├── YourPublisher.YourWorkload.Item_Item_Tests.json
│   │       └── Reports/
│   │           ├── ValidationReport.md
│   │           ├── ValidationReport.html
│   │           └── ValidationReport.pdf
```

Validators

The tool uses a set of specialized validators to check different aspects of your workload. Some validators are automated, while others require manual interaction.

Troubleshooting

Browser issues

- Ensure Chrome/Chromium is installed and accessible.
- Make sure you're logged into Microsoft Fabric in your browser.
- Check network connectivity to Fabric services.

Manifest download issues

- Verify workload name matches exactly (case-sensitive).
- Ensure workload is deployed and accessible in Fabric Workload Hub.
- Check authentication and permissions.

Test execution issues

- Review validation logs in the Results directory.
- Check individual test case documentation in `validator/validators/`.
- Verify all required workload assets are accessible.

Related content

- [Microsoft Fabric Extensibility Toolkit](#) - Official starter kit for building Fabric workloads

Last updated on 12/16/2025

Troubleshooting and Support

Having issues with the Microsoft Fabric Extensibility Toolkit? This guide helps you find the right support channel.

Where to Get Help



GitHub Repository - Toolkit Issues

Use for: Bugs, feature requests, documentation issues, development problems



[Microsoft Fabric Extensibility Toolkit on GitHub](#)

Report issues for:

- DevGateway or DevServer problems
- Authentication errors during development
- Manifest validation failures
- Starter kit or sample issues
- Documentation corrections
- Feature requests for the toolkit



Microsoft Fabric Support - Platform Issues

Use for: Account, service, and production workload issues



[Microsoft Fabric Support](#)

Get help with:

- Account and billing issues
- Workspace permissions and access
- Published workload problems
- Fabric service outages
- Tenant administration
- Security and compliance questions

Quick Decision Guide

Issue Type	Where to Go
Toolkit won't start	GitHub
Can't access workspace	Fabric Support
Sample code not working	GitHub
Billing questions	Fabric Support
Missing documentation	GitHub
Performance in production	Fabric Support
Feature request	GitHub
Account permissions	Fabric Support

Before You Report

1. **Check the documentation** - Review [Getting Started](#) and relevant guides
2. **Search existing issues** - Your problem might already be reported
3. **Gather information** - Error messages, steps to reproduce, environment details

Quick Links

- [Getting Started](#) - Start here for setup help
- [Key Concepts](#) - Understand core concepts
- [Authentication Guide](#) - Authentication troubleshooting
- [Starter Kit](#) [↗](#) - Working examples
- [Community Forums](#) [↗](#) - Community discussions

Last updated on 12/15/2025